

Convolutional Neural Networks

Aashutosh Pyakurel

Content (2 Hours)

- Deep Learning for Computer Vision and Convolutional Neural Networks
- Transfer Learning
- Auto-Encoders and Variational-Auto-Encoders (VAEs)

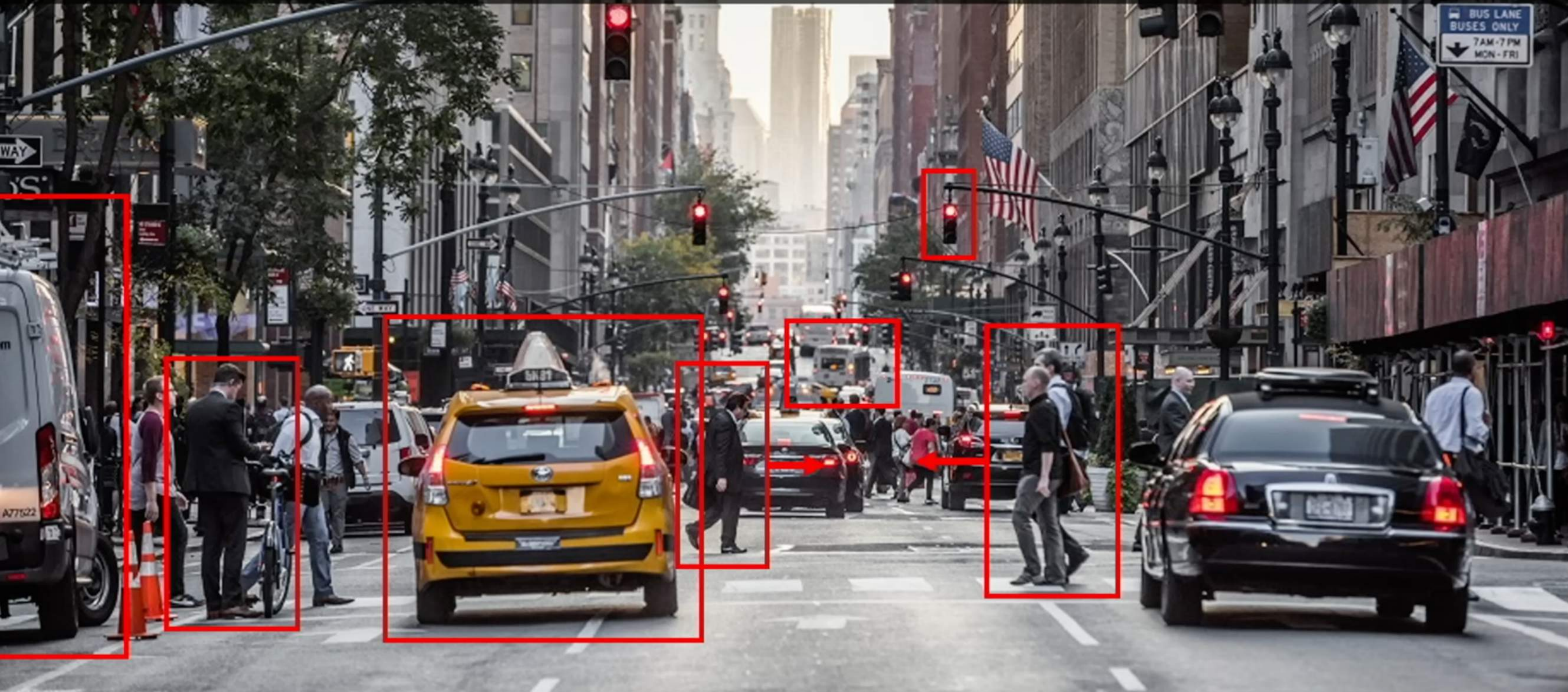
Courtesy

- Slide content from [MIT 6.S191](#) and Fusemachines Academics Department
- Free graduate level Deep Learning course by Philipp Krähenbühl
http://www.philkr.net/dl_class/
- Papers listed in Deep Learning Seminar course <http://www.philkr.net/cs395t/>



**“To know what is
where by looking.”**

To discover from images what is present in the world, where things are, what actions are taking place, to predict and anticipate events in the world



The rise and impact of computer vision

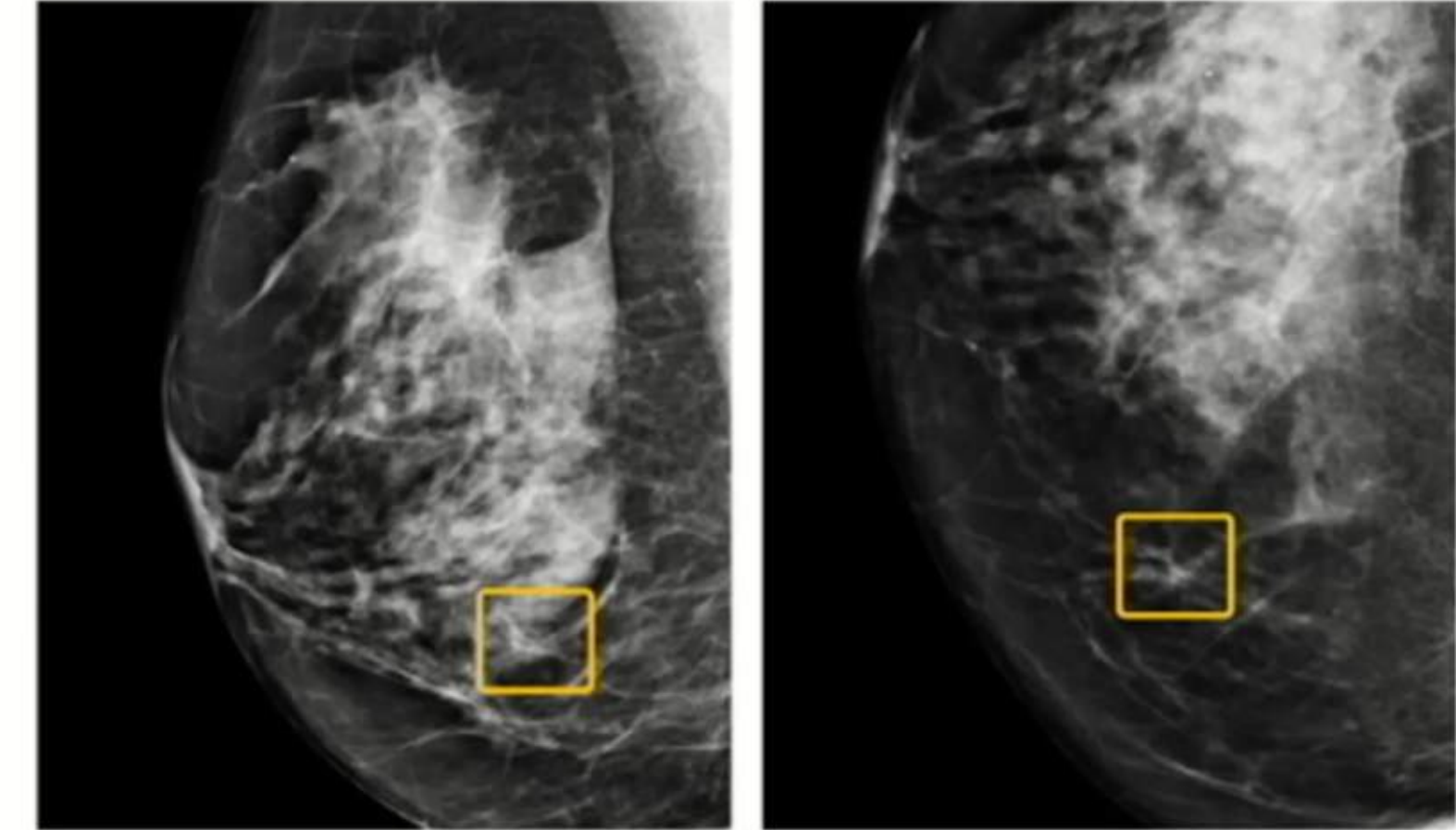
Robotics



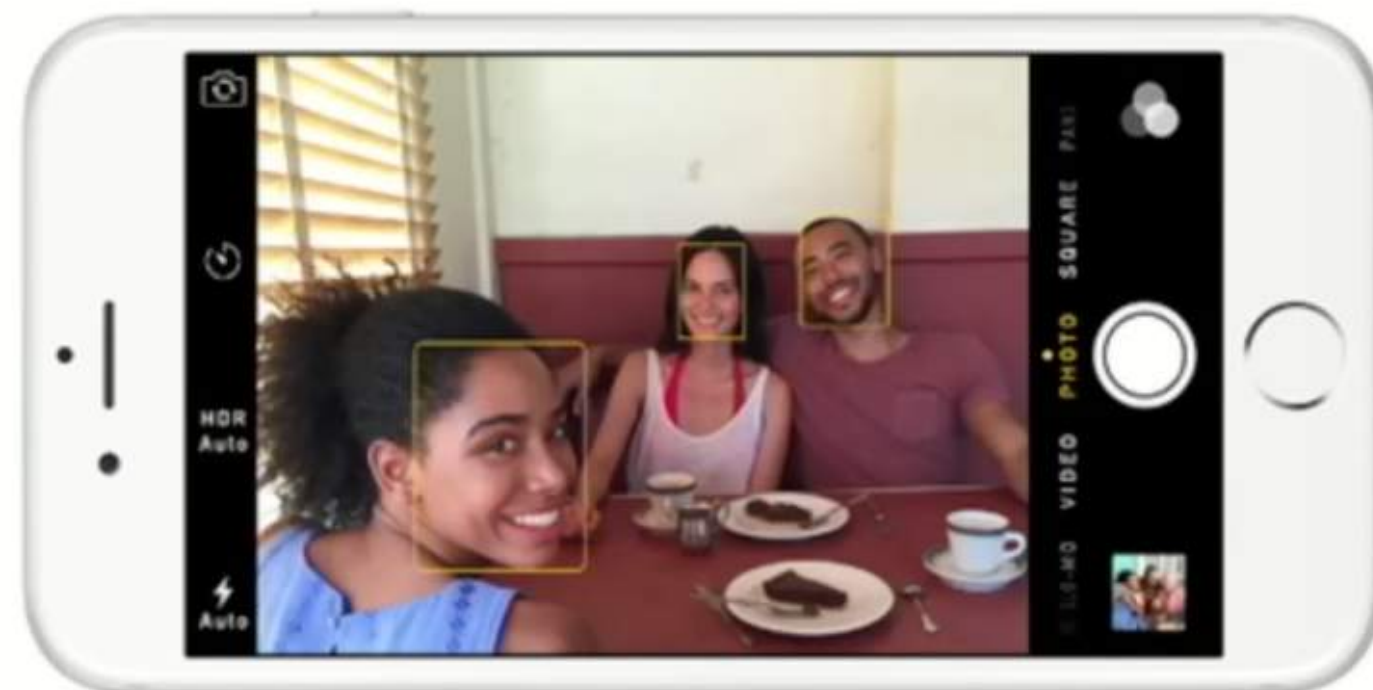
Accessibility



Biology & Medicine

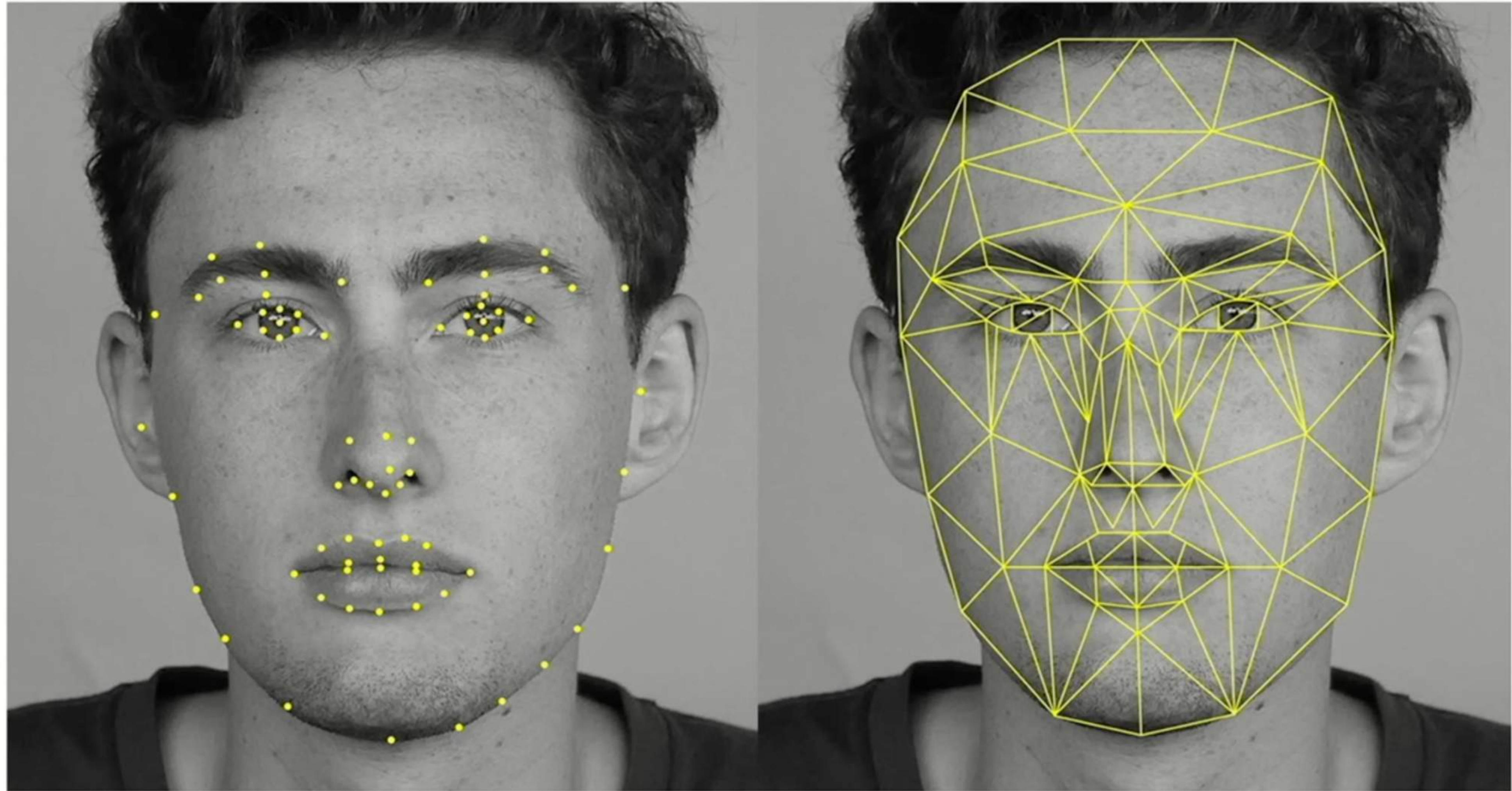
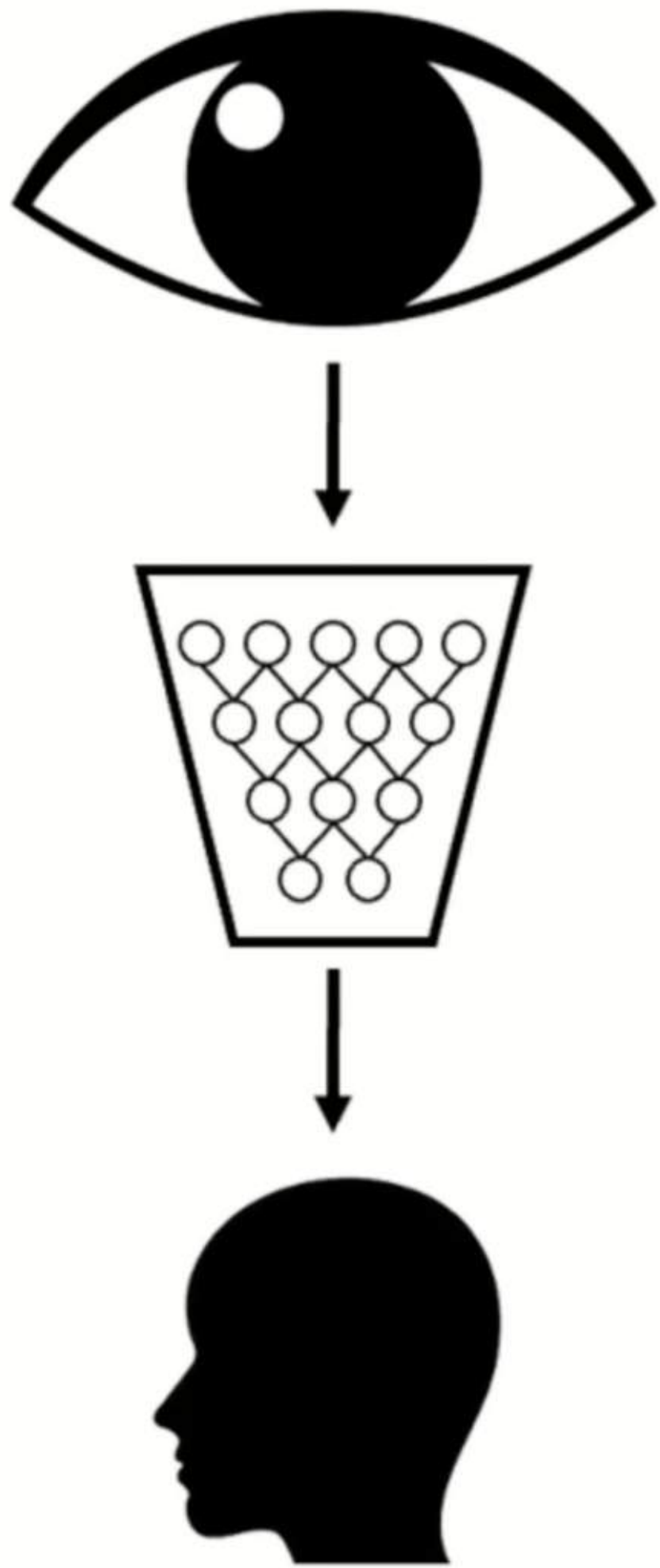


Autonomous driving

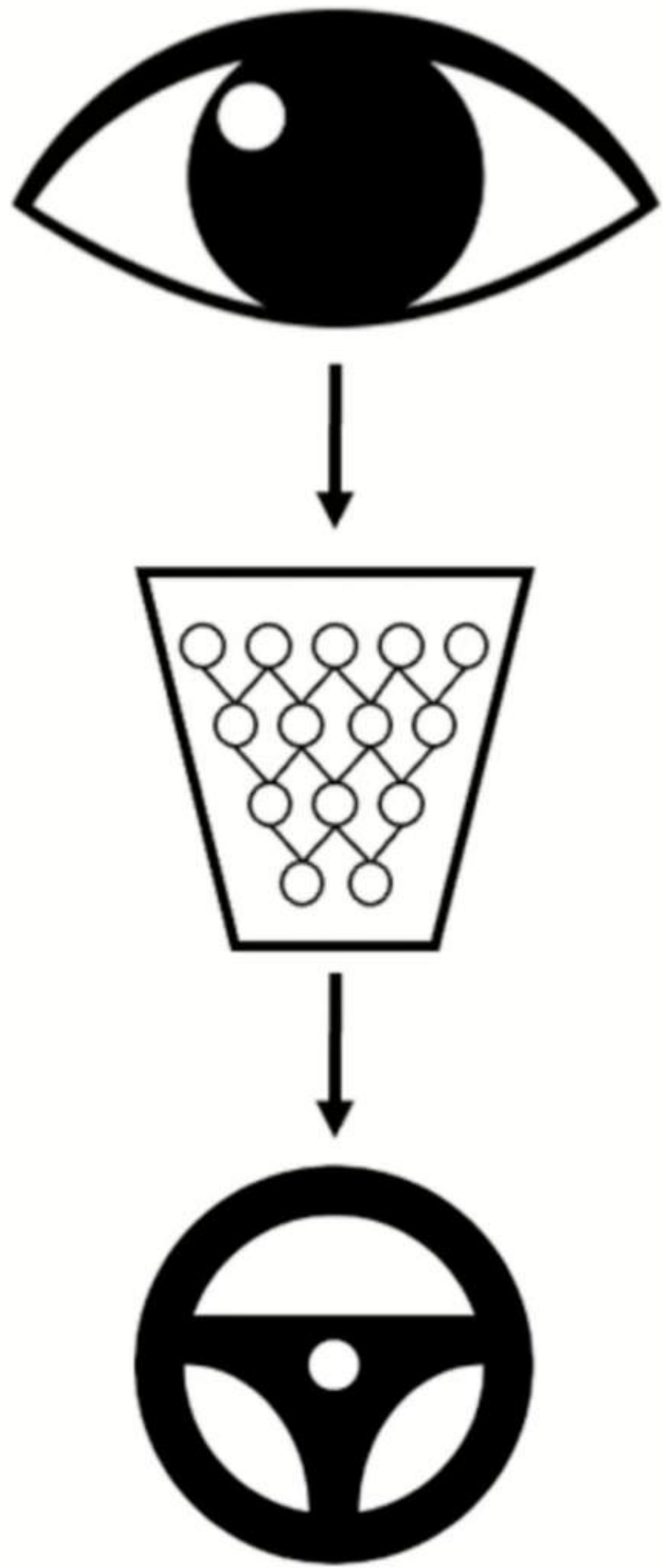


Mobile computing

Impact: Facial Detection & Recognition



Impact: Self-Driving Cars



What computer sees?

Images are Numbers



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	105	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

What the computer sees

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	105	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

An image is just a matrix of numbers $[0,255]$!
i.e., $1080 \times 1080 \times 3$ for an RGB image

Tasks in Computer Vision



Input Image



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Pixel Representation



classification

Lincoln

Washington

Jefferson

Obama

$$\begin{bmatrix} 0.8 \\ 0.1 \\ 0.05 \\ 0.05 \end{bmatrix}$$

- **Regression:** output variable takes continuous value
- **Classification:** output variable takes class label. Can produce probability of belonging to a particular class

High Level Feature Detection

Let's identify key features in each image category



Nose,
Eyes,
Mouth

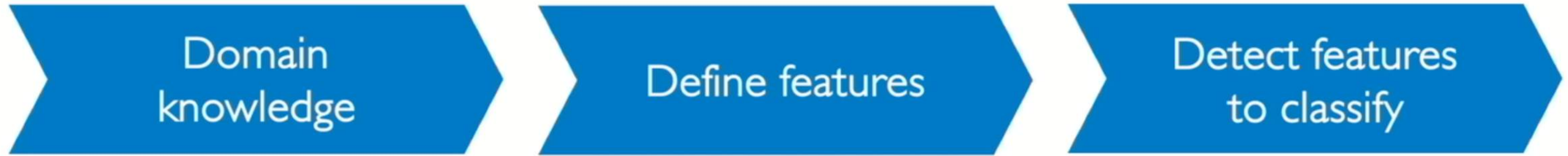


Wheels,
License Plate,
Headlights



Door,
Windows,
Steps

Manual Feature Extraction



Problems?

Manual Feature Extraction

Domain
knowledge

Define features

Detect features
to classify

Viewpoint variation



Scale variation



Deformation



Occlusion



Illumination conditions



Background clutter



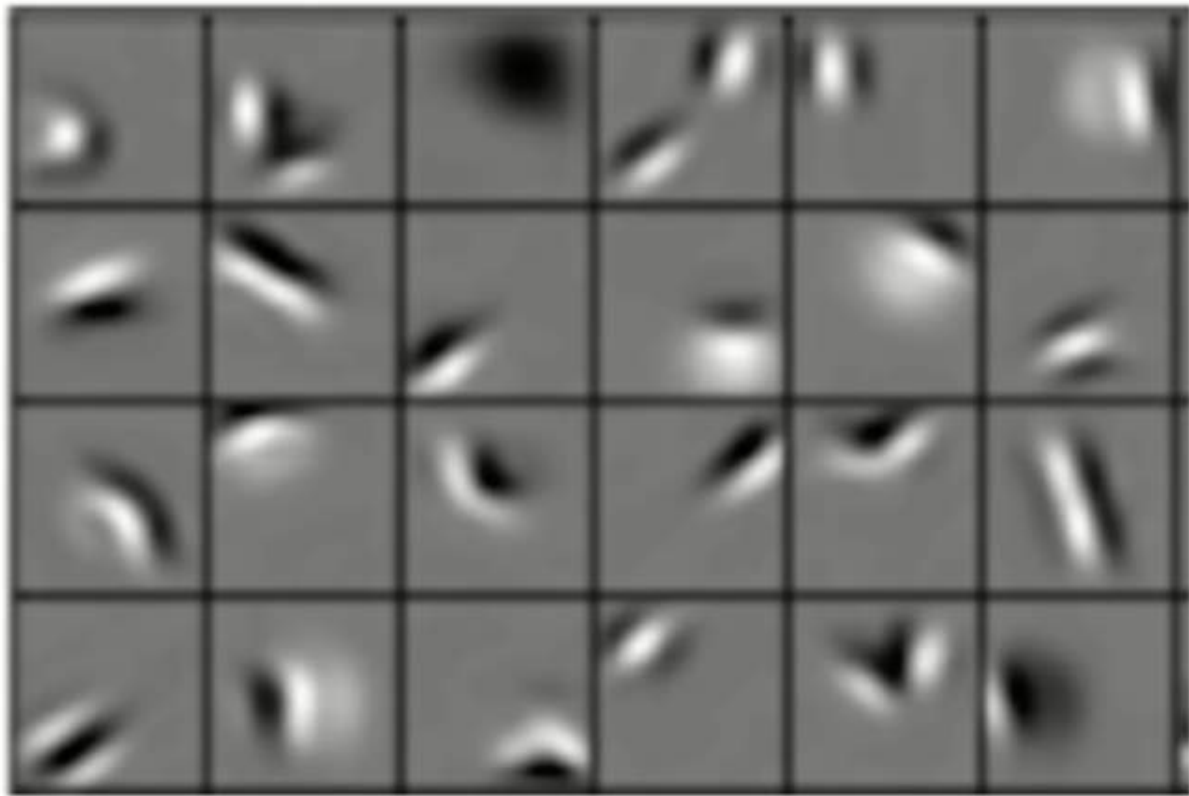
Intra-class variation



Learning Feature Representations

Can we learn a **hierarchy of features** directly from the data instead of hand engineering?

Low level features



Edges, dark spots

Mid level features



Eyes, ears, nose

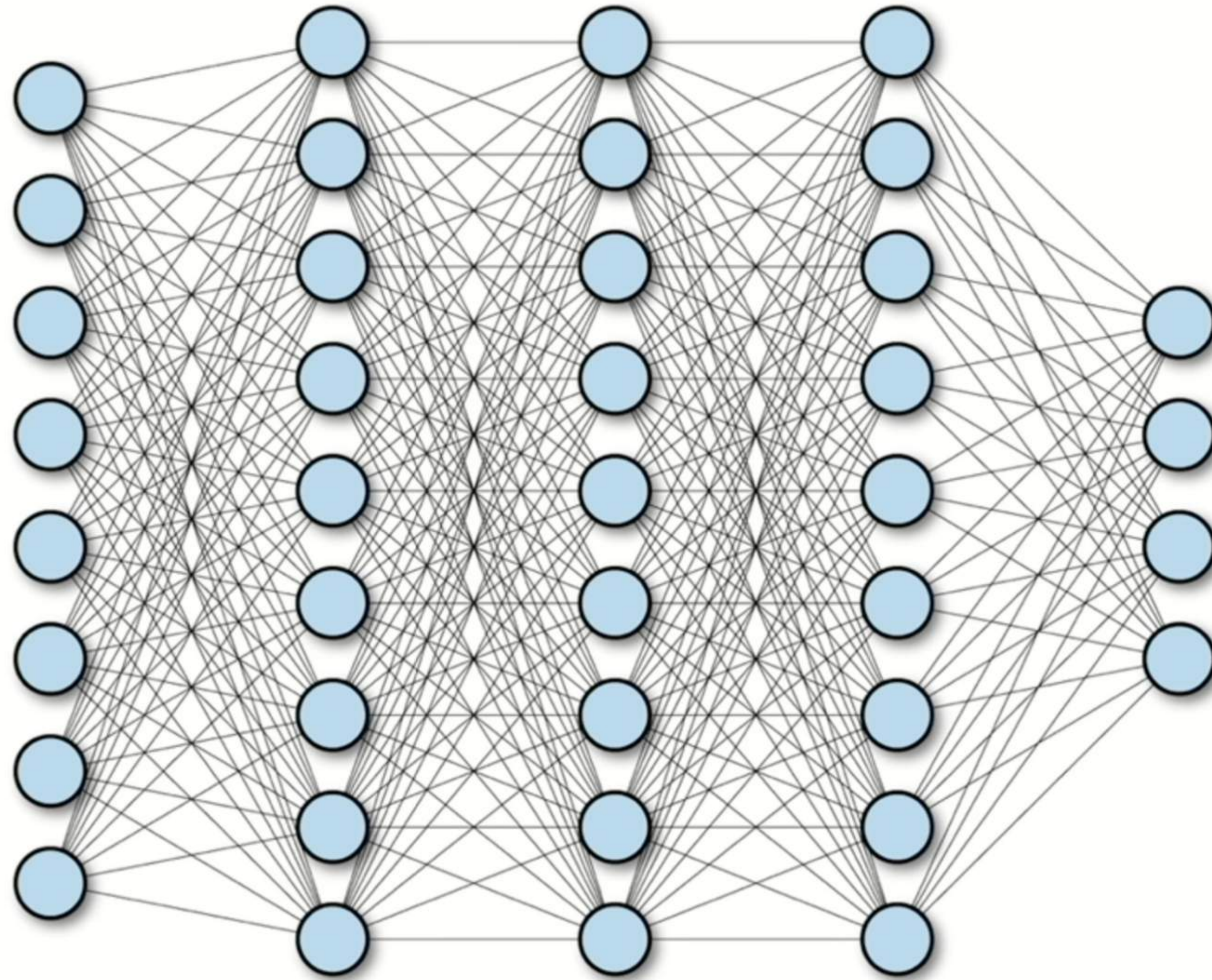
High level features



Facial structure

Learning Visual Features

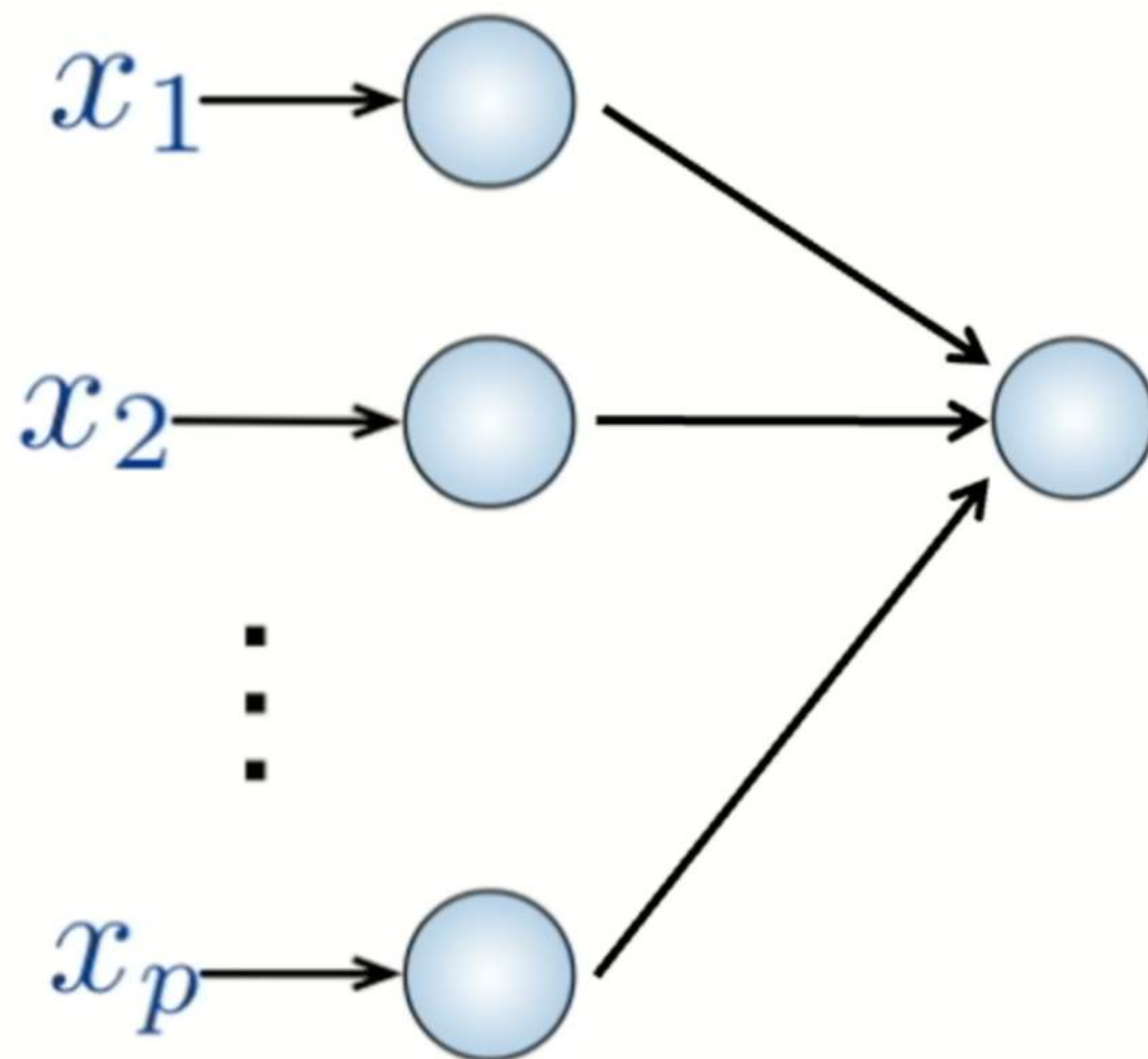
Fully Connected Neural Network



Fully Connected Neural Network

Input:

- 2D image
- Vector of pixel values



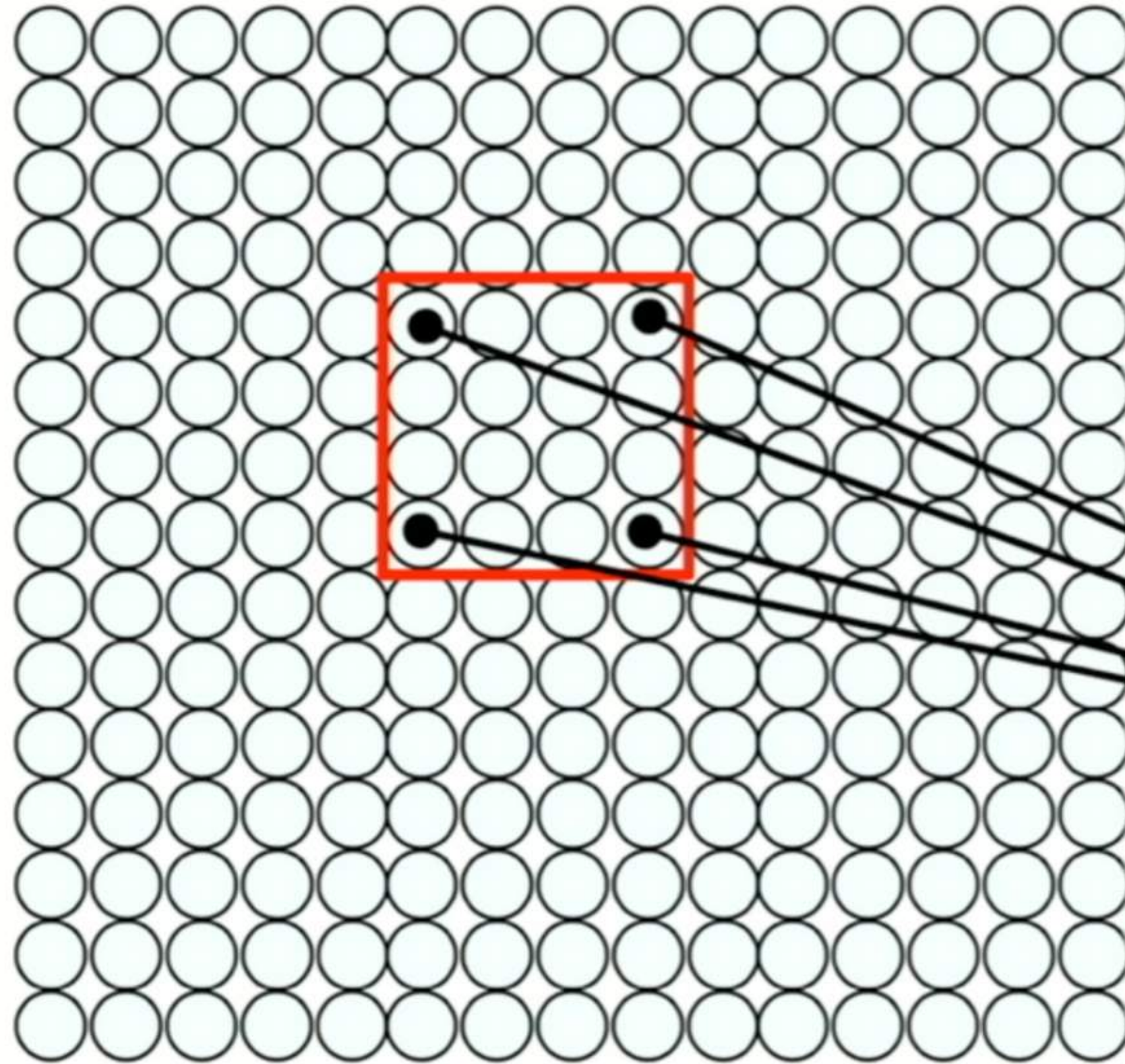
Fully Connected:

- Connect neuron in hidden layer to all neurons in input layer
- No spatial information!
- And many, many parameters!

How can we use **spatial structure** in the input to inform the architecture of the network?

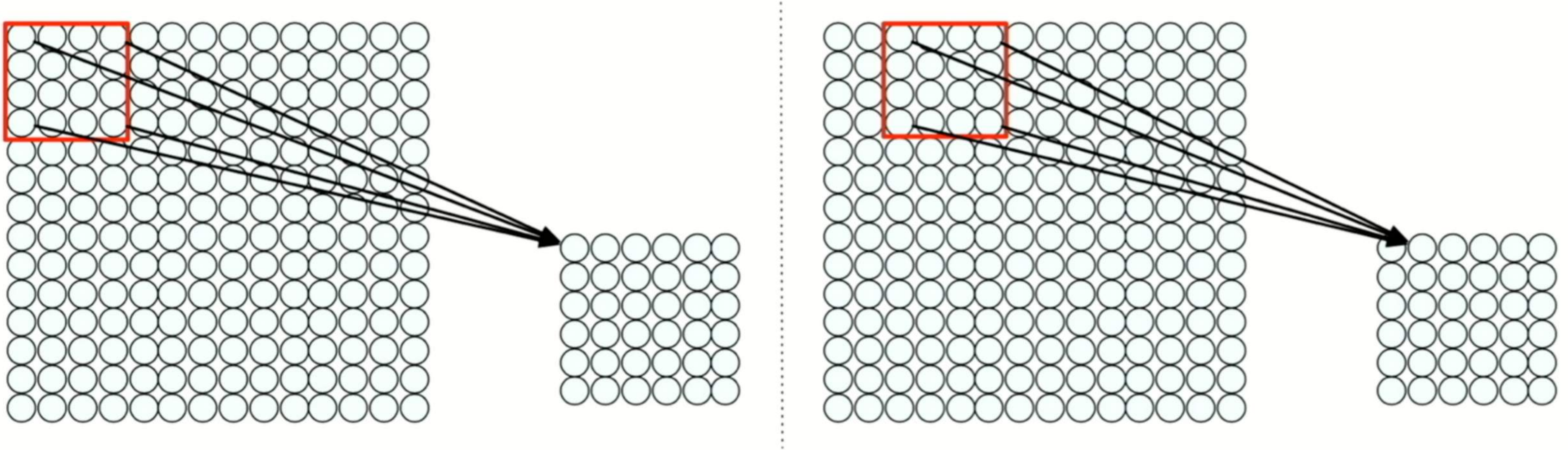
Using Spatial Structure

Input: 2D image.
Array of pixel values



Idea: connect patches of input
to neurons in hidden layer.
Neuron connected to region of
input. Only "sees" these values.

Using Spatial Structure

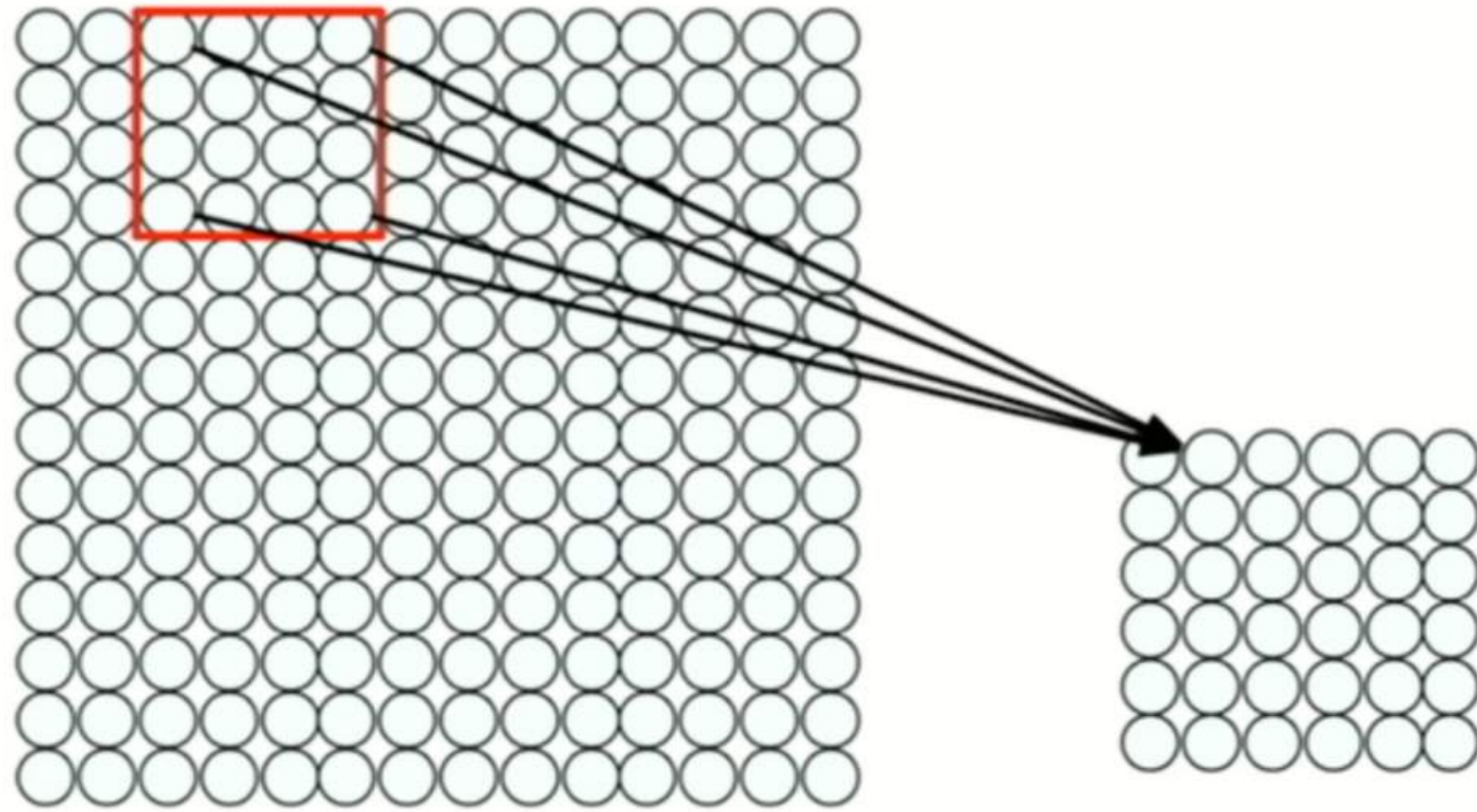


Connect patch in input layer to a single neuron in subsequent layer.

Use a sliding window to define connections.

How can we **weight** the patch to detect particular features?

Feature Extraction with Convolution



- Filter of size 4x4 : 16 different weights
- Apply this same filter to 4x4 patches in input
- Shift by 2 pixels for next patch

This “patchy” operation is **convolution**

- 1) Apply a set of weights – a filter – to extract **local features**
- 2) Use **multiple filters** to extract different features
- 3) **Spatially share** parameters of each filter

Feature Extraction with Convolutions

X or X?

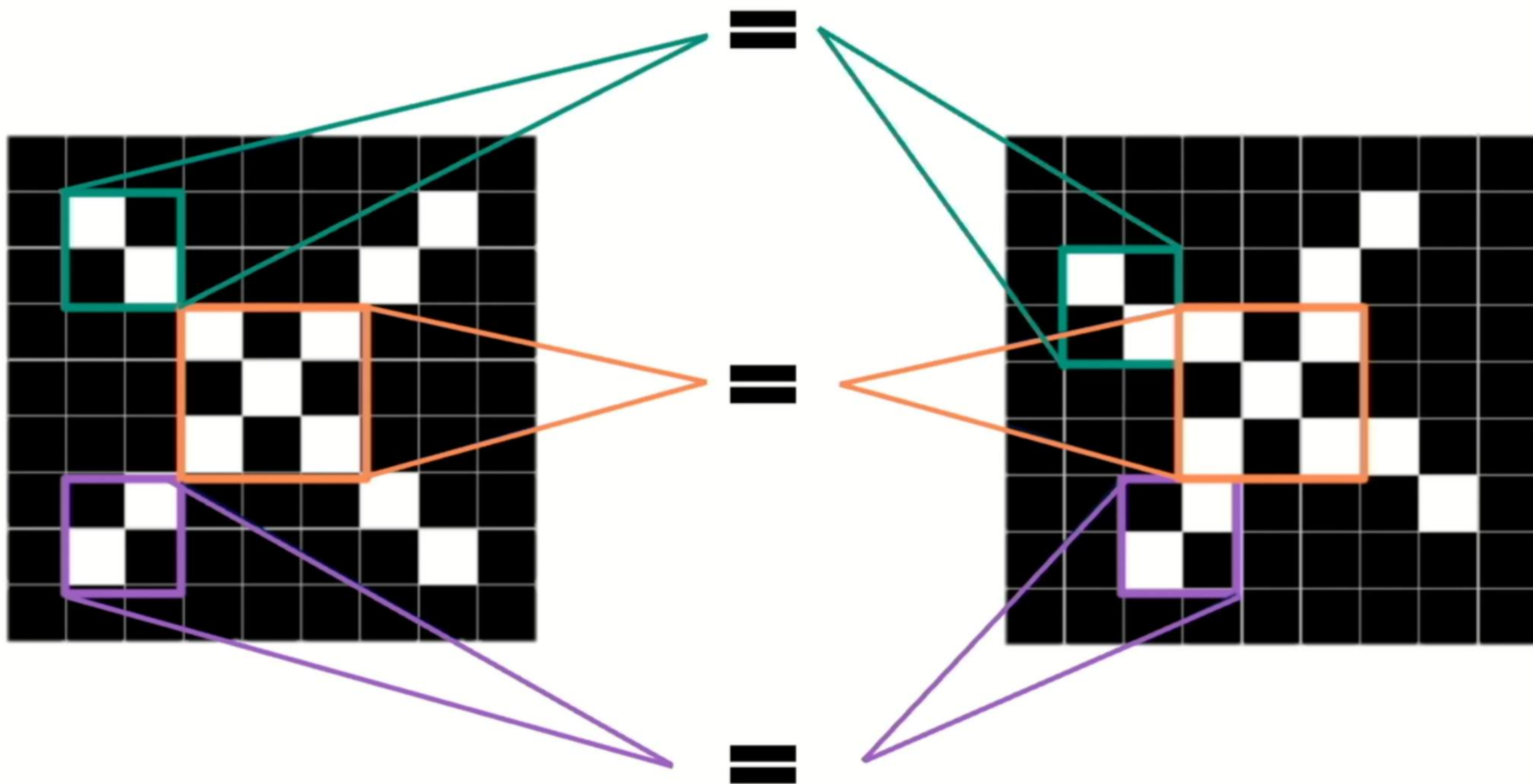
?

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	1	-1	-1	-1
-1	-1	1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	1	-1	-1
-1	-1	-1	1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

Image is represented as matrix of pixel values... and computers are literal!
We want to be able to classify an X as an X even if it's shifted, shrunk, rotated, deformed.

Features of X



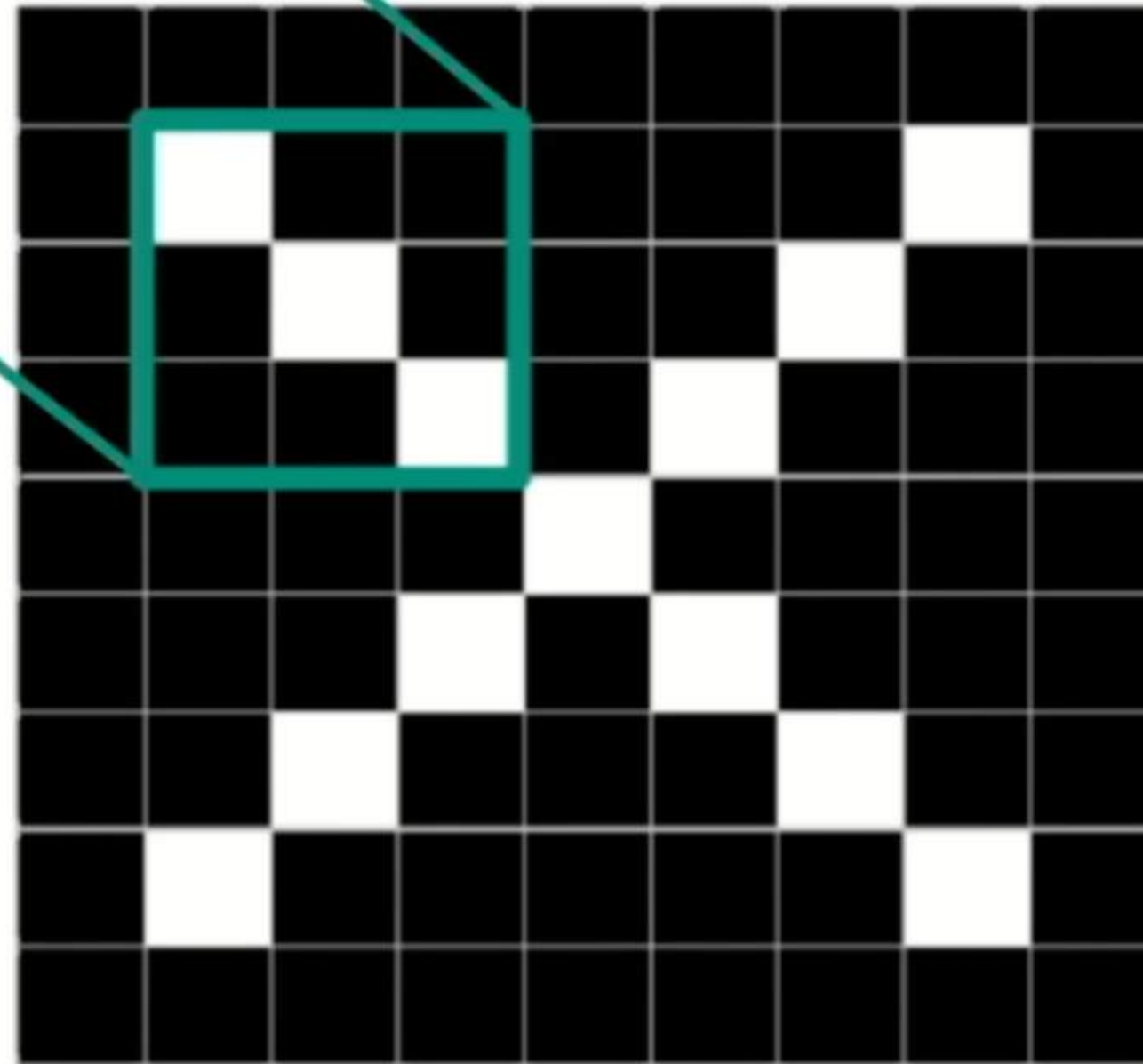
Filters to Detect X Features

filters

1	-1	-1
-1	1	-1
-1	-1	1

1	-1	1
-1	1	-1
1	-1	1

-1	-1	1
-1	1	-1
1	-1	-1



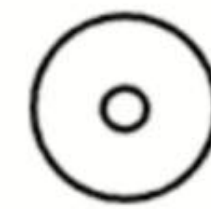
The Convolution Operation

$$\begin{bmatrix} 1 & -1 & -1 \\ -1 & 1 & -1 \\ -1 & -1 & 1 \end{bmatrix} \times \begin{bmatrix} -1 & -1 & -1 \\ 1 & -1 & -1 \\ -1 & -1 & 1 \end{bmatrix} = 1$$

element wise
multiply

add outputs

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

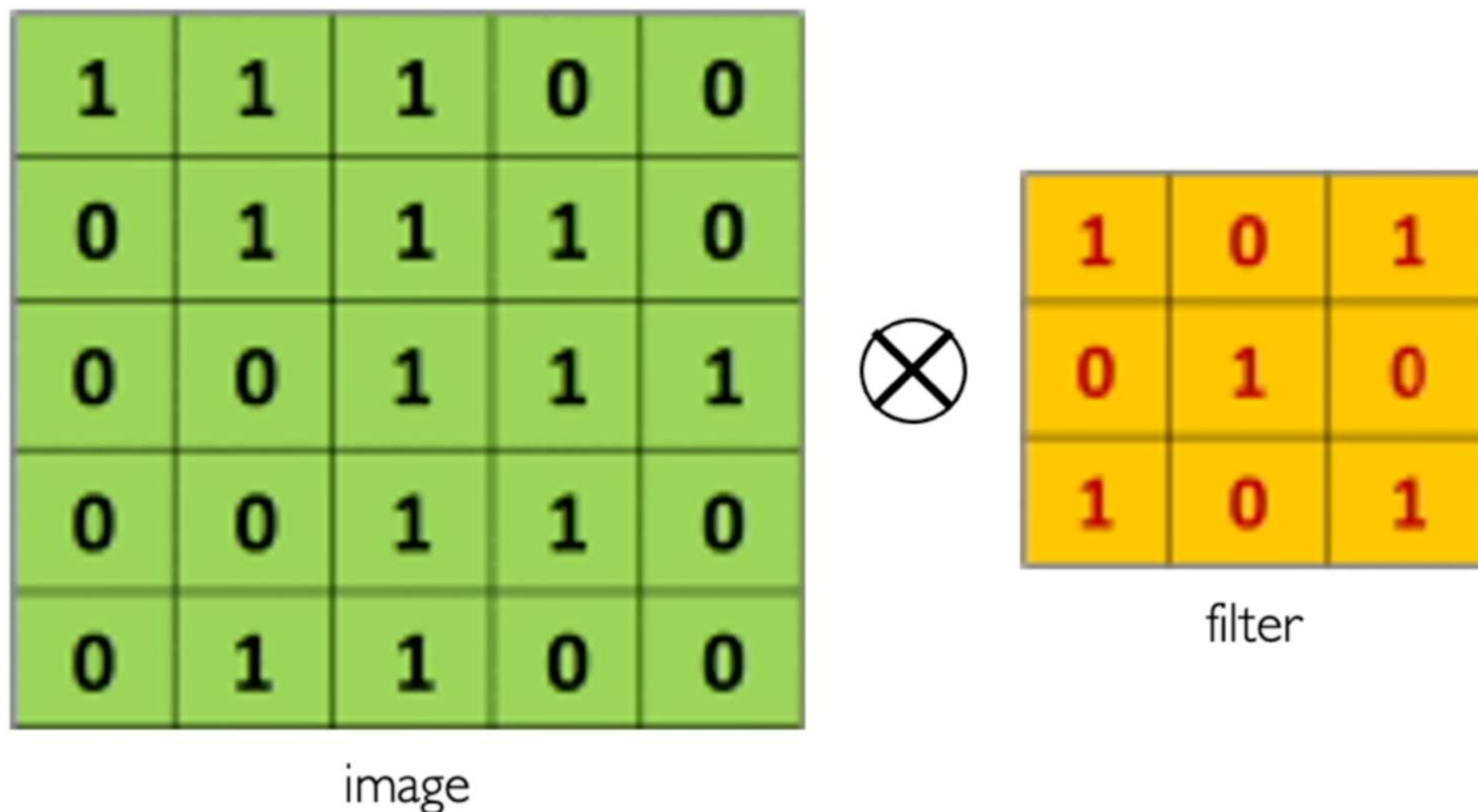


1	1	1
1	1	1
1	1	1

$$= 9$$

The Convolution Operation

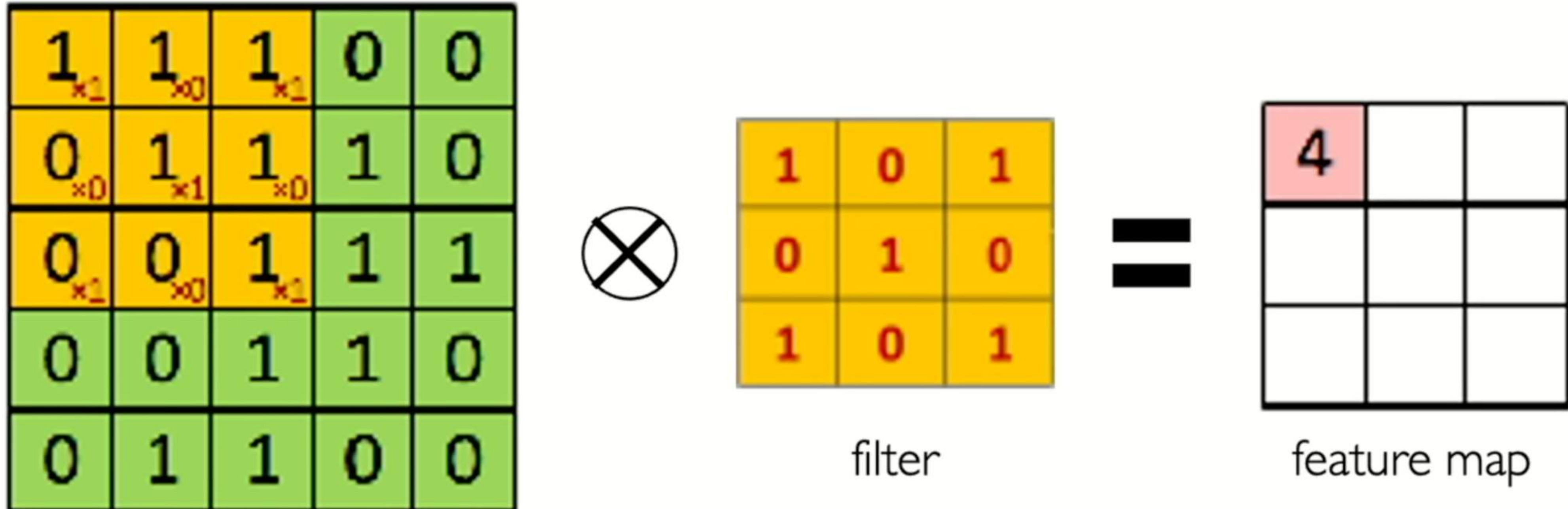
Suppose we want to compute the convolution of a 5x5 image and a 3x3 filter:



We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs...

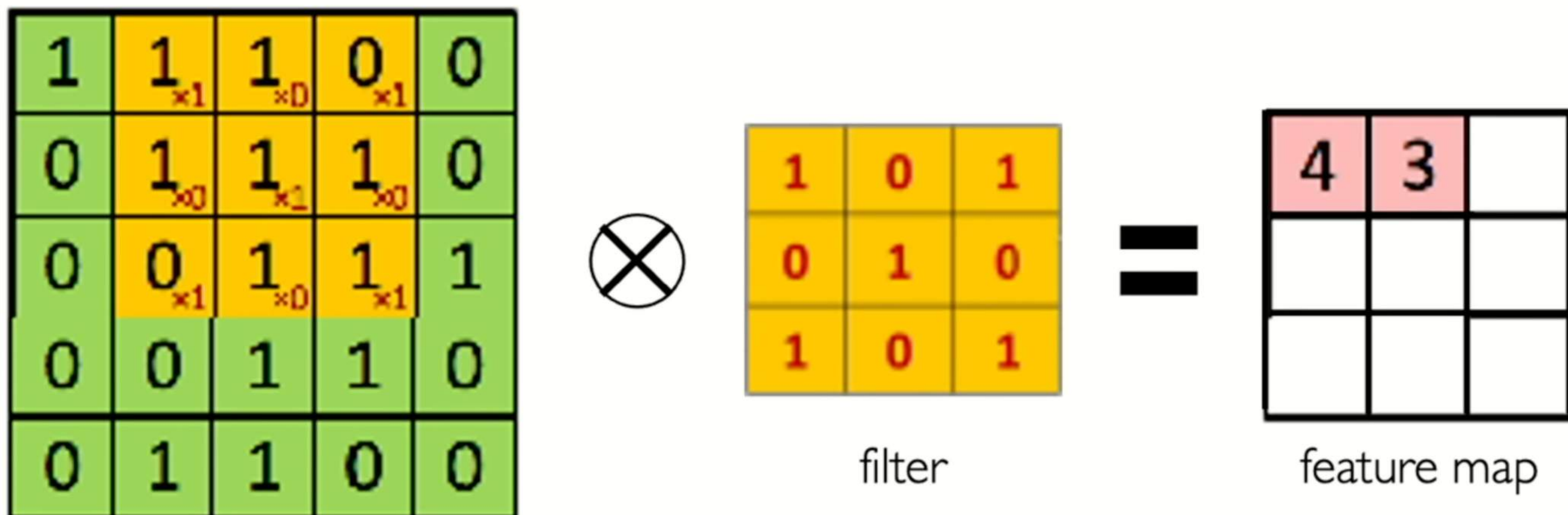
The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



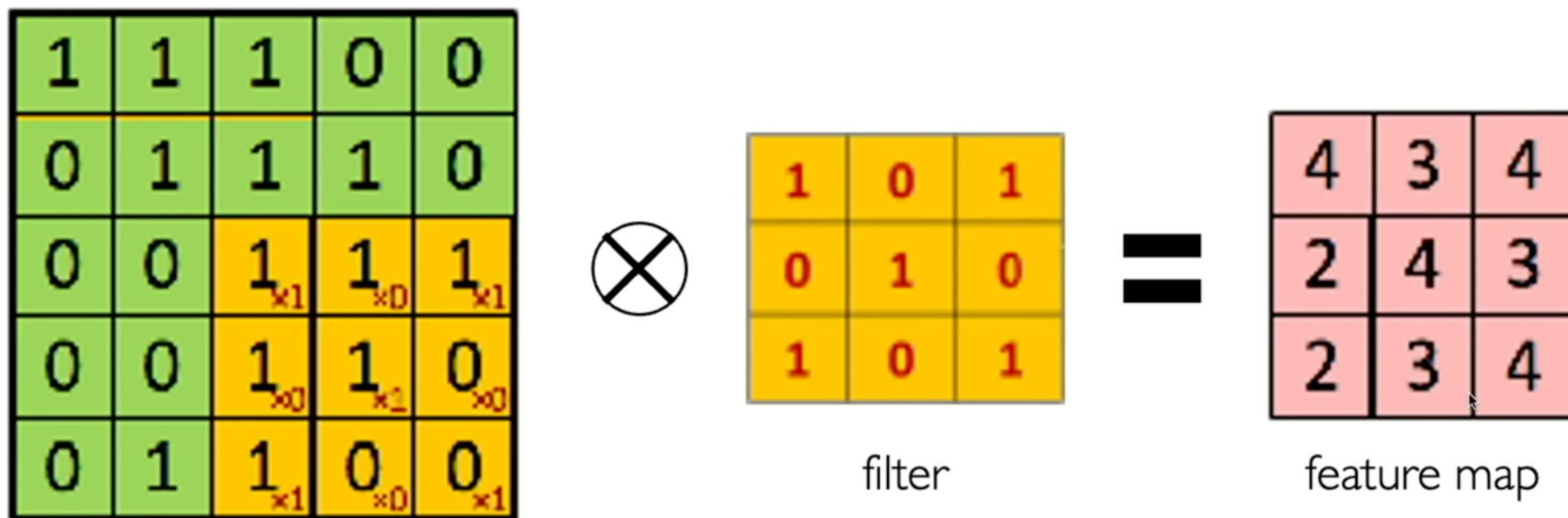
The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



Producing Feature Maps



Original



Sharpen

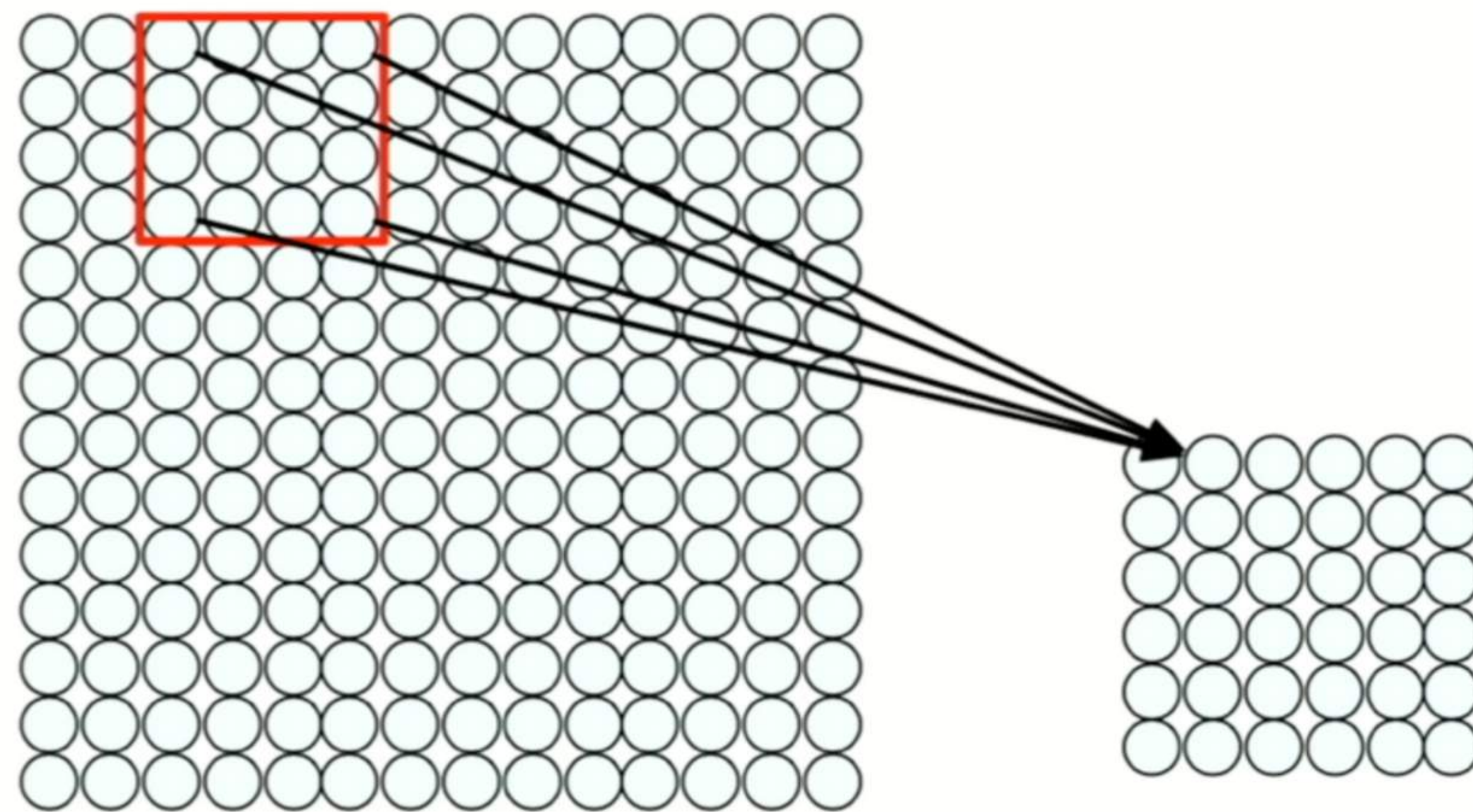


Edge Detect



"Strong" Edge
Detect

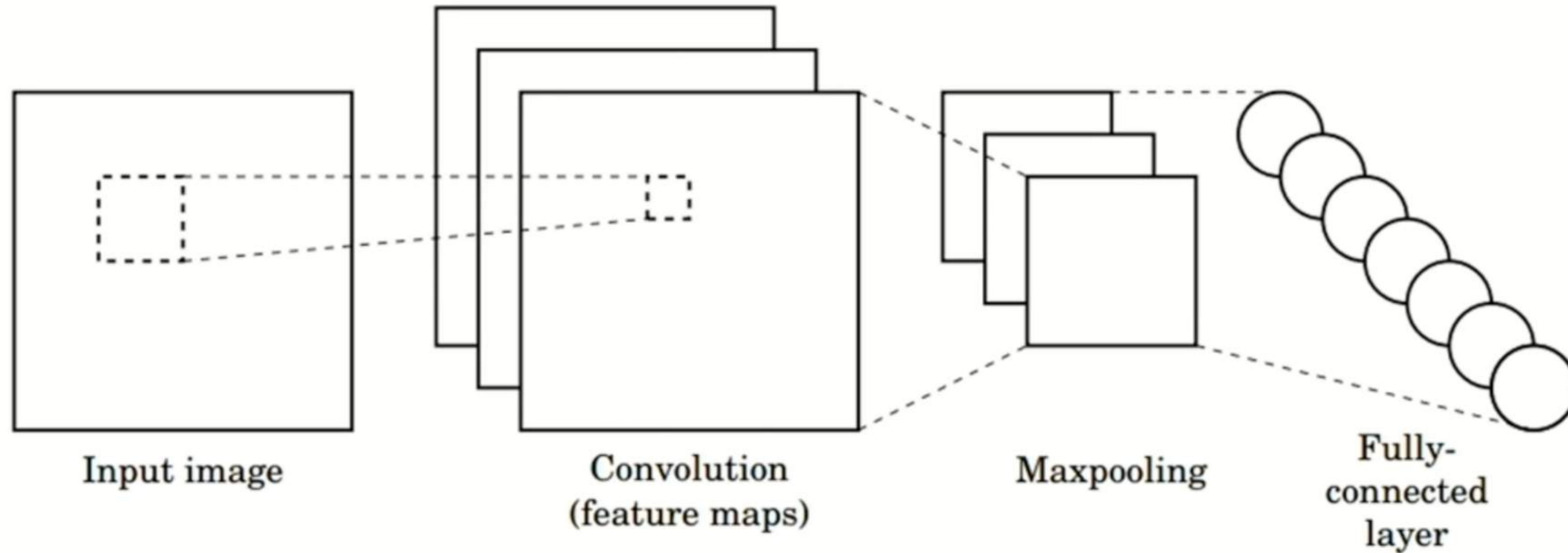
Feature Extraction with Convolution



- 1) Apply a set of weights – a filter – to extract **local features**
- 2) Use **multiple filters** to extract different features
- 3) **Spatially share** parameters of each filter

Convolutional Neural Networks

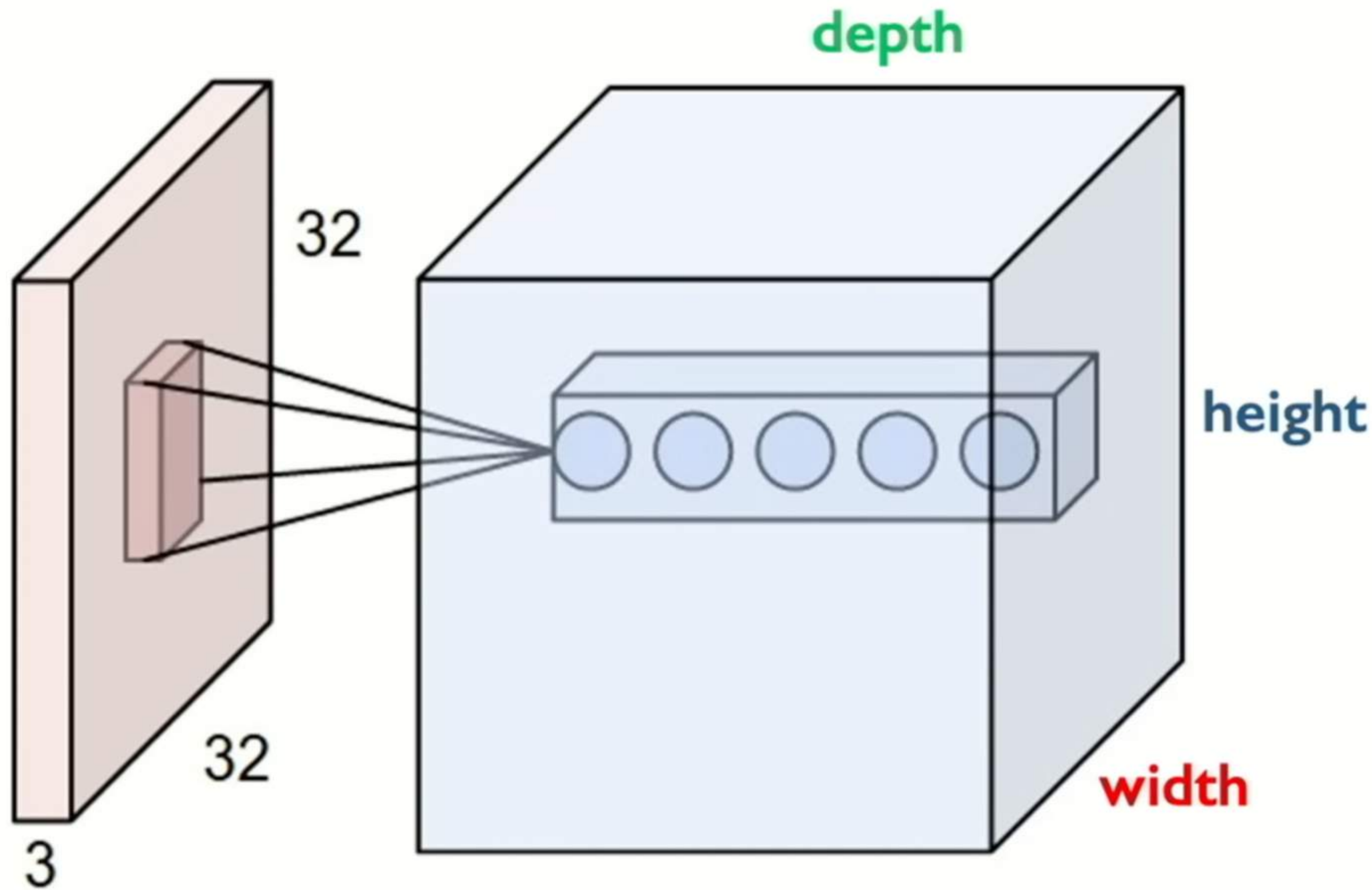
CNNs for Classification



- 1. Convolution:** Apply filters to generate feature maps.
- 2. Non-linearity:** Often ReLU.
- 3. Pooling:** Downsampling operation on each feature map.

Train model with image data.
Learn weights of filters in convolutional layers

CNNs: Spatial Arrangement of Output Volume



Layer Dimensions:

$$h \times w \times d$$

where h and w are spatial dimensions
 d (depth) = number of filters

Stride:

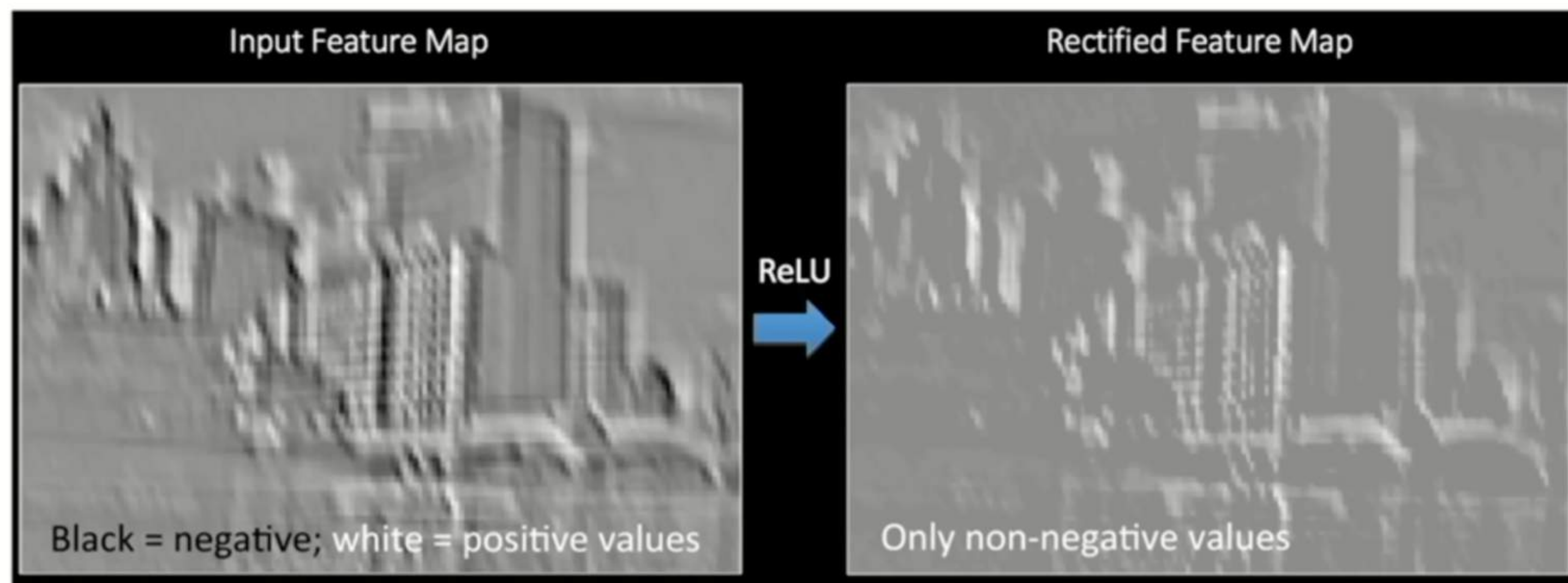
Filter step size

Receptive Field:

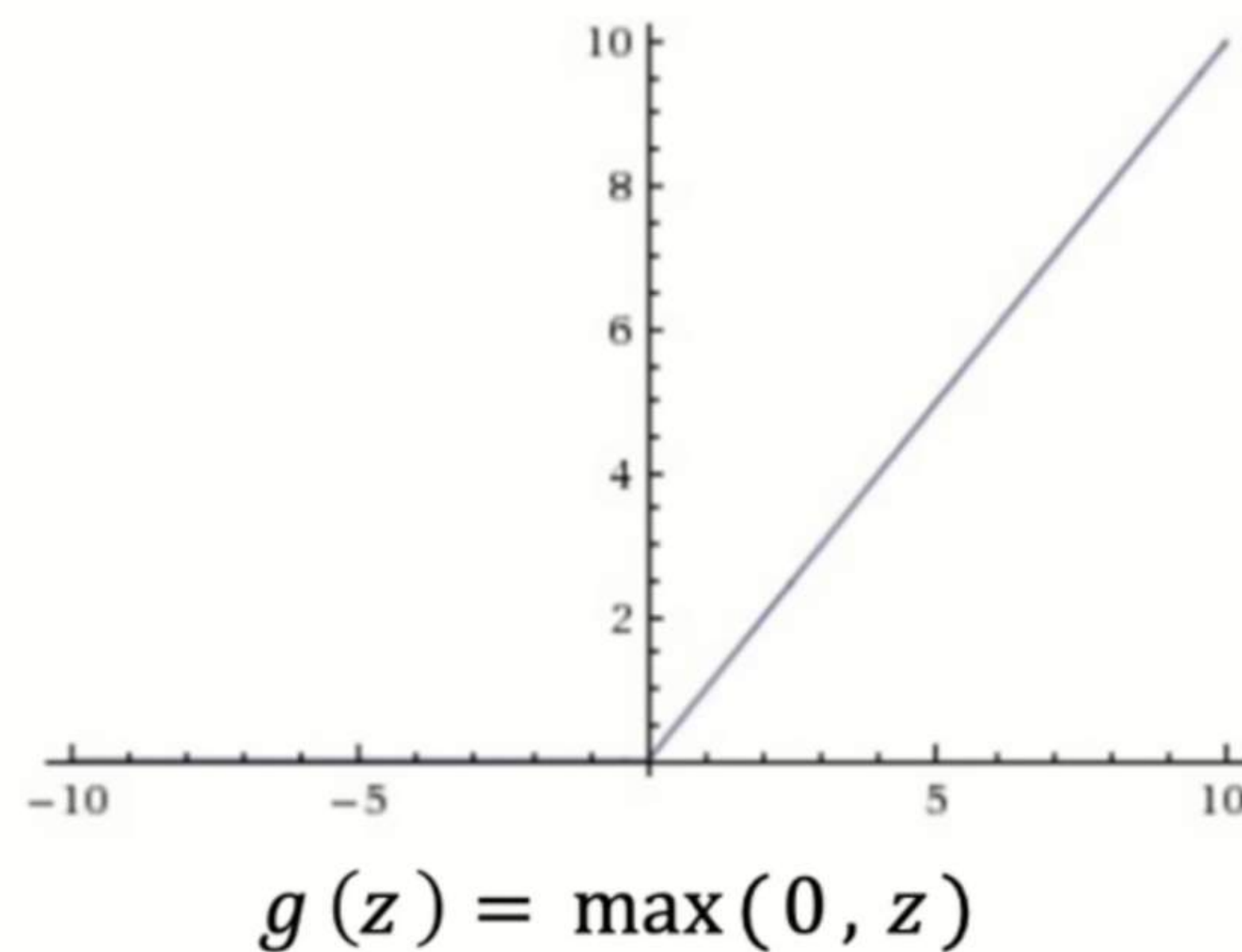
Locations in input image that
a node is path connected to

Introducing Non-Linearity

- Apply after every convolution operation (i.e., after convolutional layers)
- ReLU: pixel-by-pixel operation that replaces all negative values by zero. **Non-linear operation!**



Rectified Linear Unit (ReLU)



Pooling

x ↑

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

→ y

max pool with 2x2 filters
and stride 2

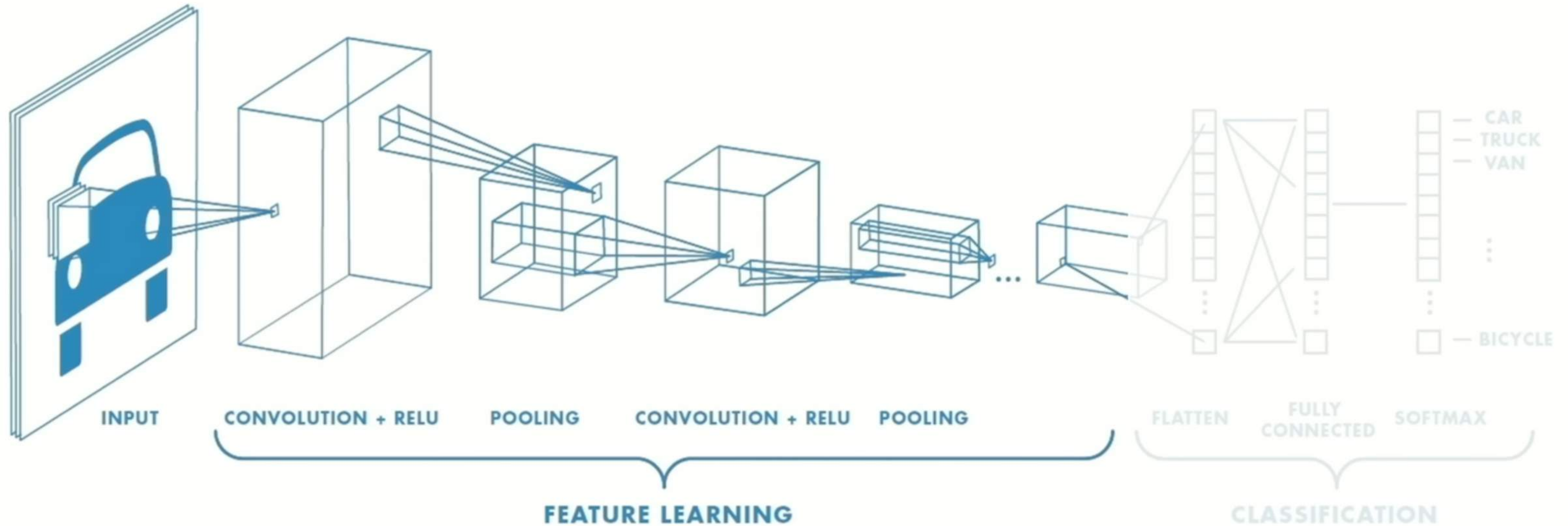


6	8
3	4

- 1) Reduced dimensionality
- 2) Spatial invariance

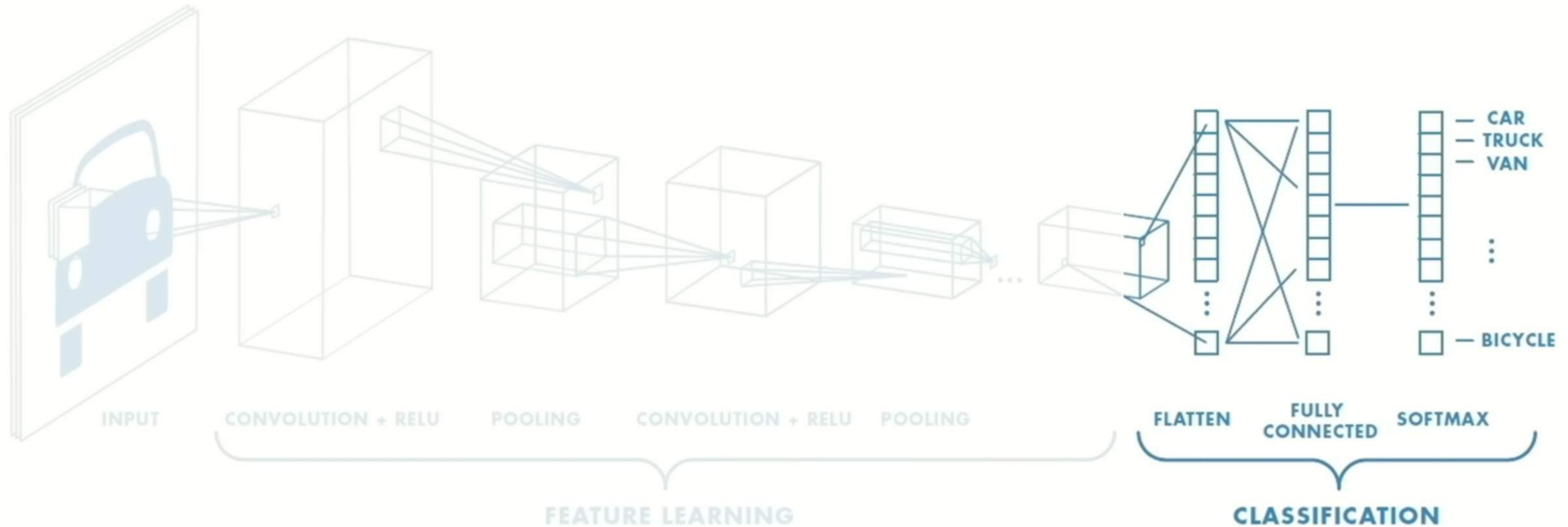
How else can we downsample and preserve spatial invariance?

CNNs for Classification: Feature Learning



1. Learn features in input image through **convolution**
2. Introduce **non-linearity** through activation function (real-world data is non-linear!)
3. Reduce dimensionality and preserve spatial invariance with **pooling**

CNNs for Classification: Class Probabilities



- CONV and POOL layers output high-level features of input
- Fully connected layer uses these features for classifying input image
- Express output as **probability** of image belonging to a particular class

$$\text{softmax}(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

```

class ConvBlock(torch.nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        self.net = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels,
                            out_channels,
                            kernel_size=3,
                            padding=1,
                            stride=stride,
                            bias=False),
            torch.nn.BatchNorm2d(out_channels),
            torch.nn.ReLU(),
            torch.nn.Conv2d(out_channels,
                            out_channels,
                            kernel_size=3,
                            padding=1,
                            stride=1,
                            bias=False),
            torch.nn.BatchNorm2d(out_channels),
            torch.nn.ReLU()
        )

    def forward(self, x):
        return self.net(x)

```

```

def __init__(self, in_channels, n_classes):
    super().__init__()
    L = [
        torch.nn.Conv2d(in_channels, 64, kernel_size=7, padding=3, stride=2, bias=False),
        torch.nn.BatchNorm2d(64),
        torch.nn.ReLU(),
        torch.nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
    ]
    list_channels = [64, 64, 64, 128, 128, 128, 128, 256, 256, 256, 256, 512, 512, 512, 512]
    c0 = 64
    for c1 in list_channels:
        st = 1
        if c0 != c1:
            st = 2
        L.append(self.ConvBlock(c0, c1, st))
        c0 = c1

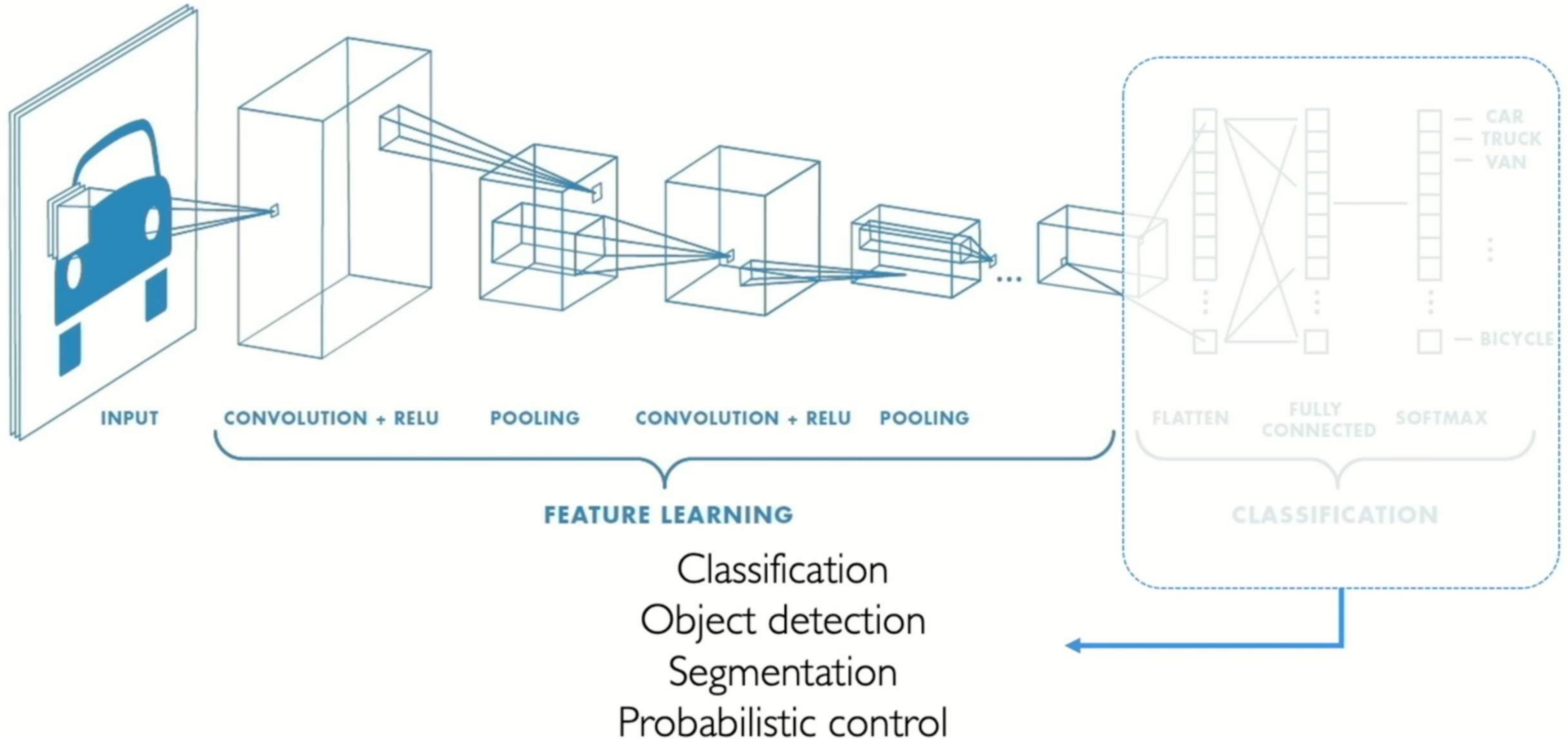
    self.network = torch.nn.Sequential(*L)
    self.classifier = torch.nn.Linear(c0, n_classes)

def forward(self, x):
    y = self.network(x)
    y = y.mean(dim=[2, 3])
    return self.classifier(y)

```

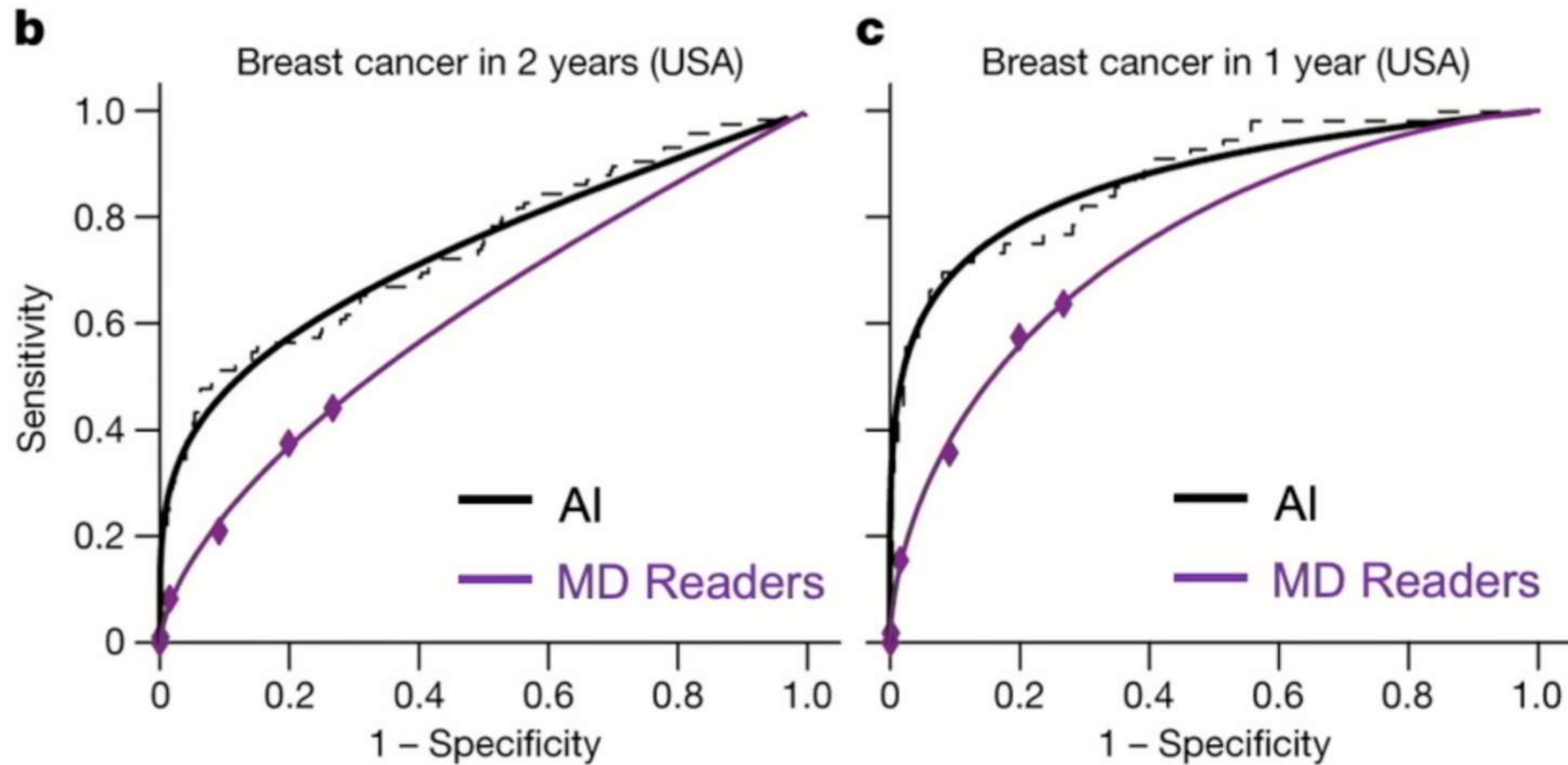
An Architecture for Many Applications

An Architecture for Many Applications

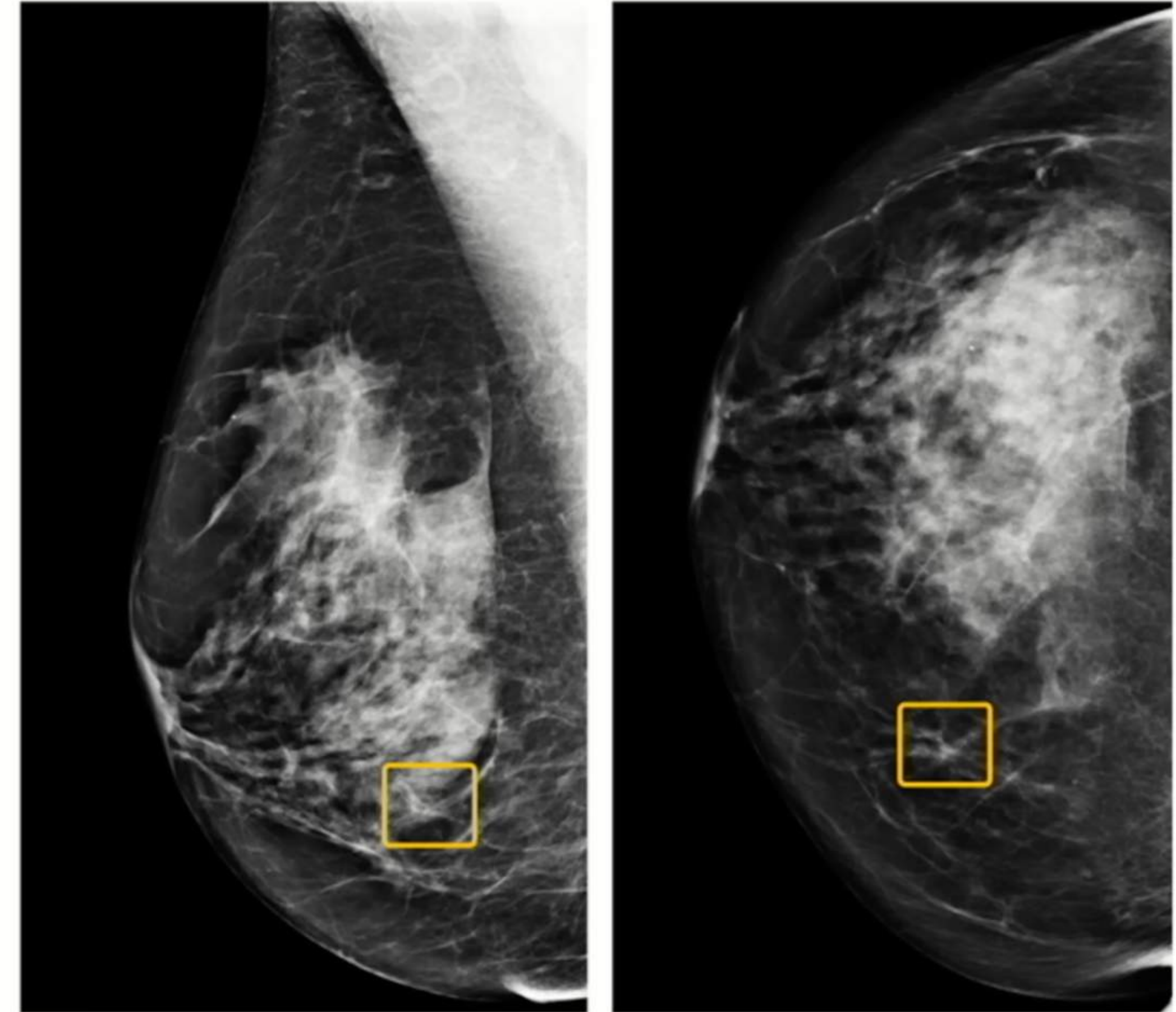


Classification: Breast Cancer Screening

International evaluation of an AI system for breast cancer screening nature



CNN-based system outperformed expert radiologists at detecting breast cancer from mammograms



Breast cancer case missed by radiologist but detected by AI

Object Detection



Image x



CNN



Taxi

Class label y

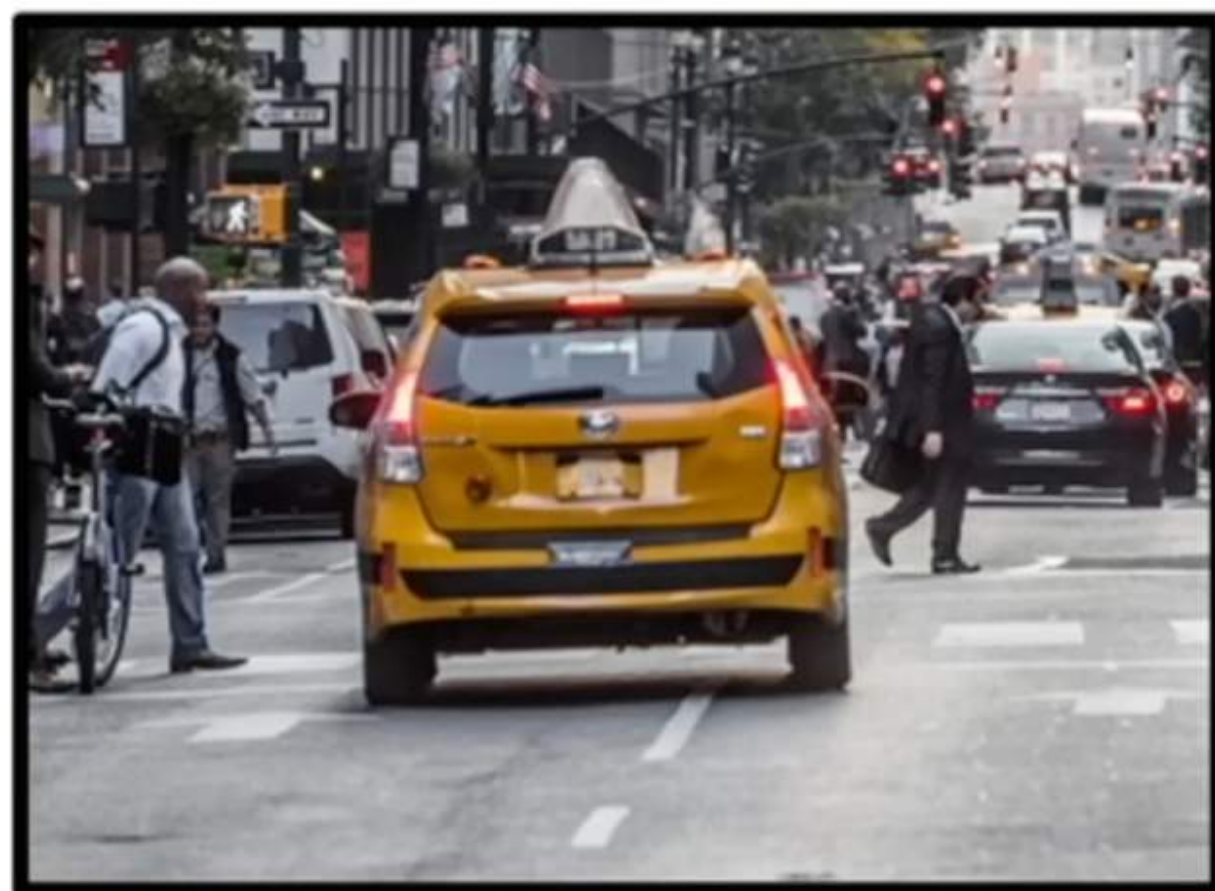


Image x



CNN



Label (x, y, w, h)

Object Detection



Image X

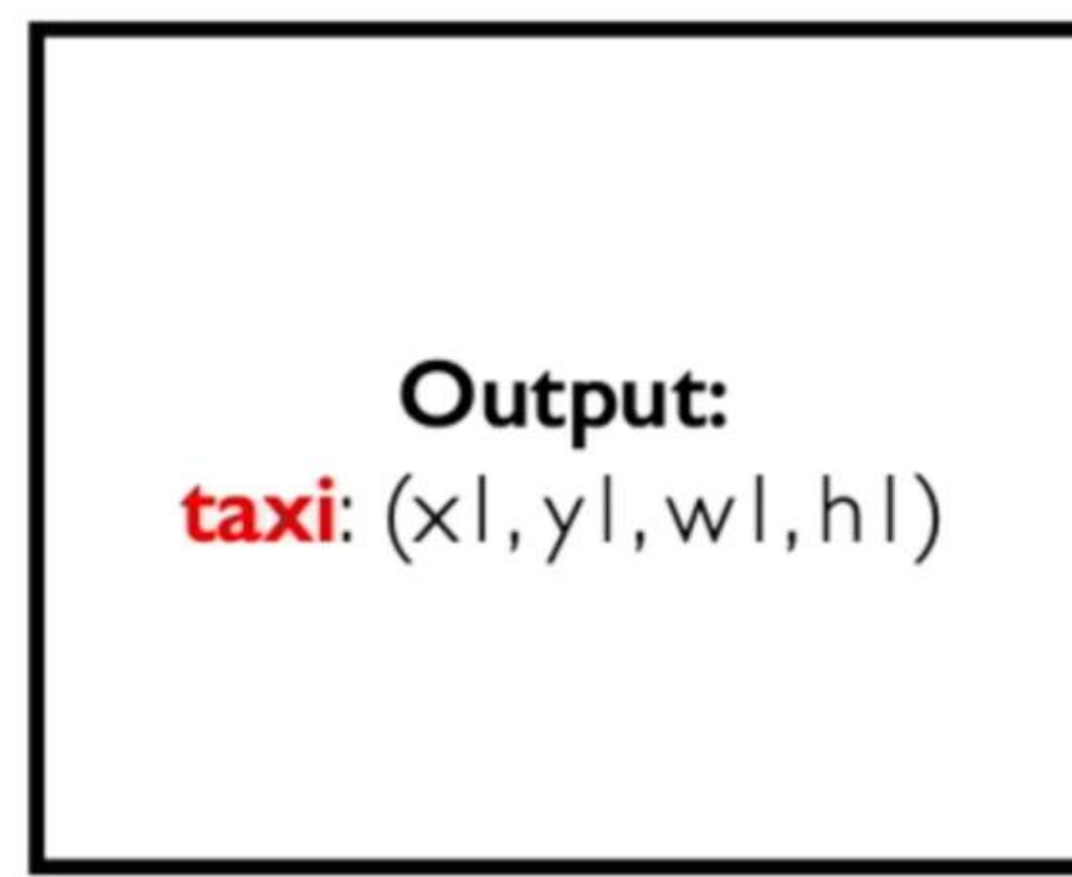
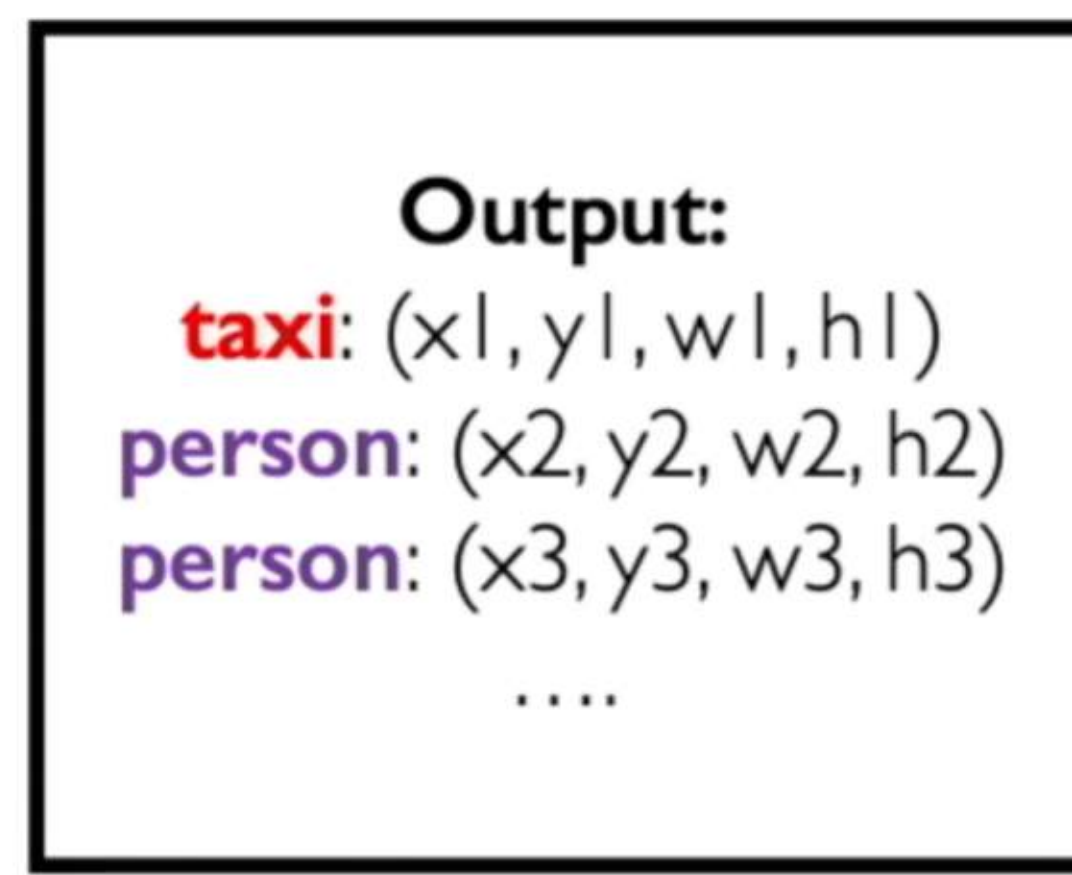


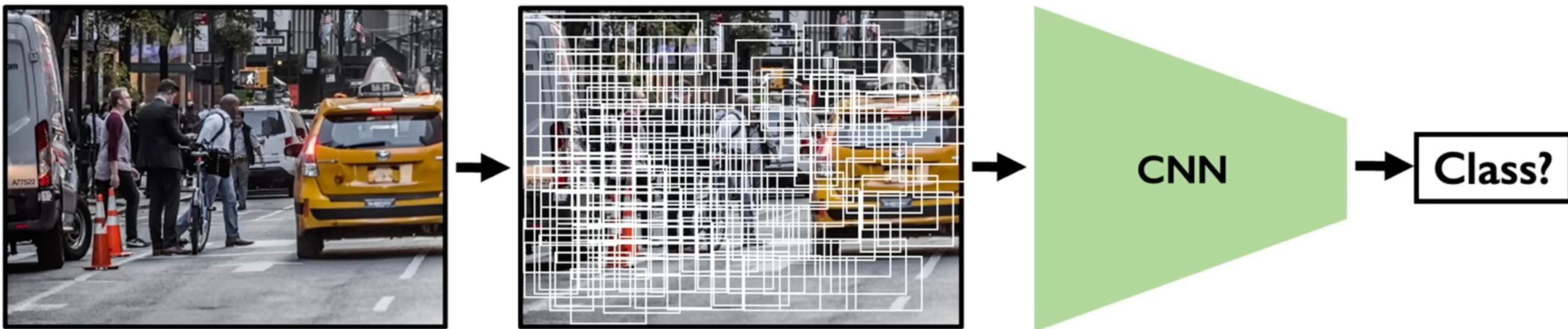
Image X



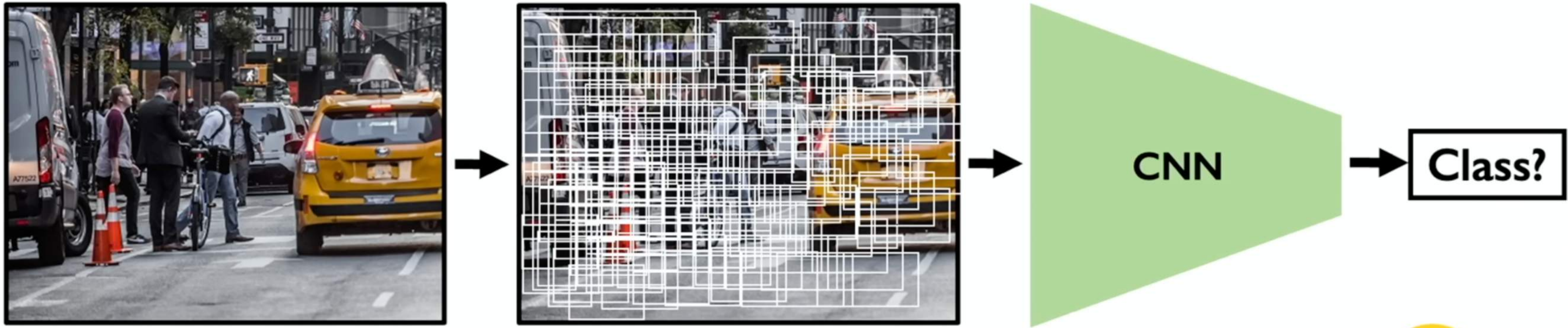
Naïve Solution to Object Detection



Naïve Solution to Object Detection



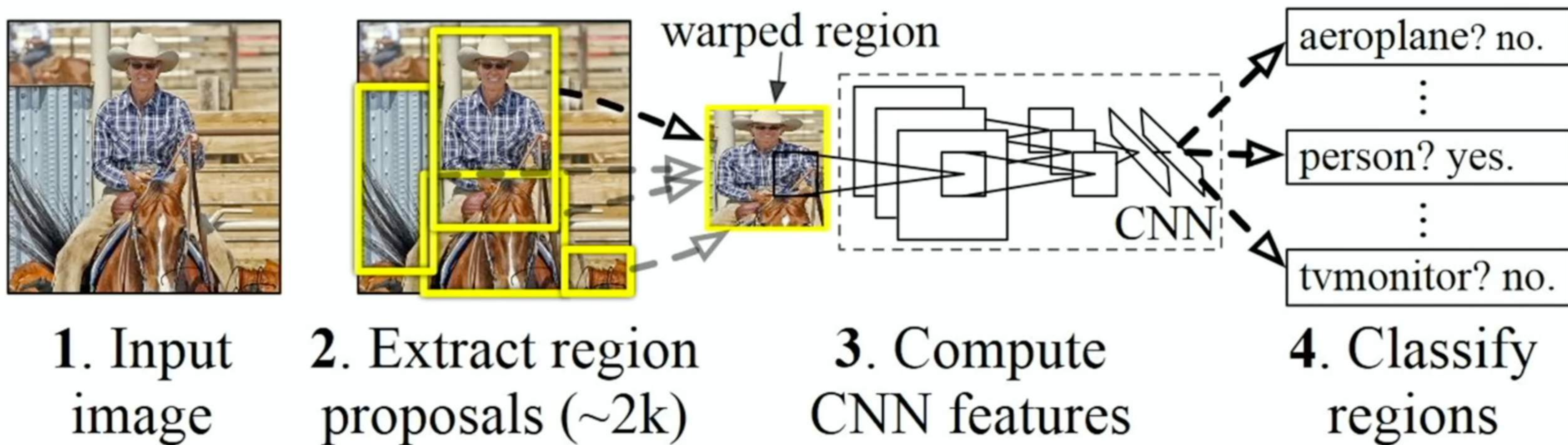
Naïve Solution to Object Detection



Problem: Way too many inputs! This results in too many scales, positions, sizes!

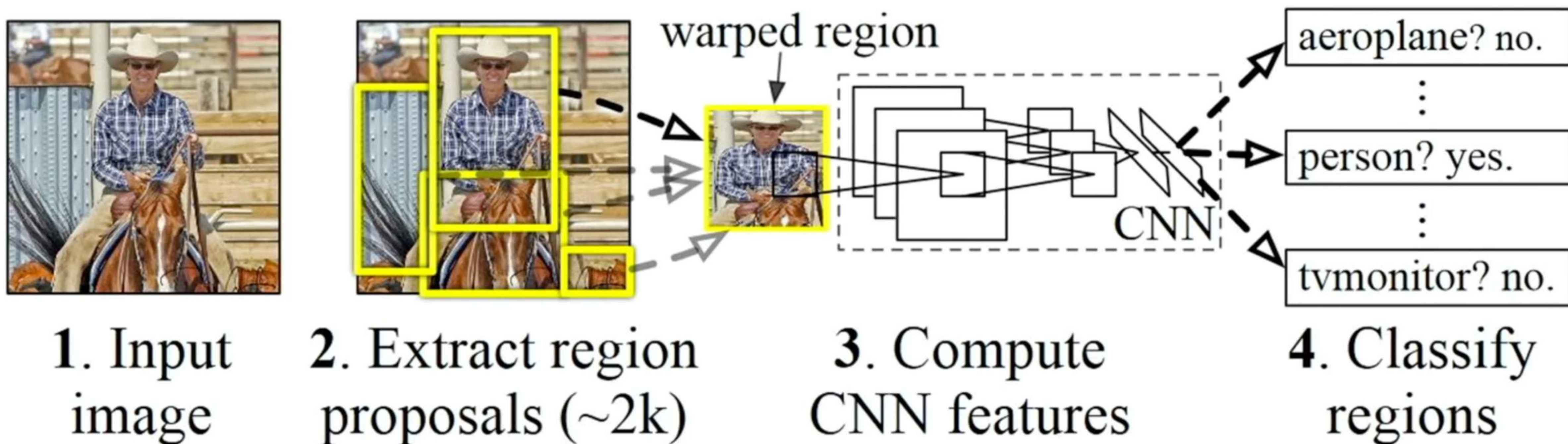
Object Detection with R-CNNs

R-CNN algorithm: Find regions that we think have objects. Use CNN to classify.



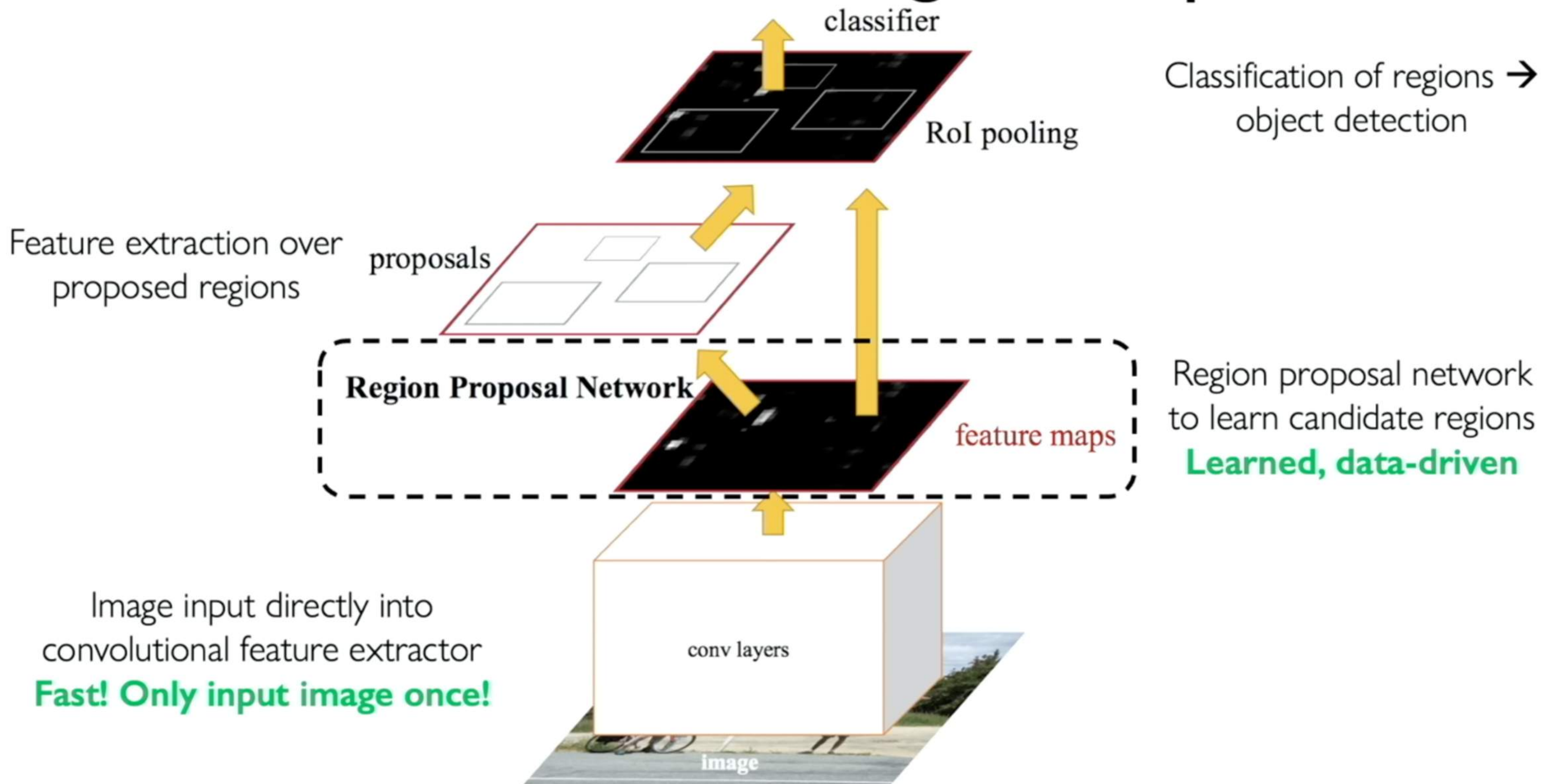
Object Detection with R-CNNs

R-CNN algorithm: Find regions that we think have objects. Use CNN to classify.



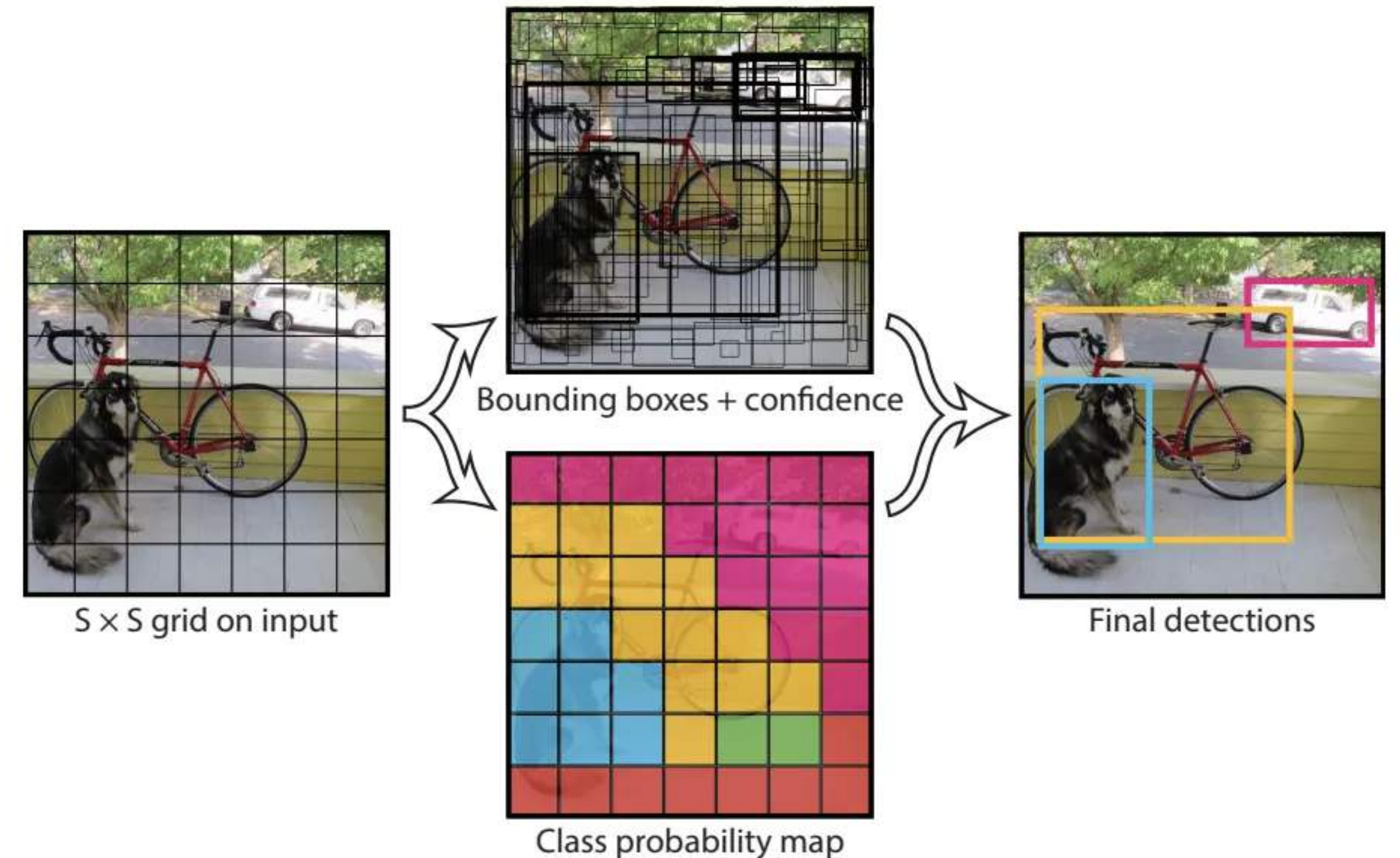
Problems: 1) Slow! Many regions; time intensive inference.
2) Brittle! Manually defined region proposals.

Faster R-CNN Learns Region Proposals



YOLO: You Only Look Once

- Divide the input image into an $S \times S$ grid.
- Each grid cell predicts B bounding boxes with coordinates, confidence scores, and class probabilities.
- Calculate confidence scores for each bounding box, reflecting object presence and accuracy.
- Predict class probabilities for the objects in each grid cell.
- Apply NMS to filter out low-confidence boxes and retain the best ones.
- Output the final bounding boxes with associated class labels and confidence scores.



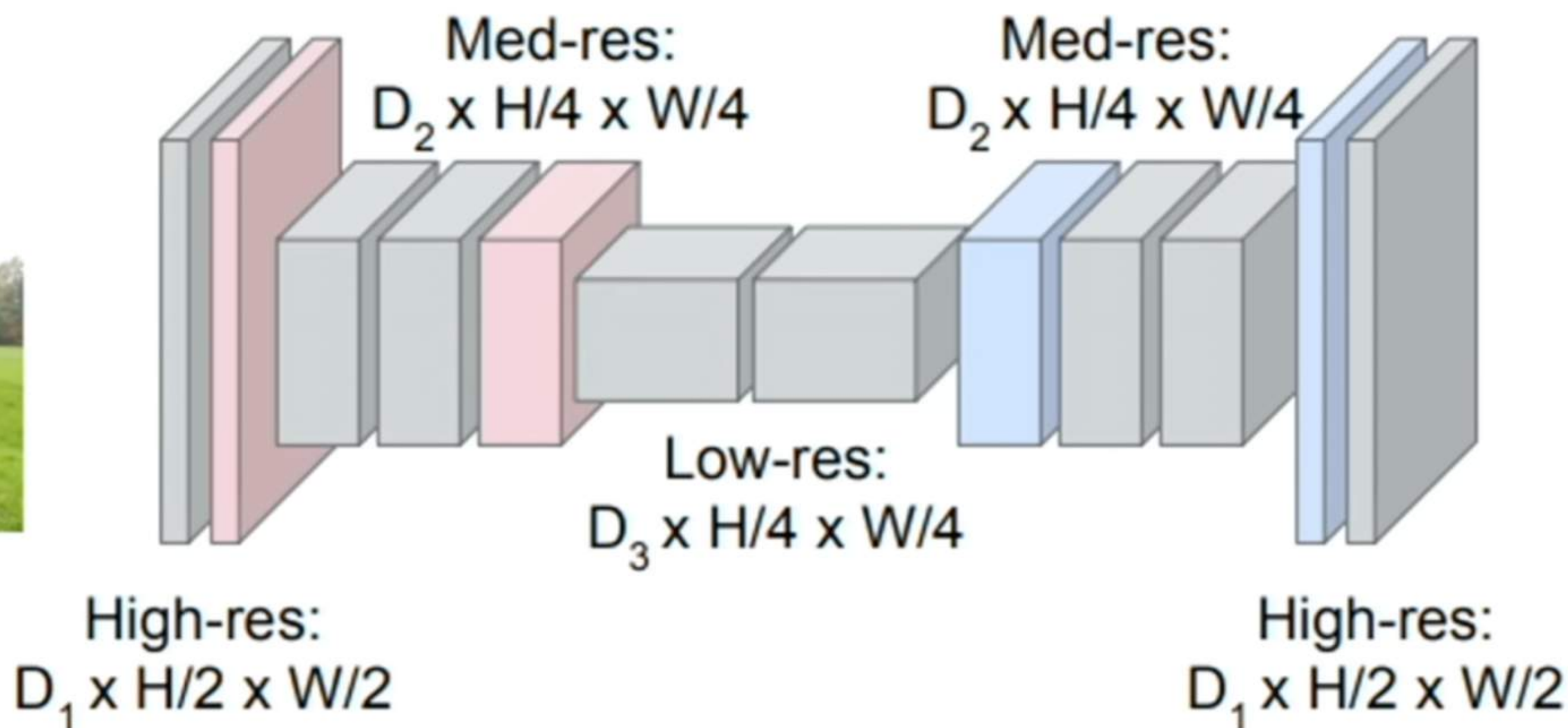
Semantic Segmentation: Fully Convolutional Networks

FCN: Fully Convolutional Network.

Network designed with all convolutional layers, with **downsampling** and **upsampling** operations



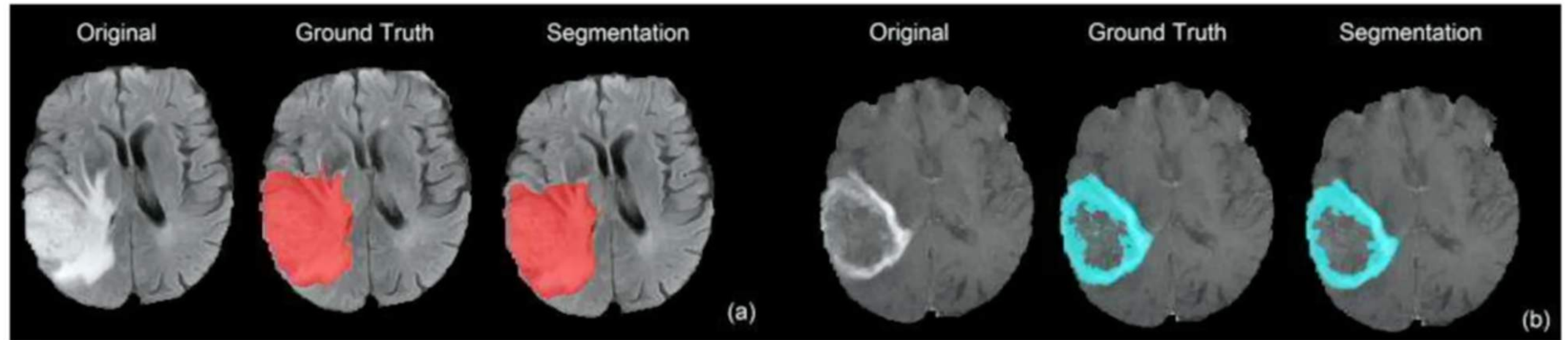
Input:
 $3 \times H \times W$



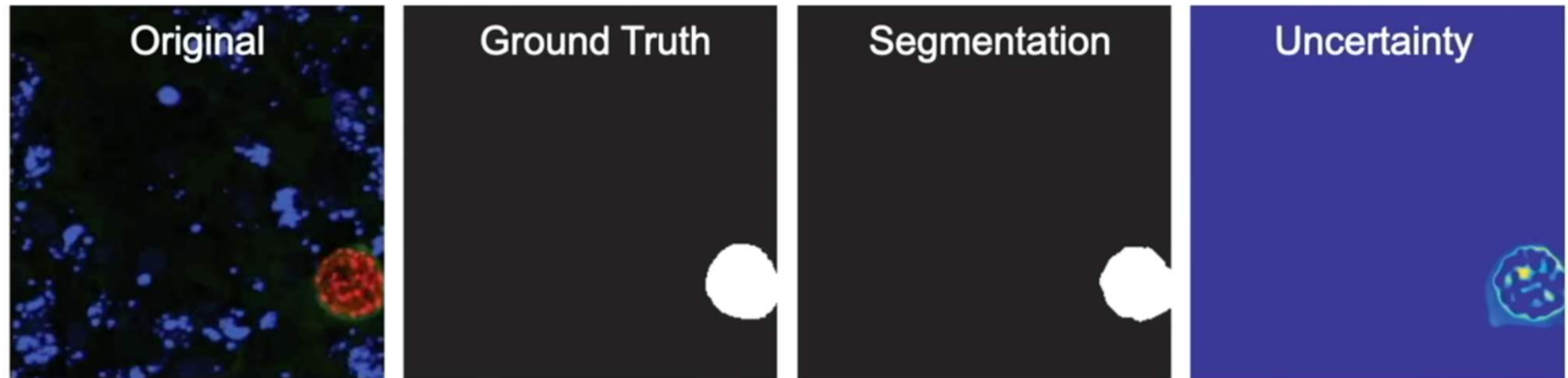
Predictions:
 $H \times W$

Semantic Segmentation: Biomedical Image Analysis

Brain Tumors
Dong+ *MIUA* 2017.



Malaria Infection
Soleimany+ *arXiv* 2019.

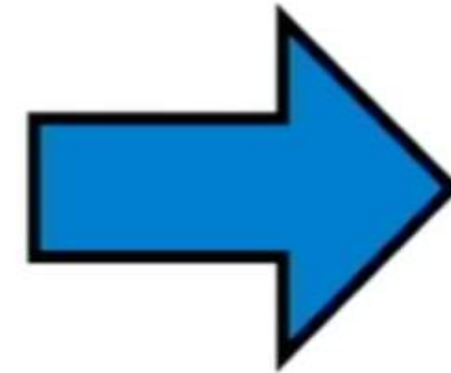
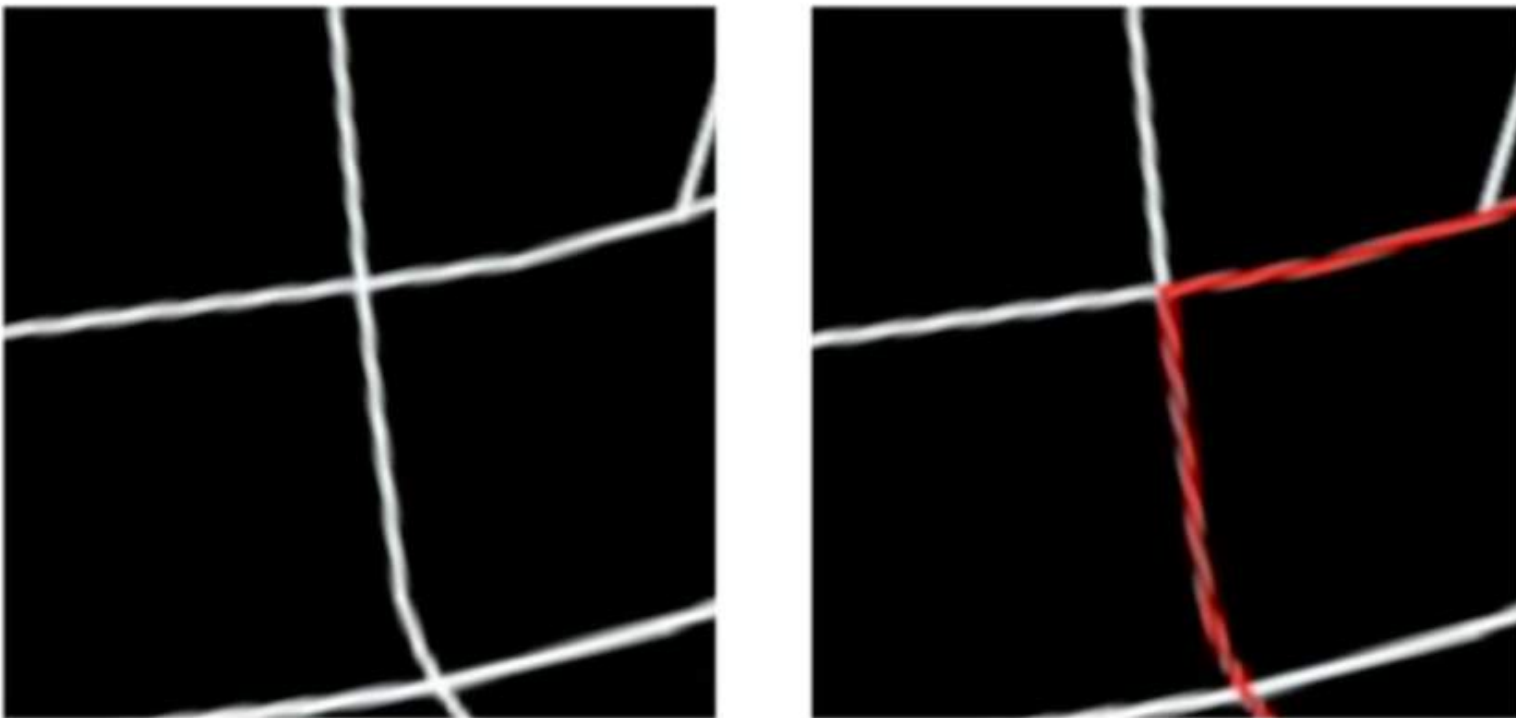


Continuous Control: Navigation from Vision

Raw Perception
 I
(ex. camera)



Coarse Maps
 M
(ex. GPS)

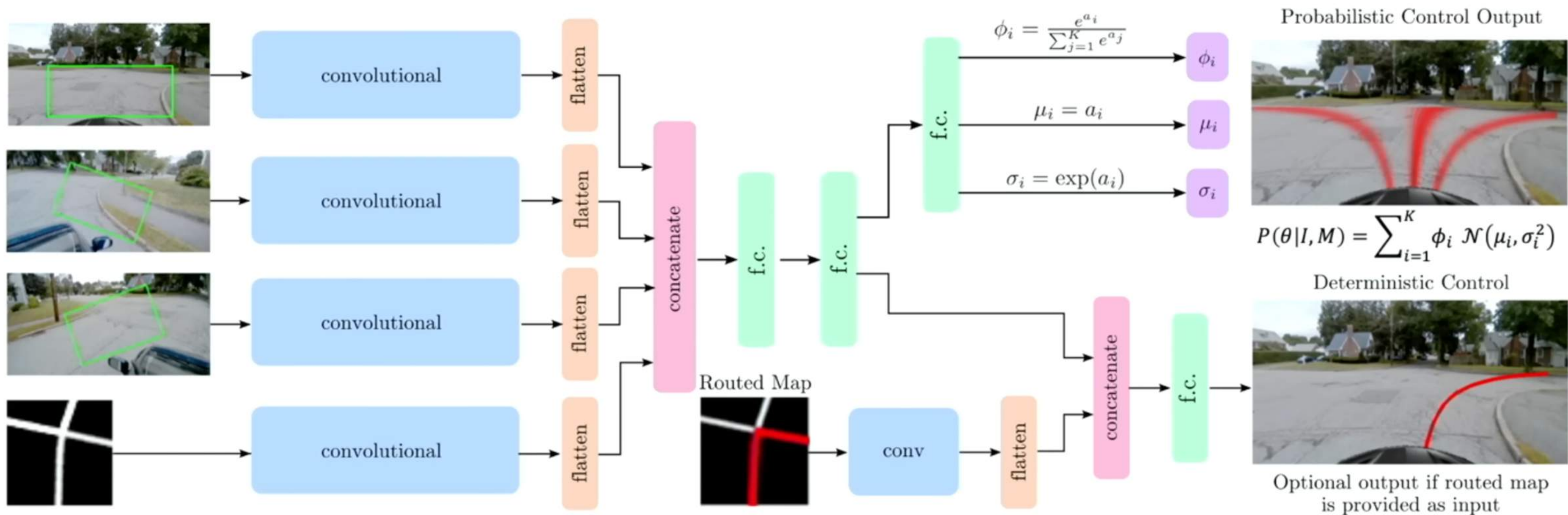


Possible Control Commands



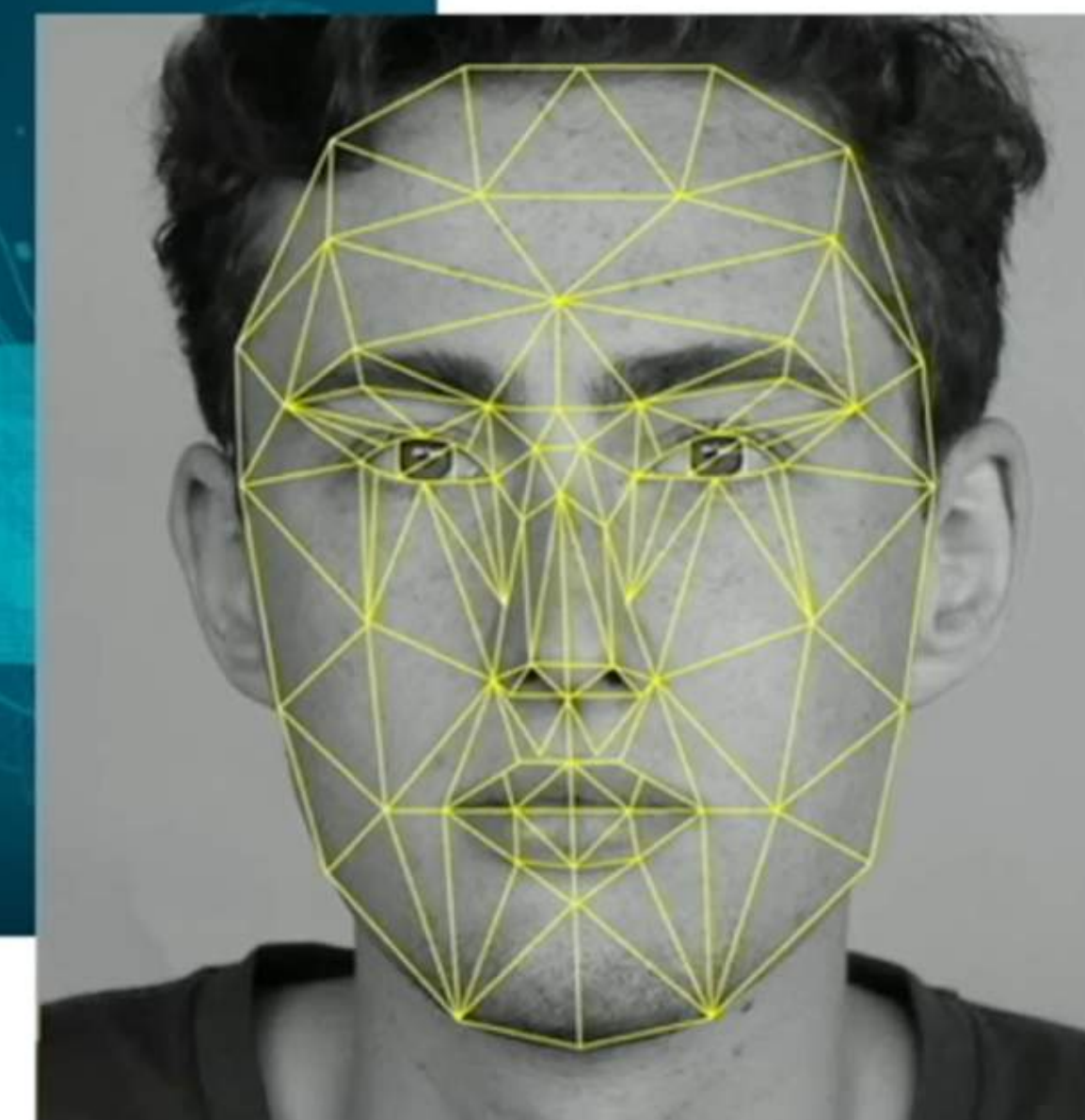
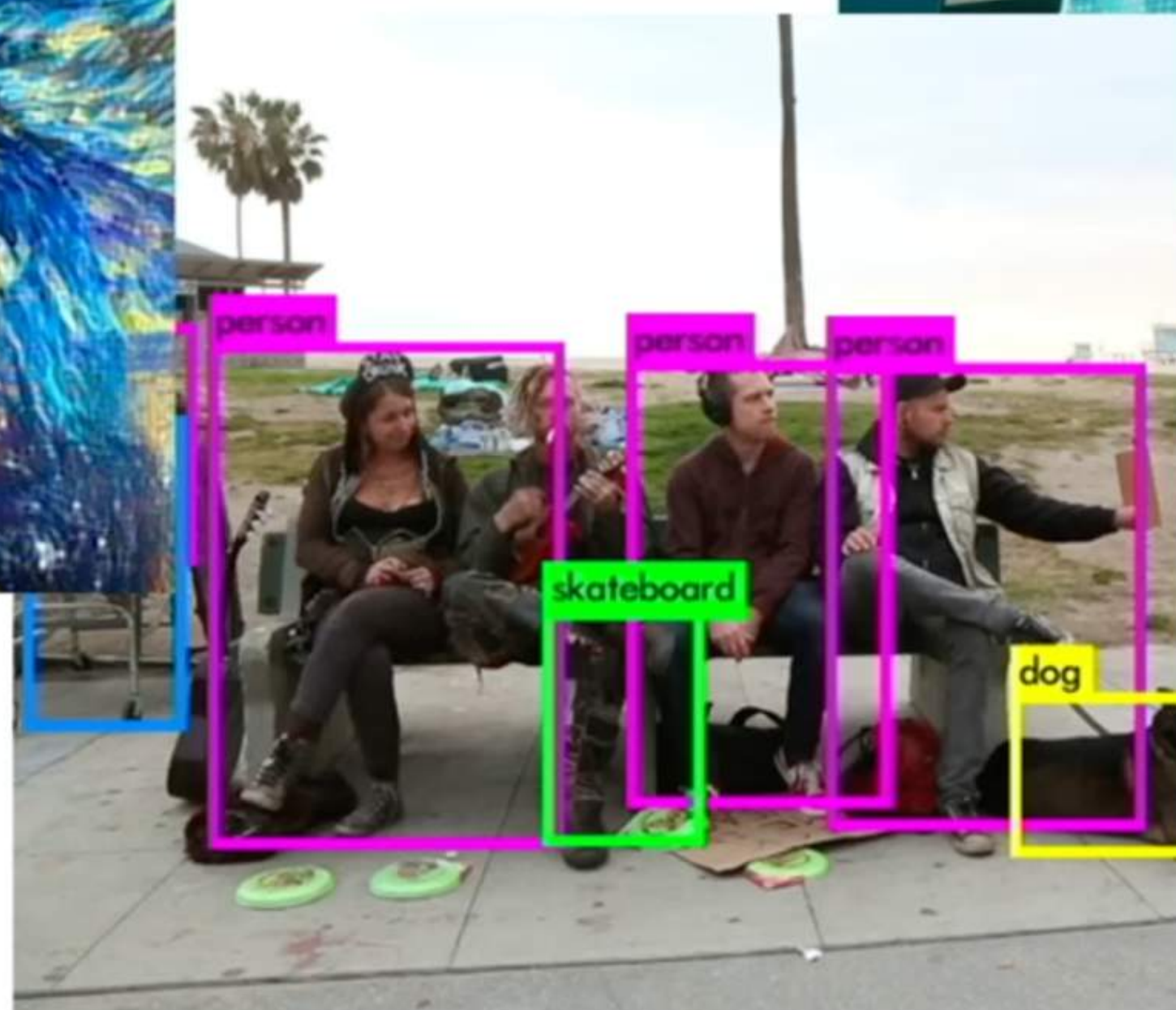
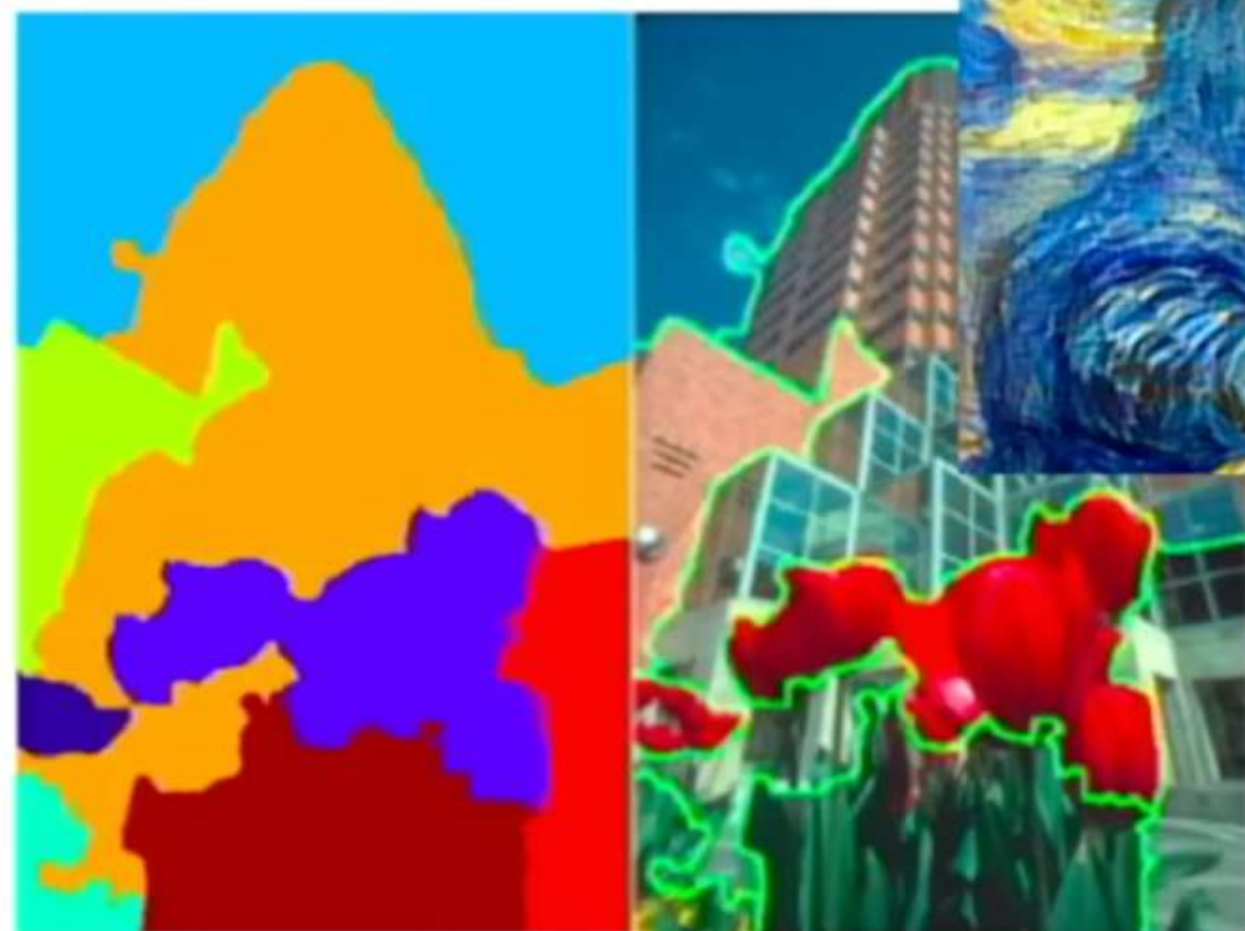
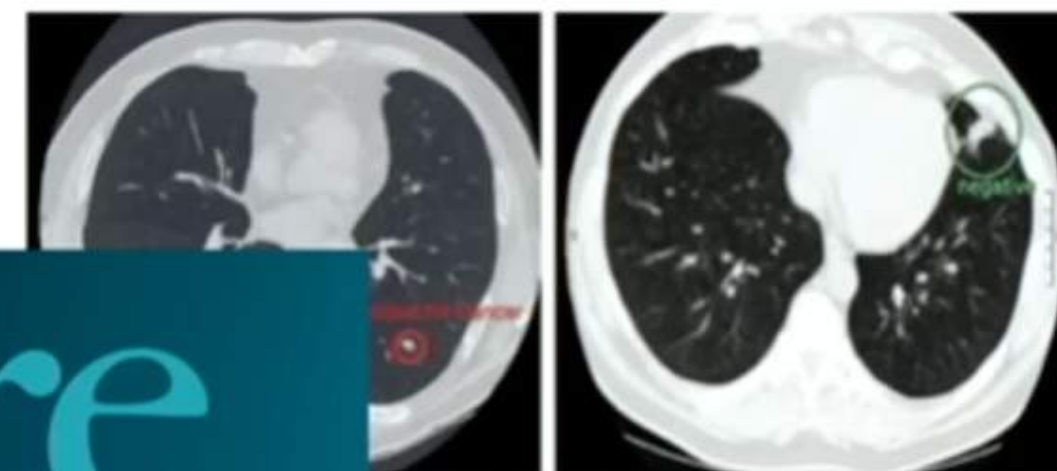
End-to-End Framework for Autonomous Navigation

Entire model is trained end-to-end **without any human labelling or annotations**



$$L = -\log(P(\theta|I, M))$$

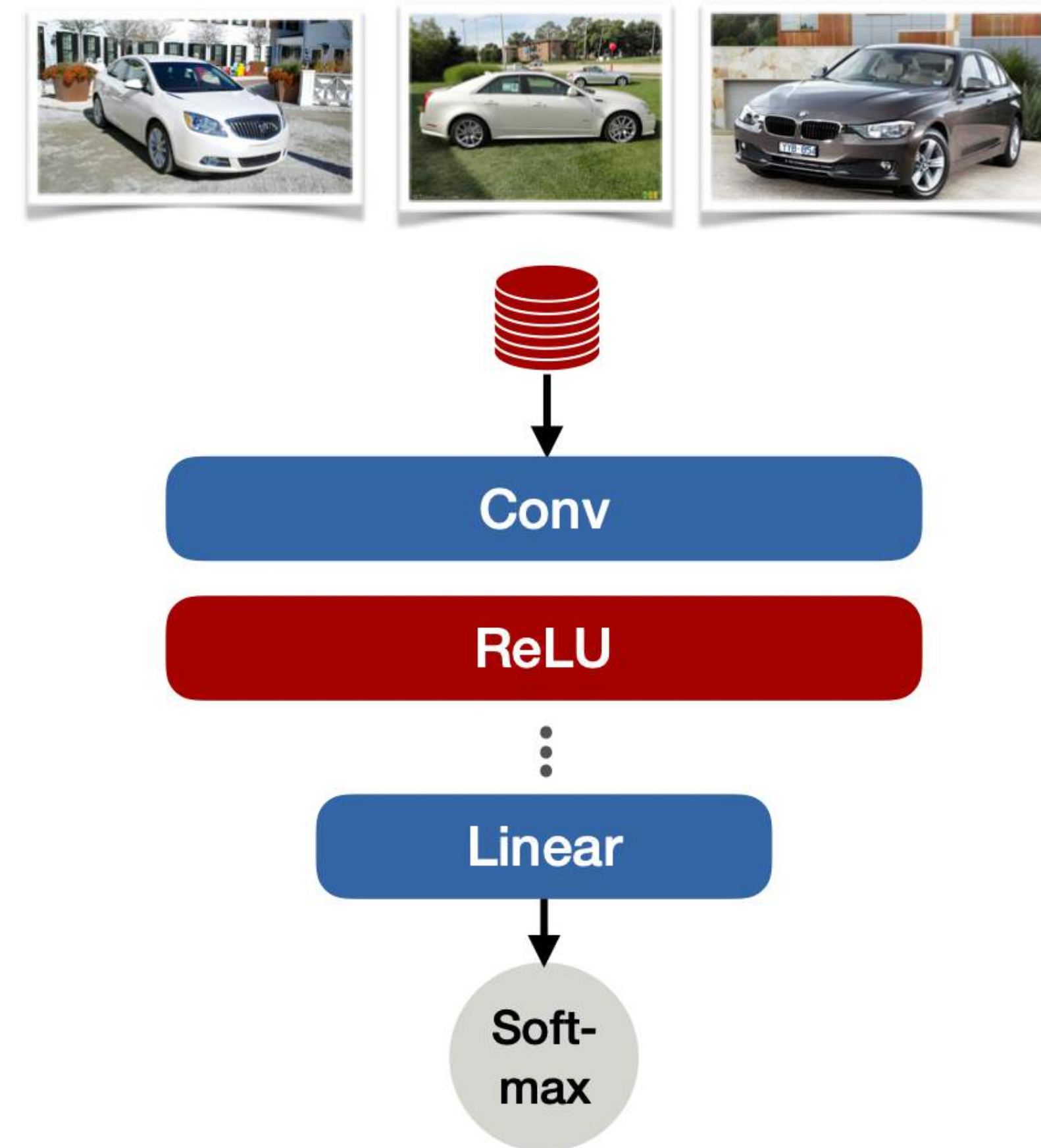
Deep Learning for Computer Vision: Impact



Transfer Learning

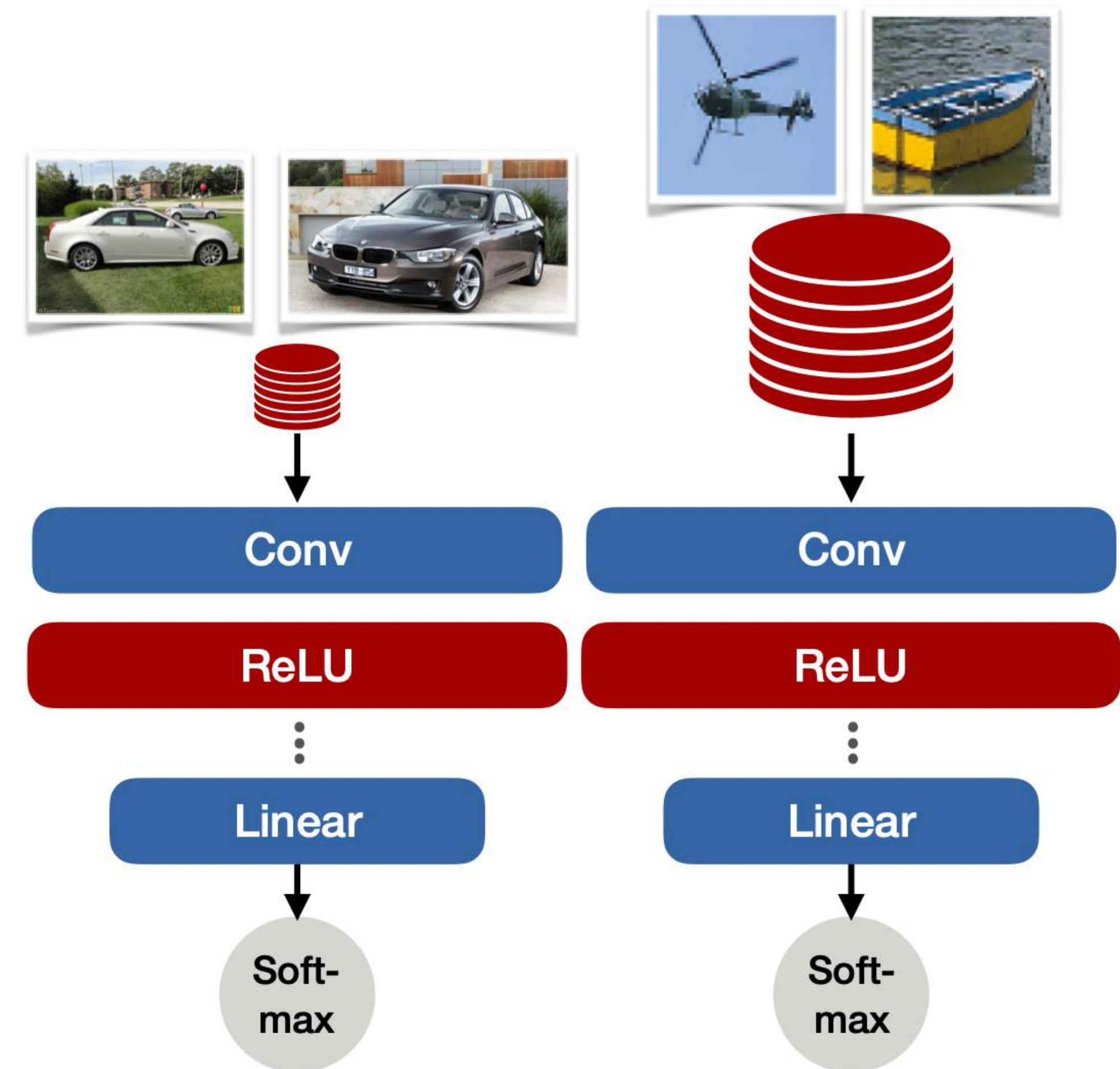
Training on small datasets

- How to train
 - A large model
 - On a small dataset



Solution: Pre-training/Fine-tuning

- Pre-training: Train model on a large dataset for example COCO dataset
 - On related task
- Copy model weights
- Fine-tuning: Continue training on small dataset



Pre-training

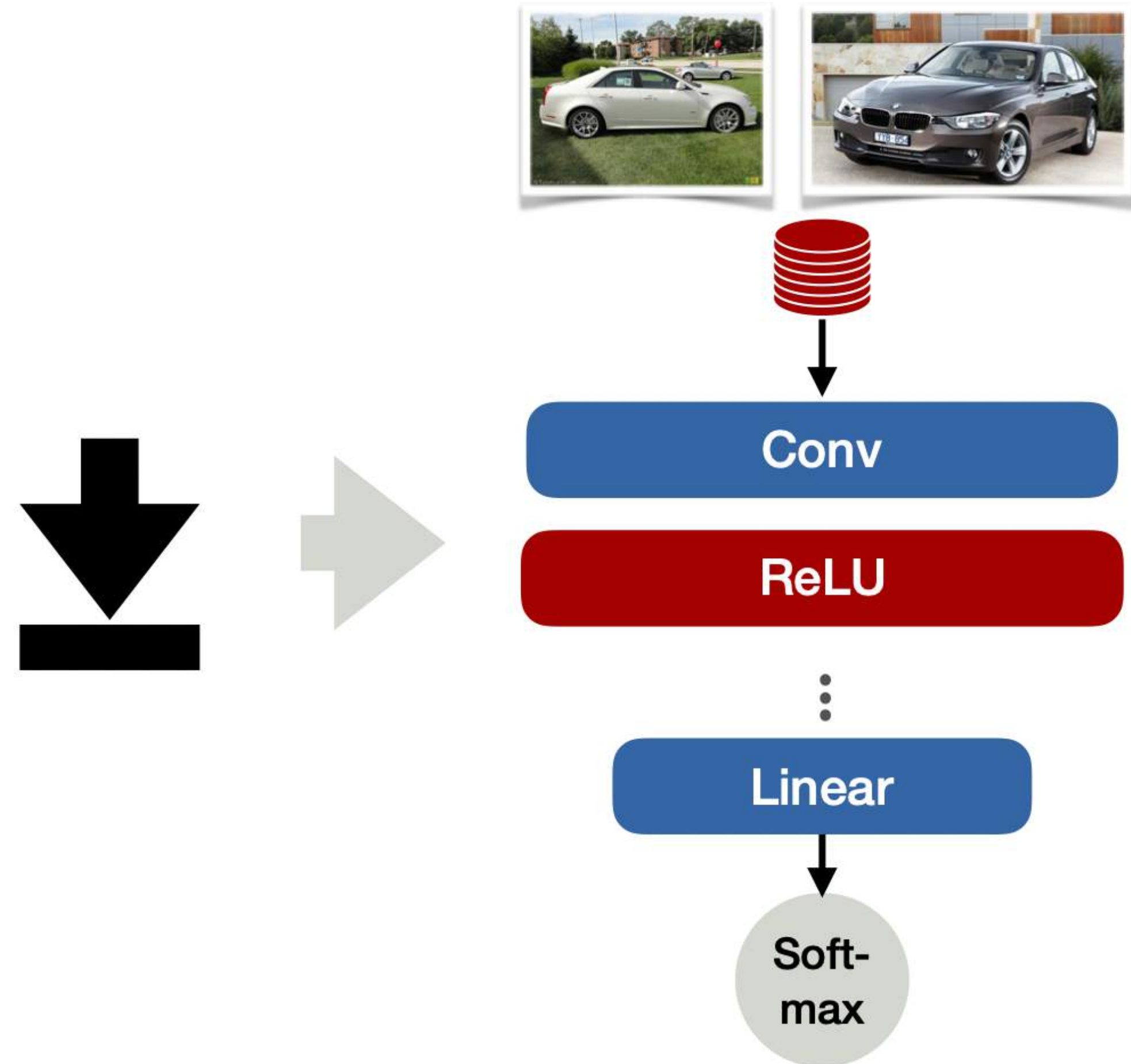
- Computer Vision
 - Supervised
 - e.g. ImageNet, COCO
- Natural Language Processing
 - Self-supervised
 - Unlabeled text



WIKIPEDIA
The Free Encyclopedia

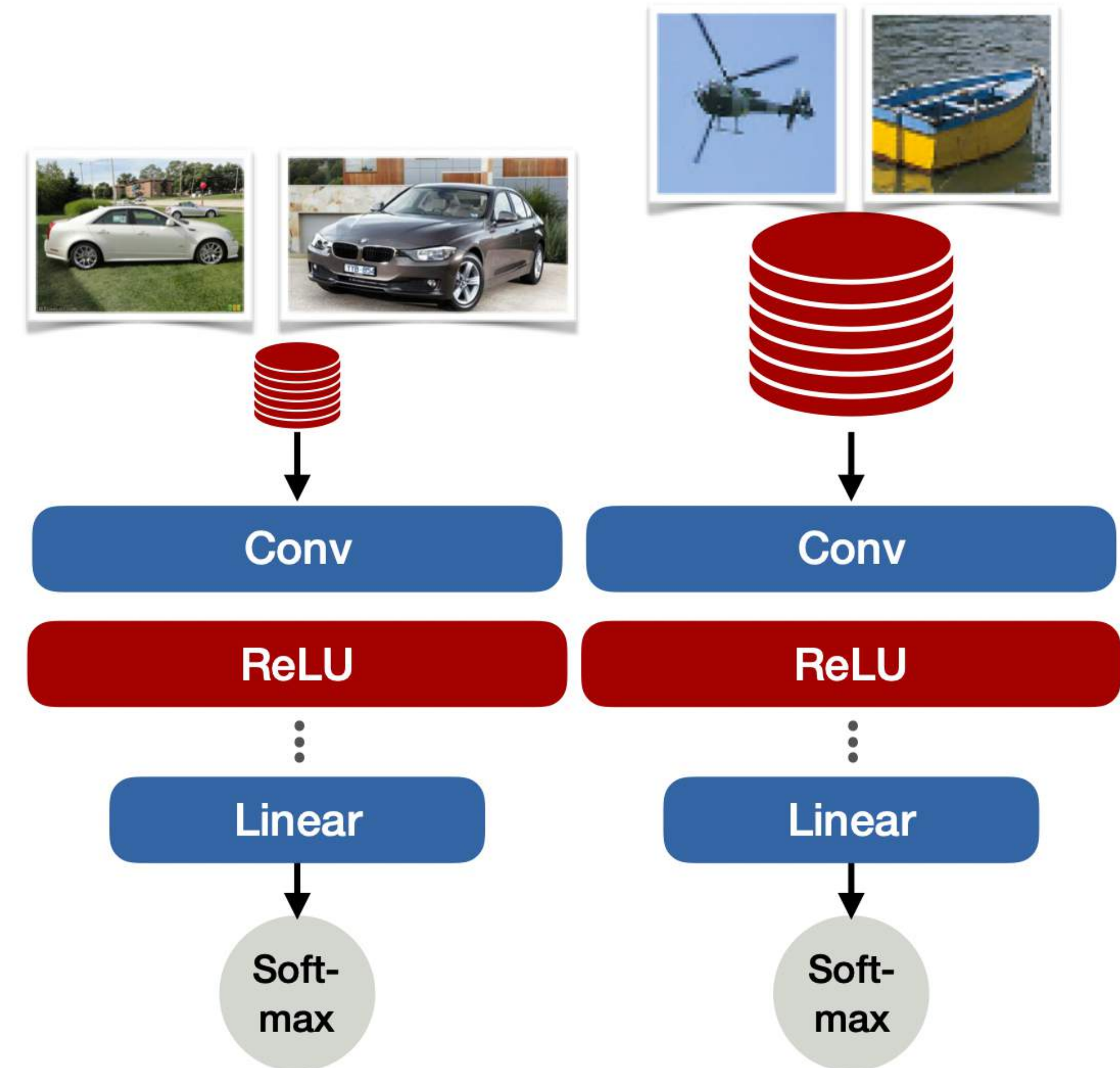
Pre-training/Fine-tuning in practice

- Download a pre-trained model
 - From model-zoo
 - Directly from author
- Run a few training iterations on small dataset



Why does transfer learning work?

- Similar inputs
 - e.g. Images, Text, ...
 - Transfer between tasks
- Good initialization
 - Learned weights can be tuned better



When to use transfer learning?

- Whenever possible
 - In early experiments
 - Large pre-trained model exists

Auto-encoders

Supervised vs unsupervised learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn function to map
 $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, etc.

Unsupervised Learning

Data: x

x is data, no labels!

Goal: Learn some *hidden* or *underlying structure* of the data

Examples: Clustering, feature or dimensionality reduction, etc.

Supervised vs unsupervised learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn function to map
 $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, etc.

Unsupervised Learning

Data: x

x is data, no labels!

Goal: Learn the *hidden* or *underlying structure* of the data

Examples: Clustering, feature or dimensionality reduction, etc.

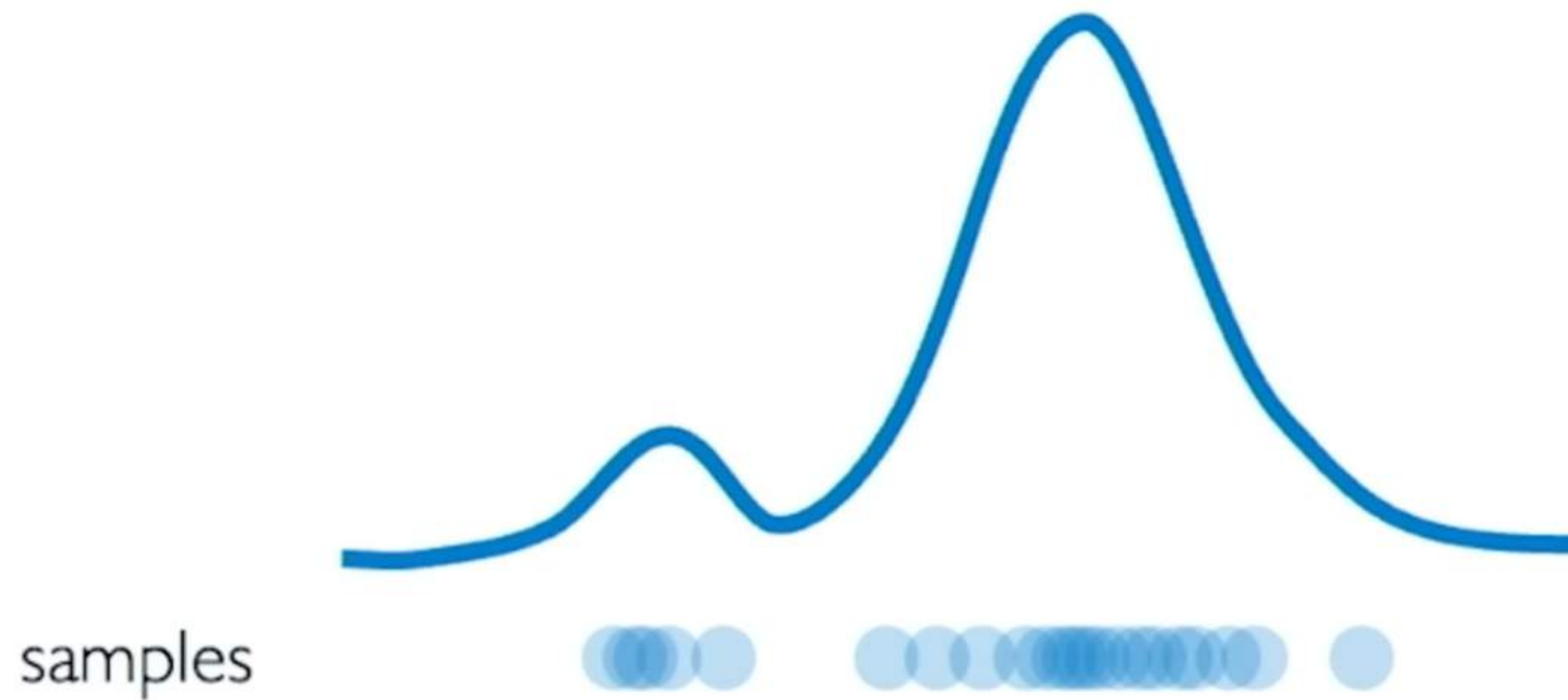
Generative modeling

Goal: Take as input training samples from some distribution and learn a model that represents that distribution

Generative modeling

Goal: Take as input training samples from some distribution and learn a model that represents that distribution

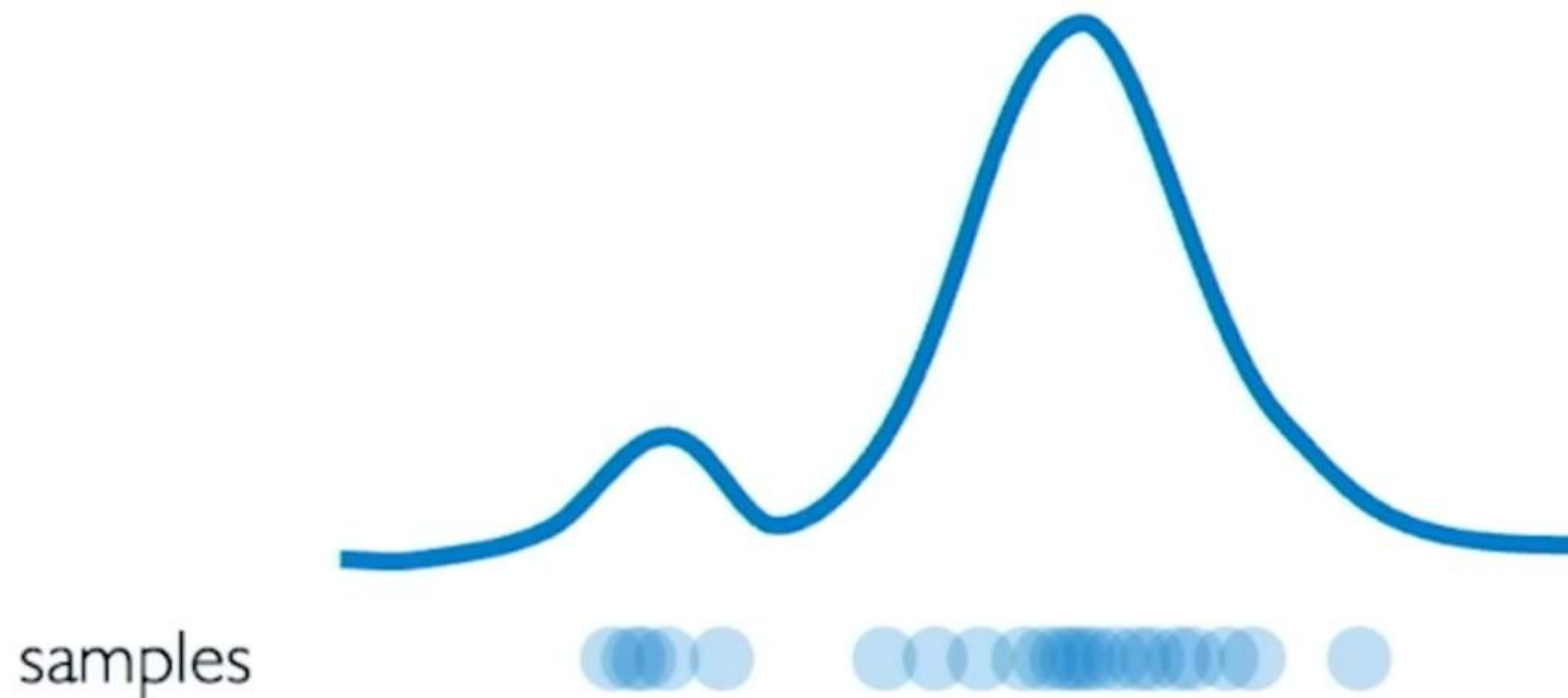
Density Estimation



Generative modeling

Goal: Take as input training samples from some distribution and learn a model that represents that distribution

Density Estimation

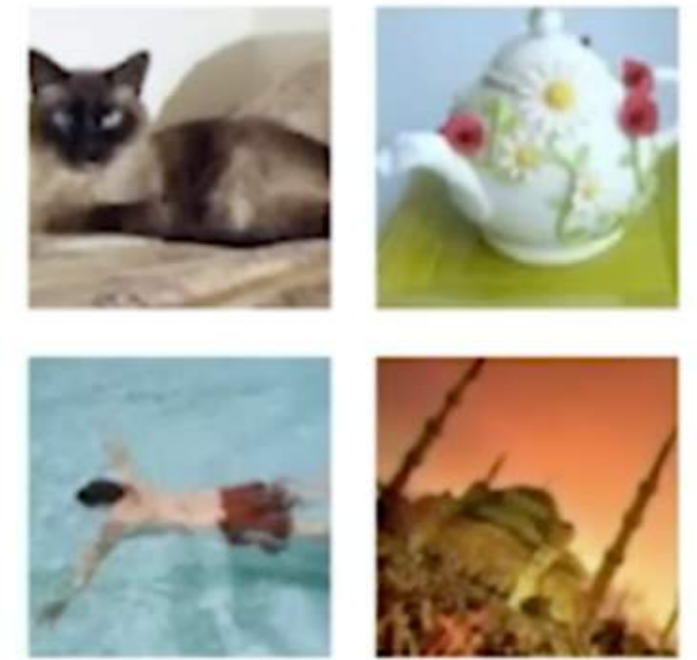


Sample Generation



Input samples

Training data $\sim P_{data}(x)$



Generated samples

Generated $\sim P_{model}(x)$

How can we learn $P_{model}(x)$ similar to $P_{data}(x)$?

Why generative models? Debiasing

Capable of uncovering **underlying features** in a dataset



Homogeneous skin color, pose

VS



Diverse skin color, pose, illumination

How can we use this information to create fair and representative datasets?

Why generative models? Outlier detection

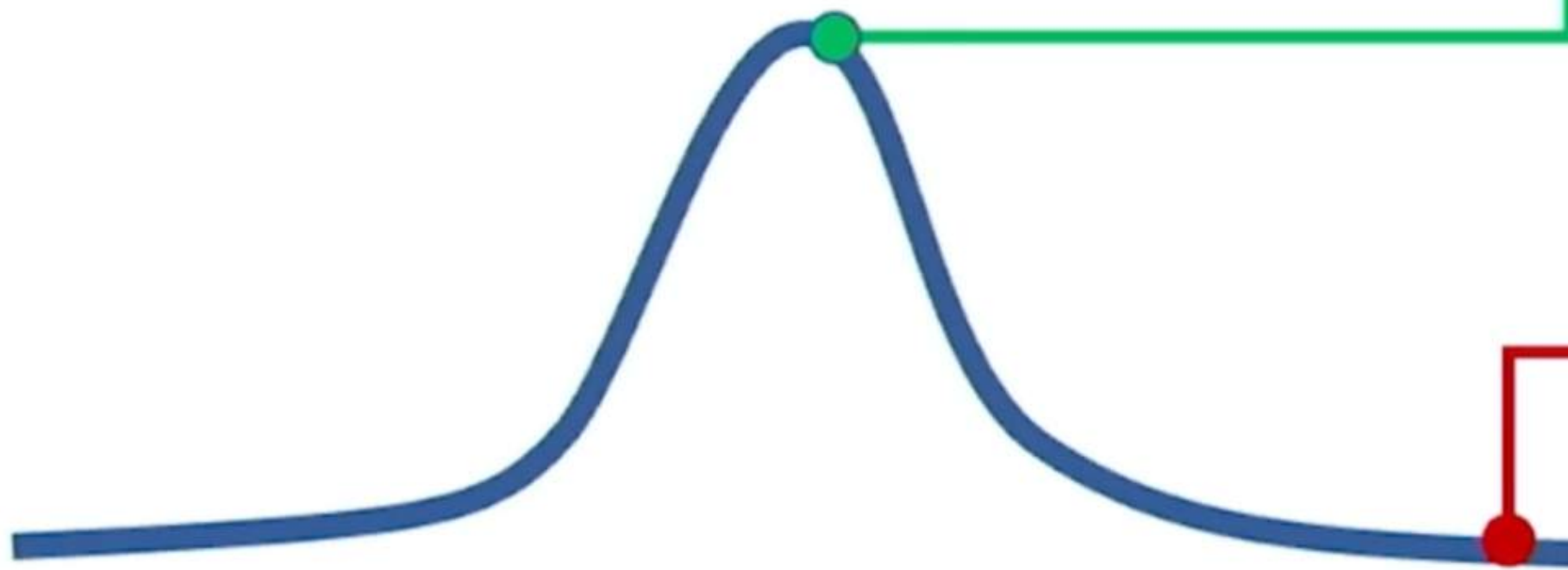
- **Problem:** How can we detect when we encounter something new or rare?
- **Strategy:** Leverage generative models, detect outliers in the distribution
- Use outliers during training to improve even more!

95% of Driving Data:

(1) sunny, (2) highway, (3) straight road



Detect outliers to avoid unpredictable behavior when training



Edge Cases



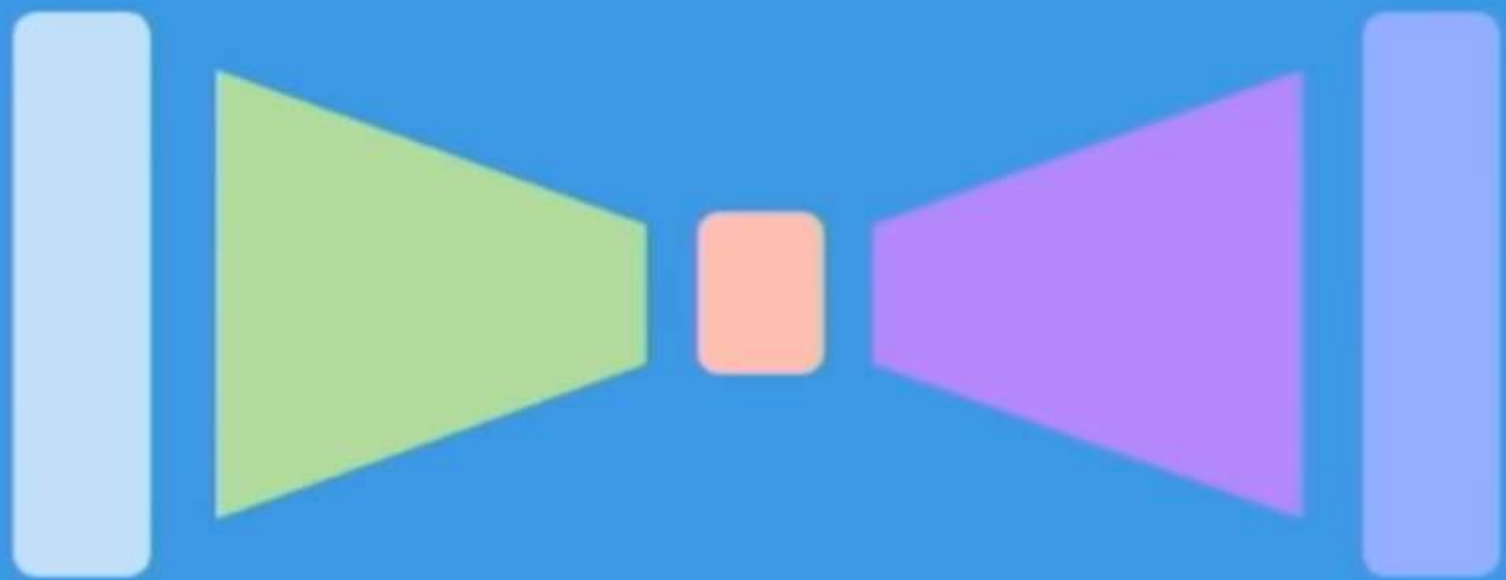
Harsh Weather



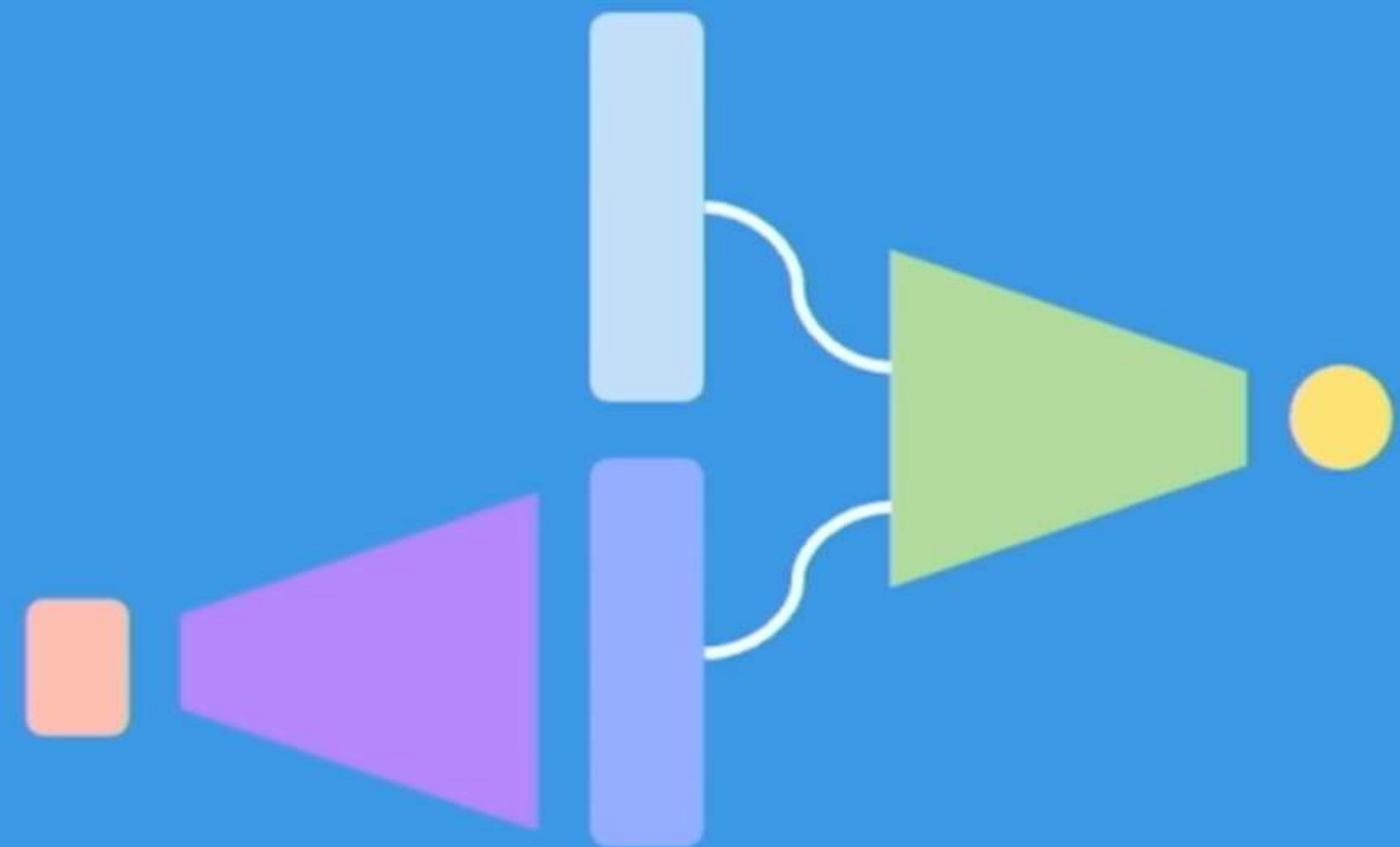
Pedestrians

Latent variable models

Autoencoders and Variational Autoencoders (VAEs)



Generative Adversarial Networks (GANs)



What is a latent variable?



Myth of the Cave
Plato, Republic

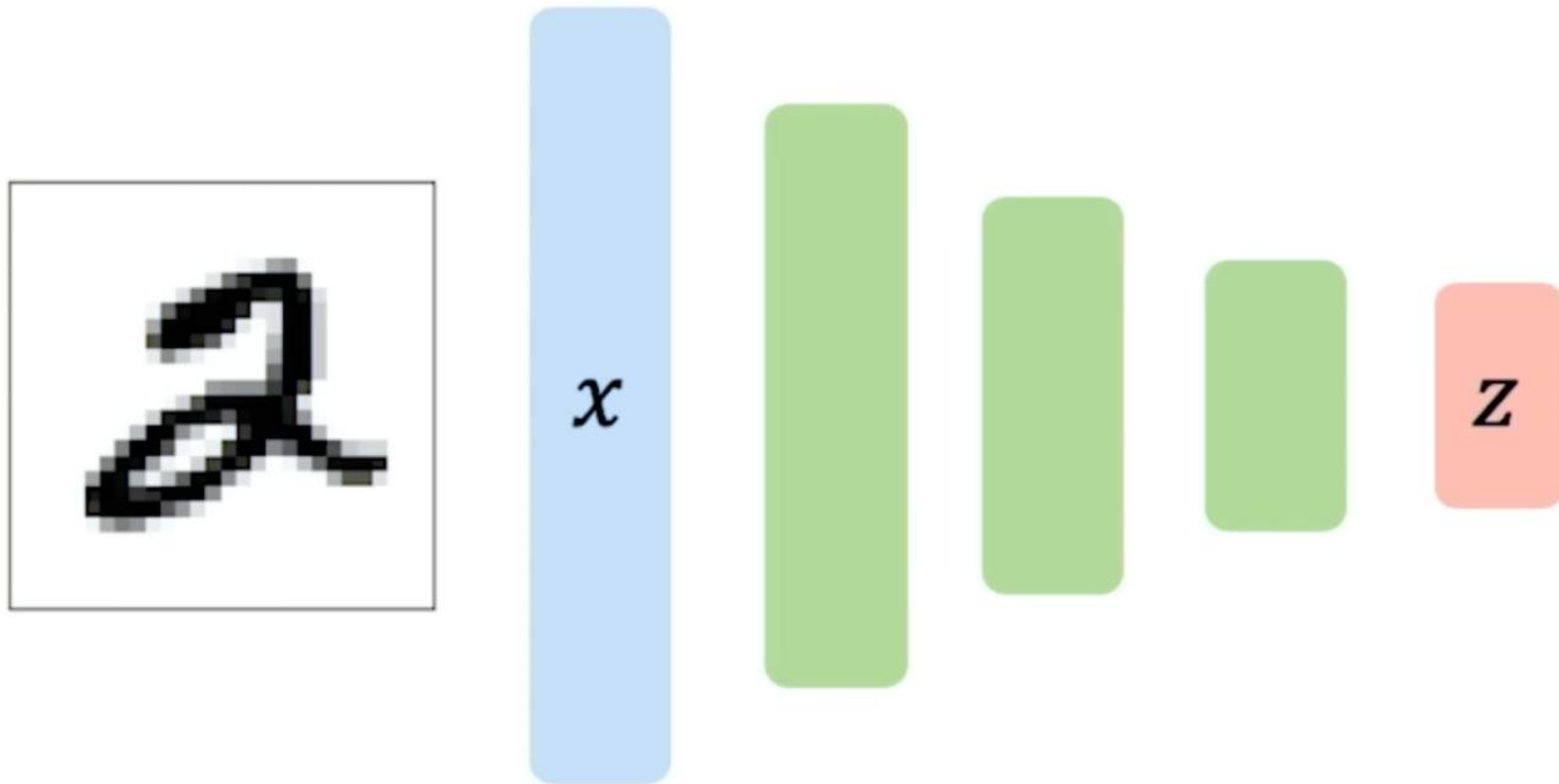
What is a latent variable?



Can we learn the **true explanatory factors**, e.g. latent variables, from only observed data?

Autoencoders: background

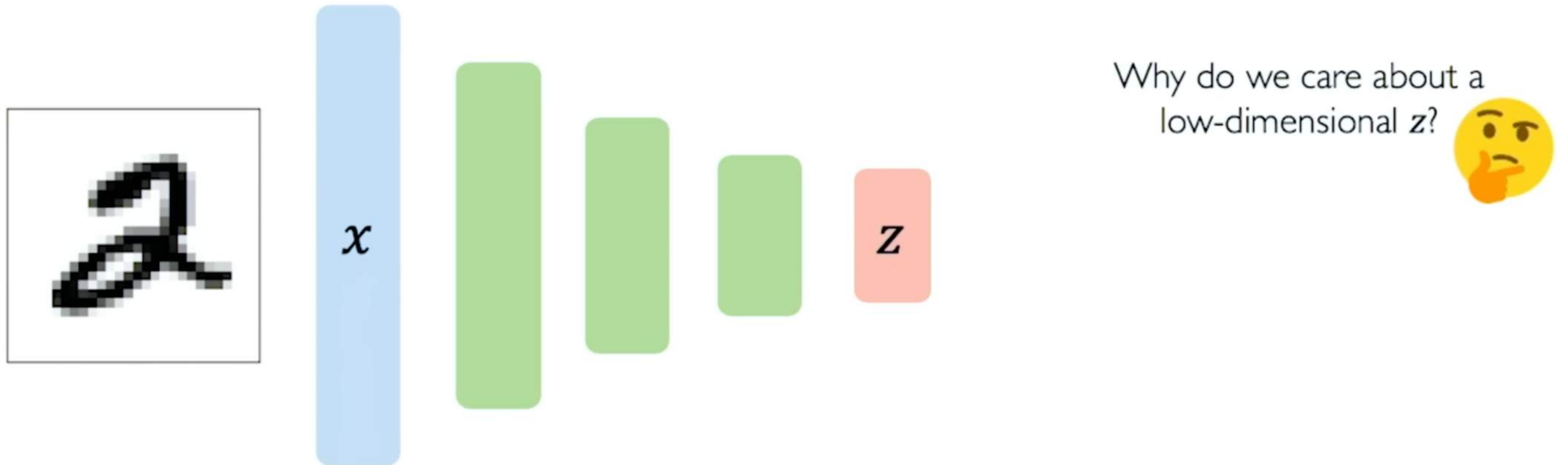
Unsupervised approach for learning a **lower-dimensional** feature representation from unlabeled training data



“Encoder” learns mapping from the data, x , to a low-dimensional latent space, z

Autoencoders: background

Unsupervised approach for learning a **lower-dimensional** feature representation from unlabeled training data

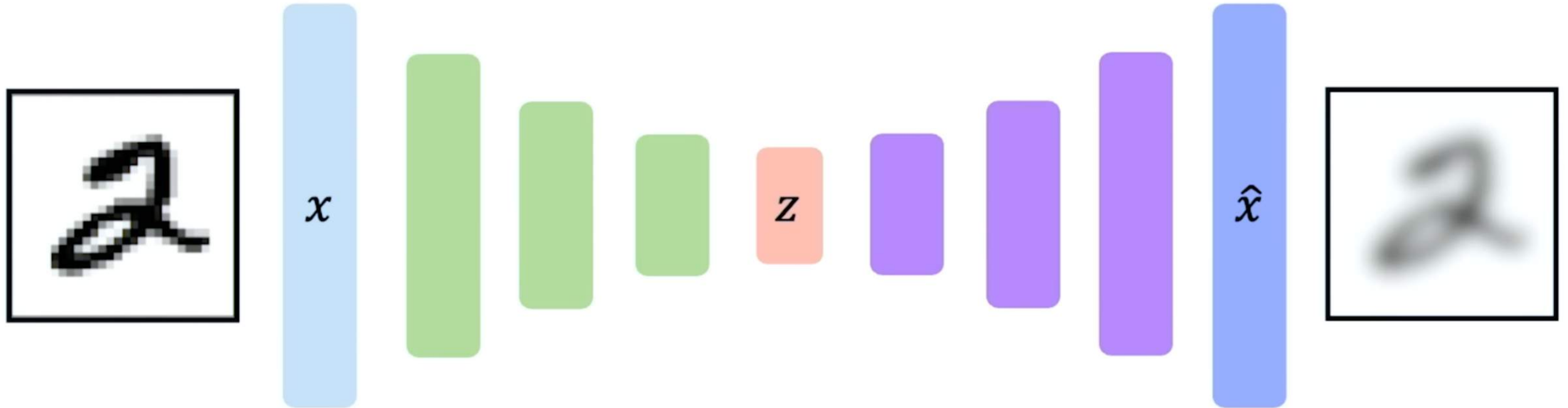


“Encoder” learns mapping from the data, x , to a low-dimensional latent space, z

Autoencoders: background

How can we learn this latent space?

Train the model to use these features to **reconstruct the original data**

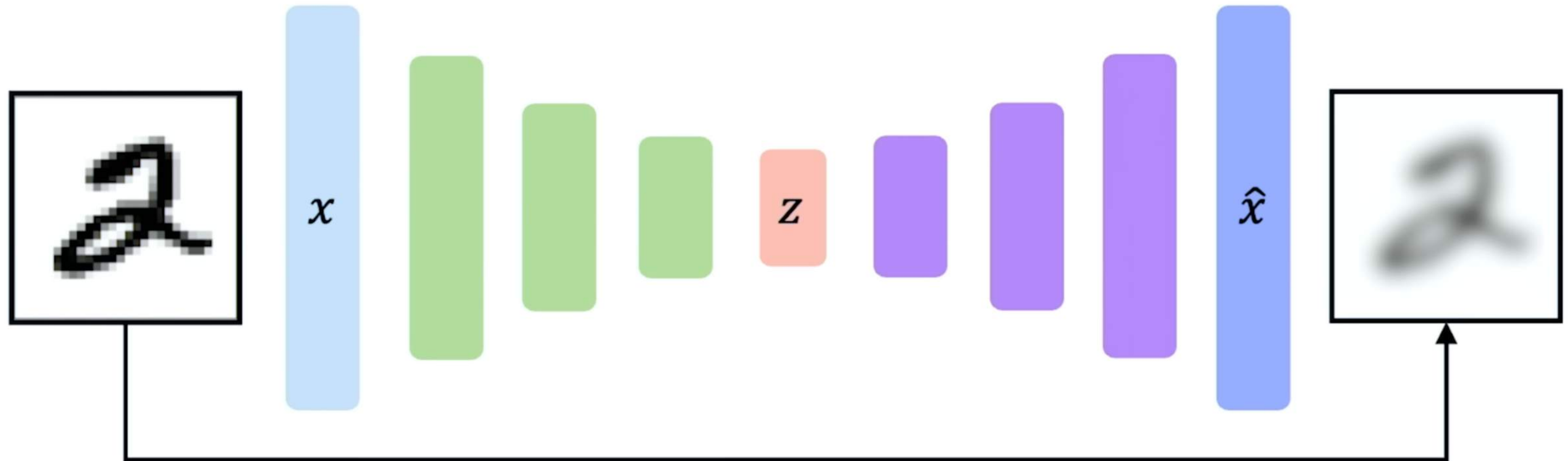


“Decoder” learns mapping back from latent space, z ,
to a reconstructed observation, $\hat{x}$

Autoencoders: background

How can we learn this latent space?

Train the model to use these features to **reconstruct the original data**



$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2$$

Loss function doesn't
use any labels!!

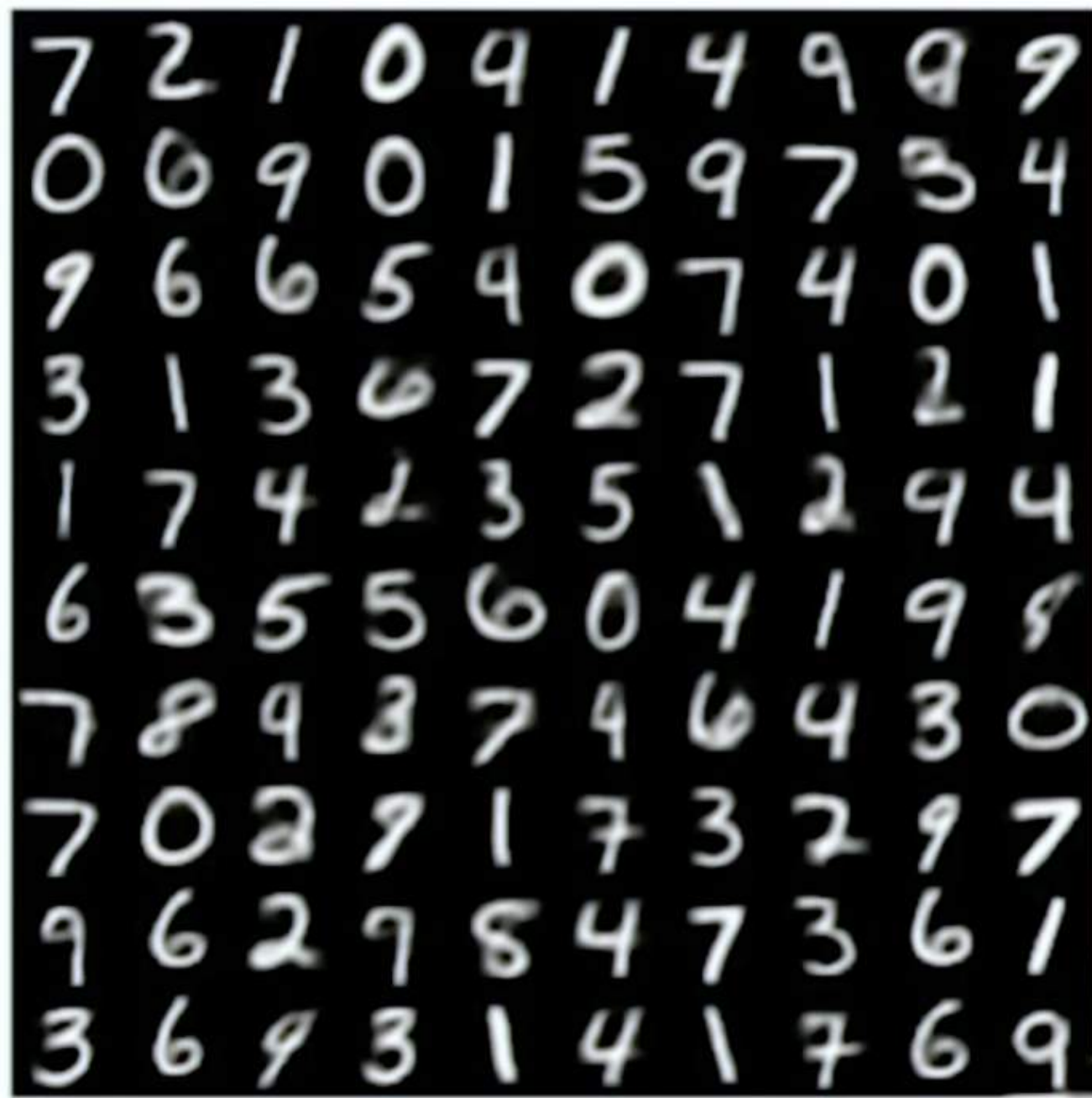
Dimensionality of latent space → reconstruction quality

Autoencoding is a form of compression!
Smaller latent space will force a larger training bottleneck

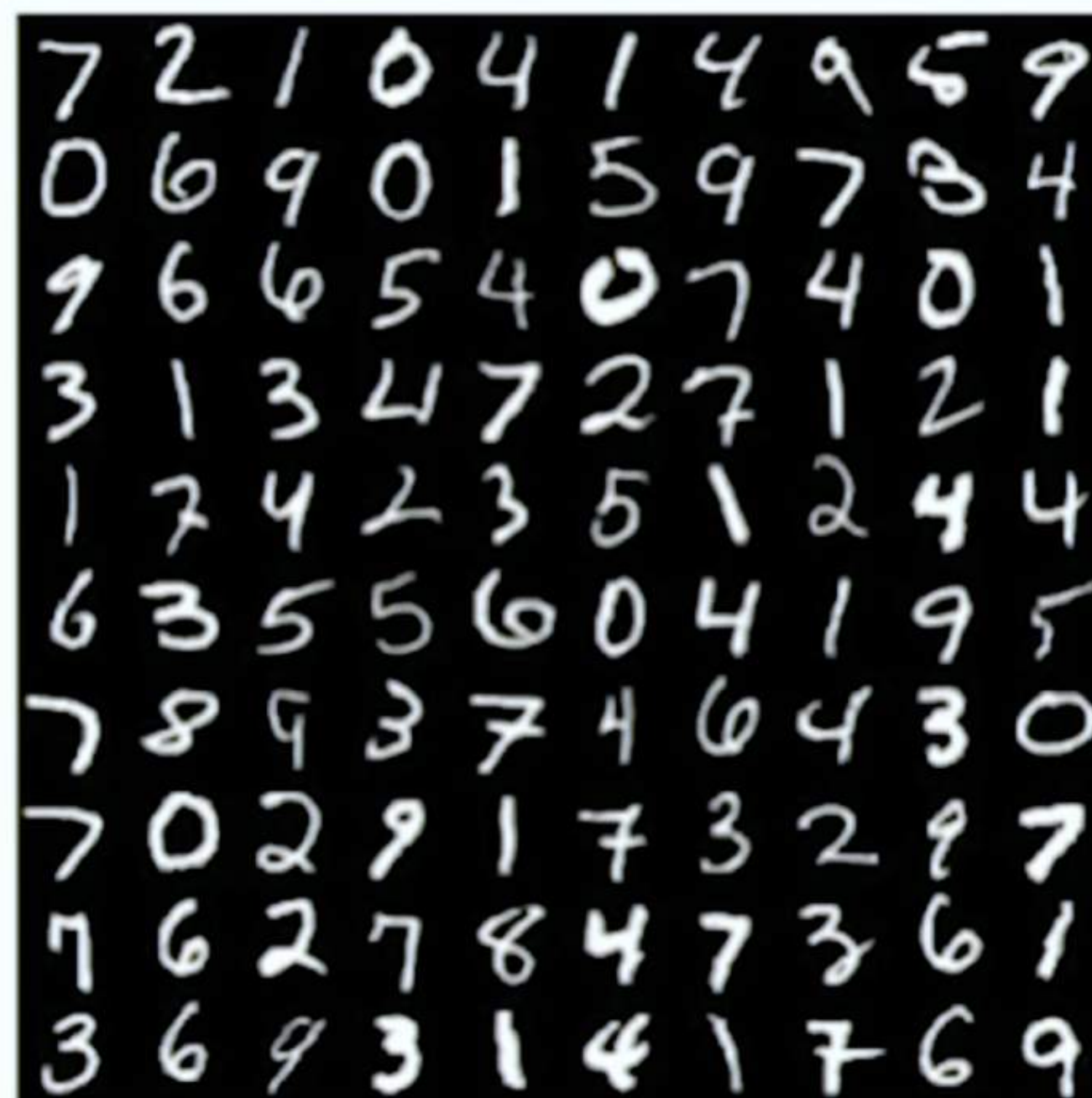
2D latent space



5D latent space



Ground Truth



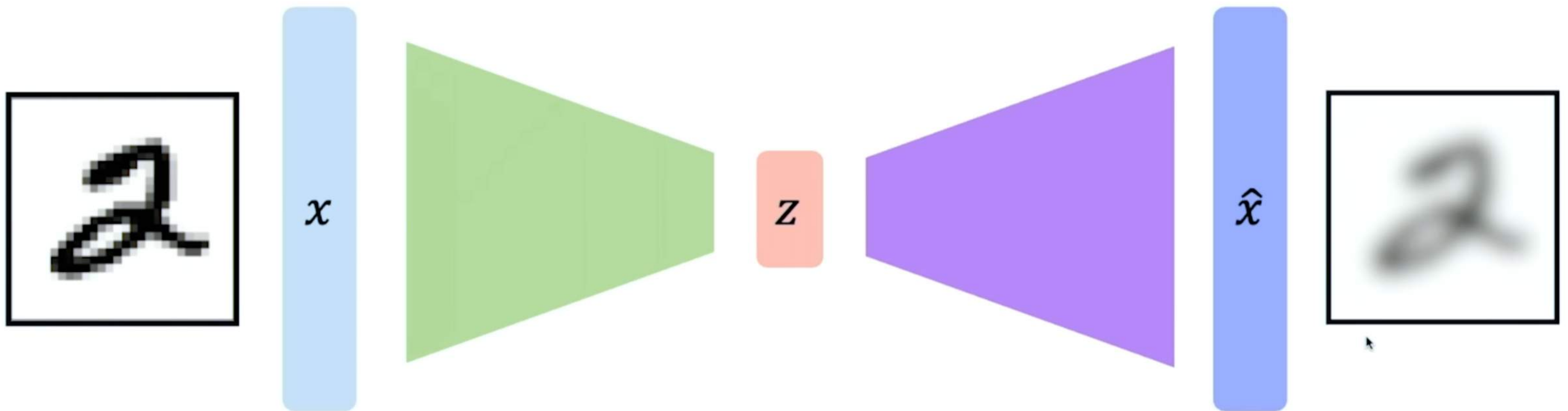
Autoencoders for representation learning

Bottleneck hidden layer forces network to learn a compressed latent representation

Reconstruction loss forces the latent representation to capture (or encode) as much “information” about the data as possible

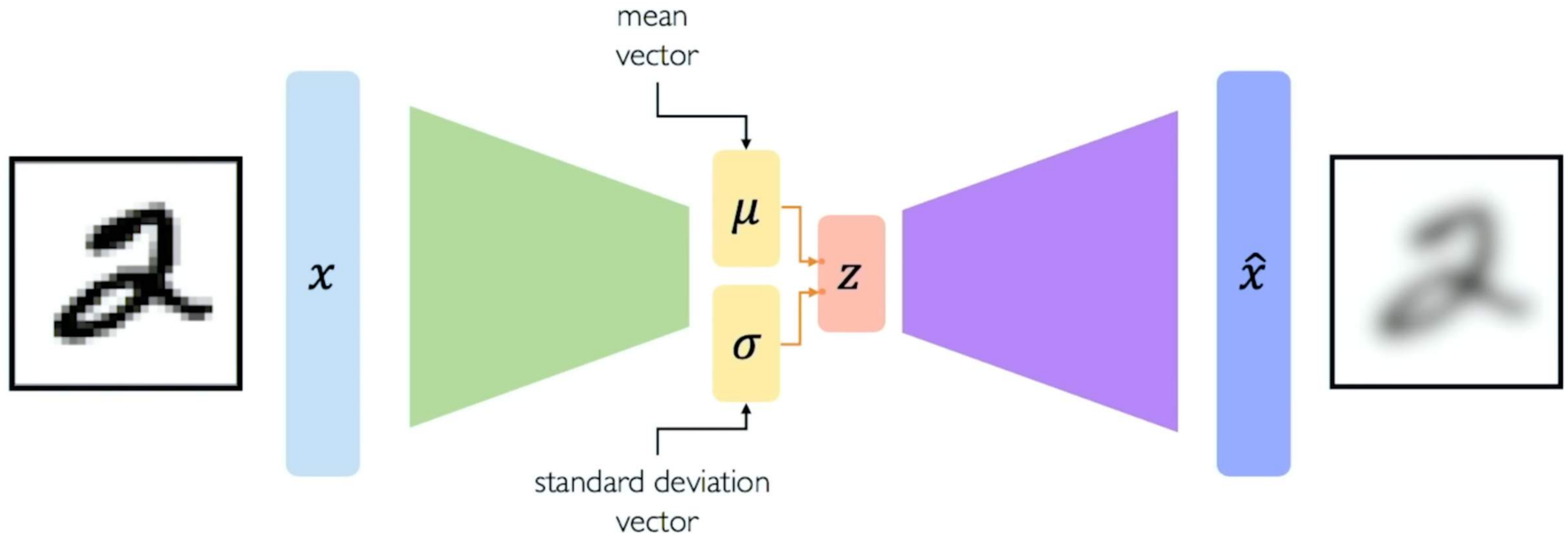
Autoencoding = **Auto** automatically **encoding** data; “Auto” = **self**-encoding

Traditional autoencoders



- z is like any other convolutional layer
- it is deterministic, meaning the same z would always generate the same output from the decoder
- this property is not desirable for generating new data

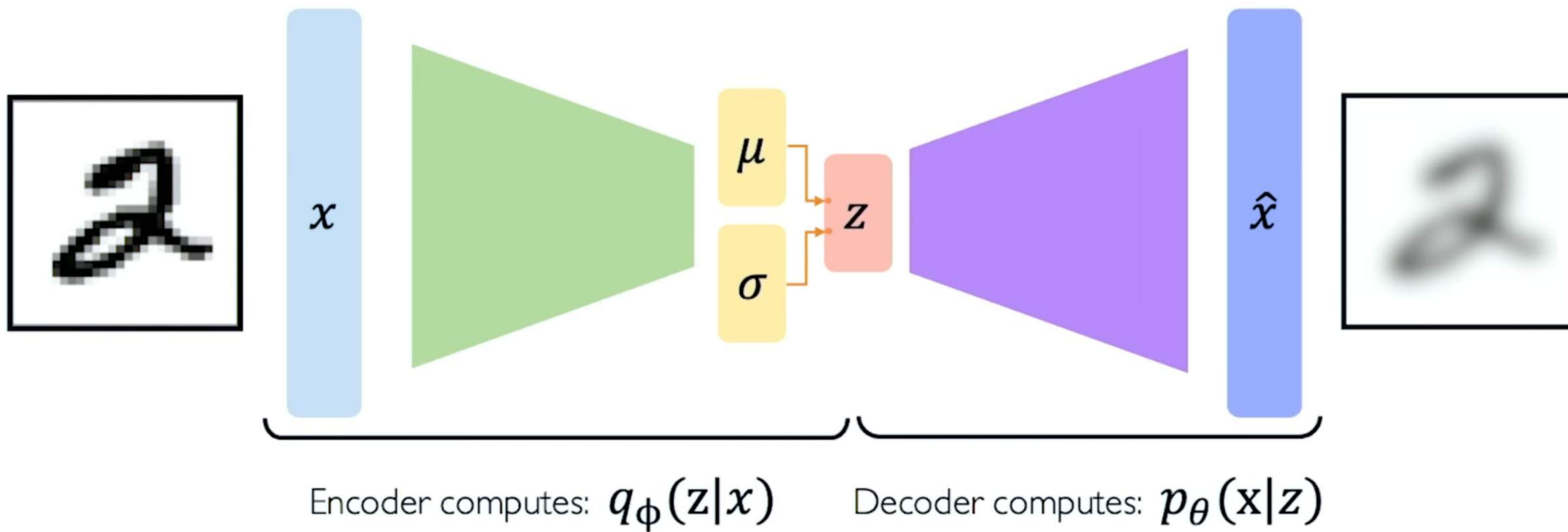
VAEs: key difference with traditional autoencoder



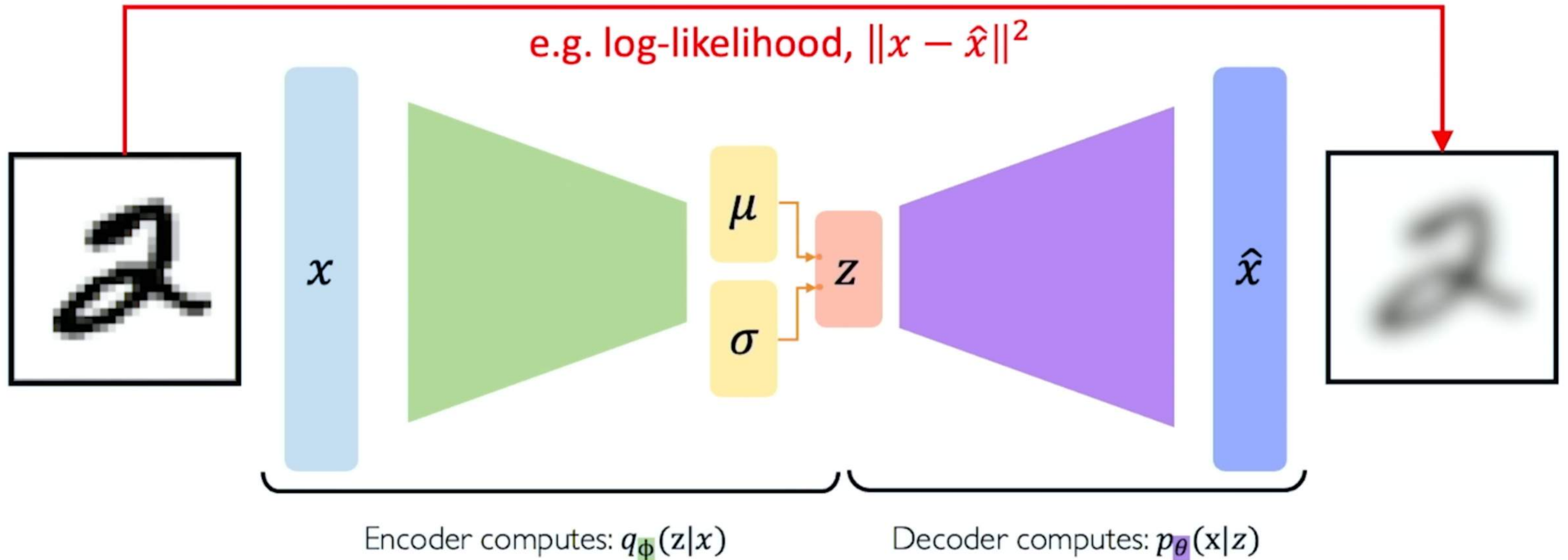
Variational autoencoders are a probabilistic twist on autoencoders!

Sample from the mean and standard deviation to compute latent sample

VAE optimization

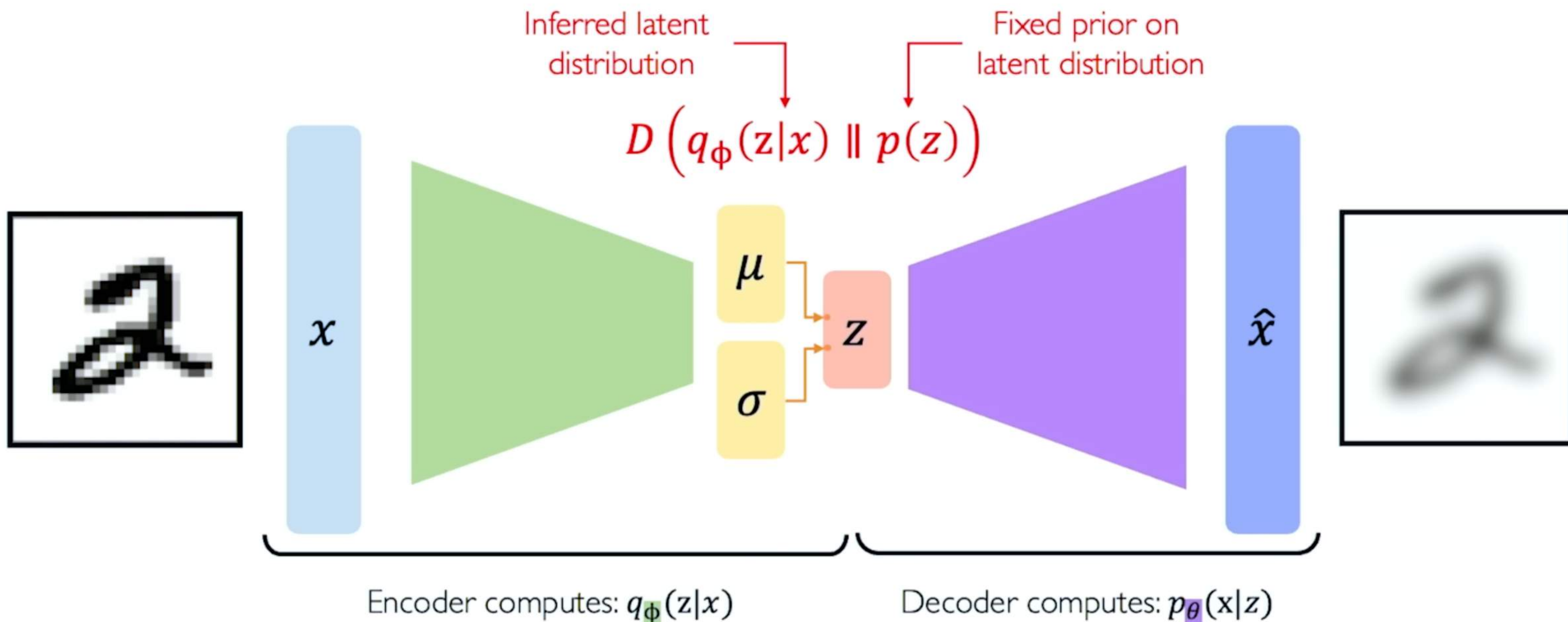


VAE optimization



$$\mathcal{L}(\phi, \theta, x) = \text{(reconstruction loss)} + \text{(regularization term)}$$

VAE optimization



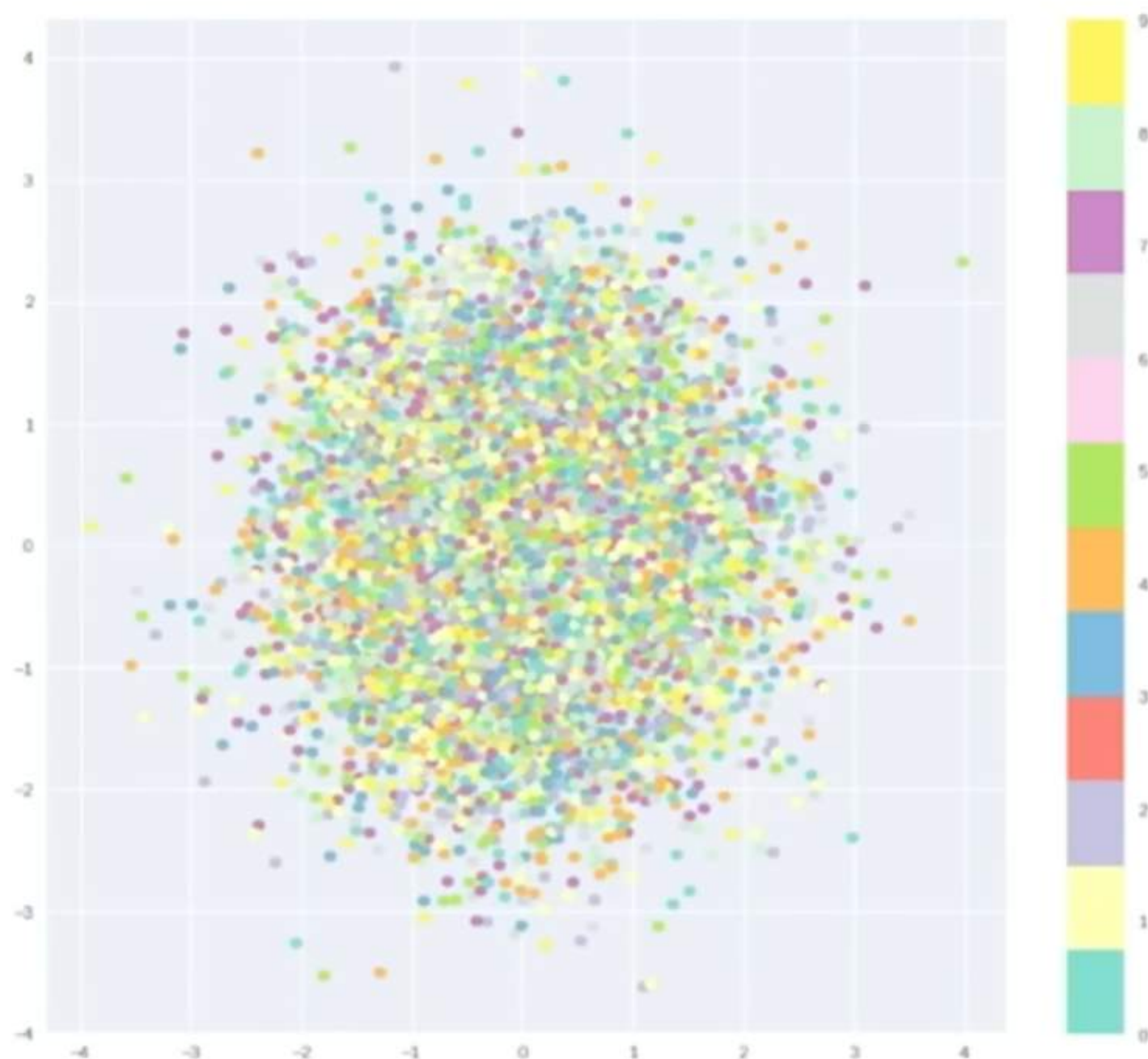
$$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + (\text{regularization term})$$

Priors on the latent distribution

$$D \left(q_{\phi}(z|x) \parallel p(z) \right)$$

Inferred latent
distribution

Fixed prior on
latent distribution



Common choice of prior – Normal Gaussian:

$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

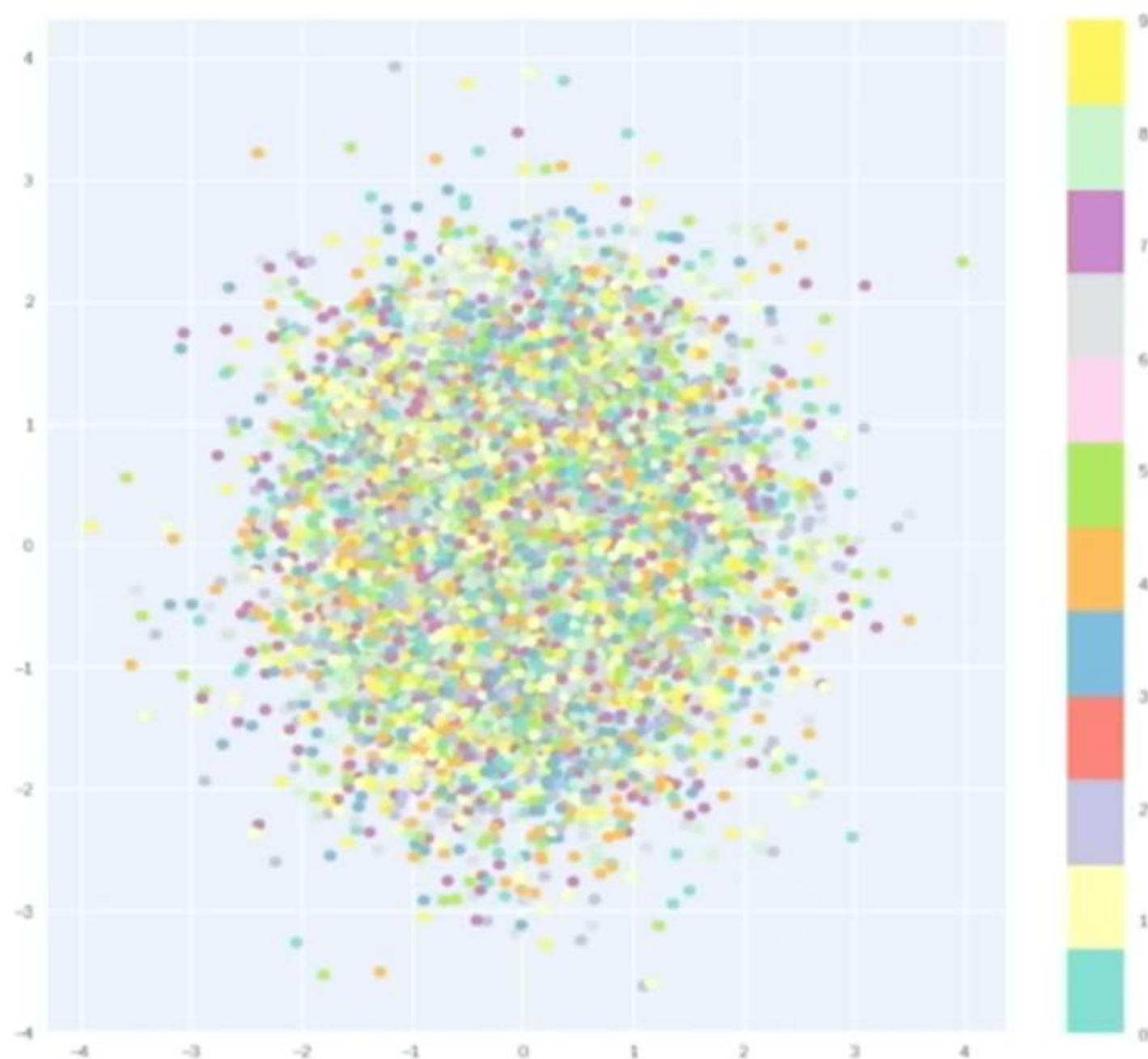
- Encourages encodings to distribute evenly around the center of the latent space
- Penalize the network when it tries to “cheat” by clustering points in specific regions (i.e., by memorizing the data)

Priors on the latent distribution

$$D \left(q_{\phi}(z|x) \parallel p(z) \right)$$

$$= -\frac{1}{2} \sum_{j=0}^{k-1} (\sigma_j + \mu_j^2 - 1 - \log \sigma_j)$$

KL-divergence
between the two
distributions



Common choice of prior – Normal Gaussian:

$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

- Encourages encodings to distribute evenly around the center of the latent space
- Penalize the network when it tries to “cheat” by clustering points in specific regions (i.e., by memorizing the data)

Intuition on regularization and the Normal prior

What properties do we want to achieve from regularization? 🤔

Intuition on regularization and the Normal prior

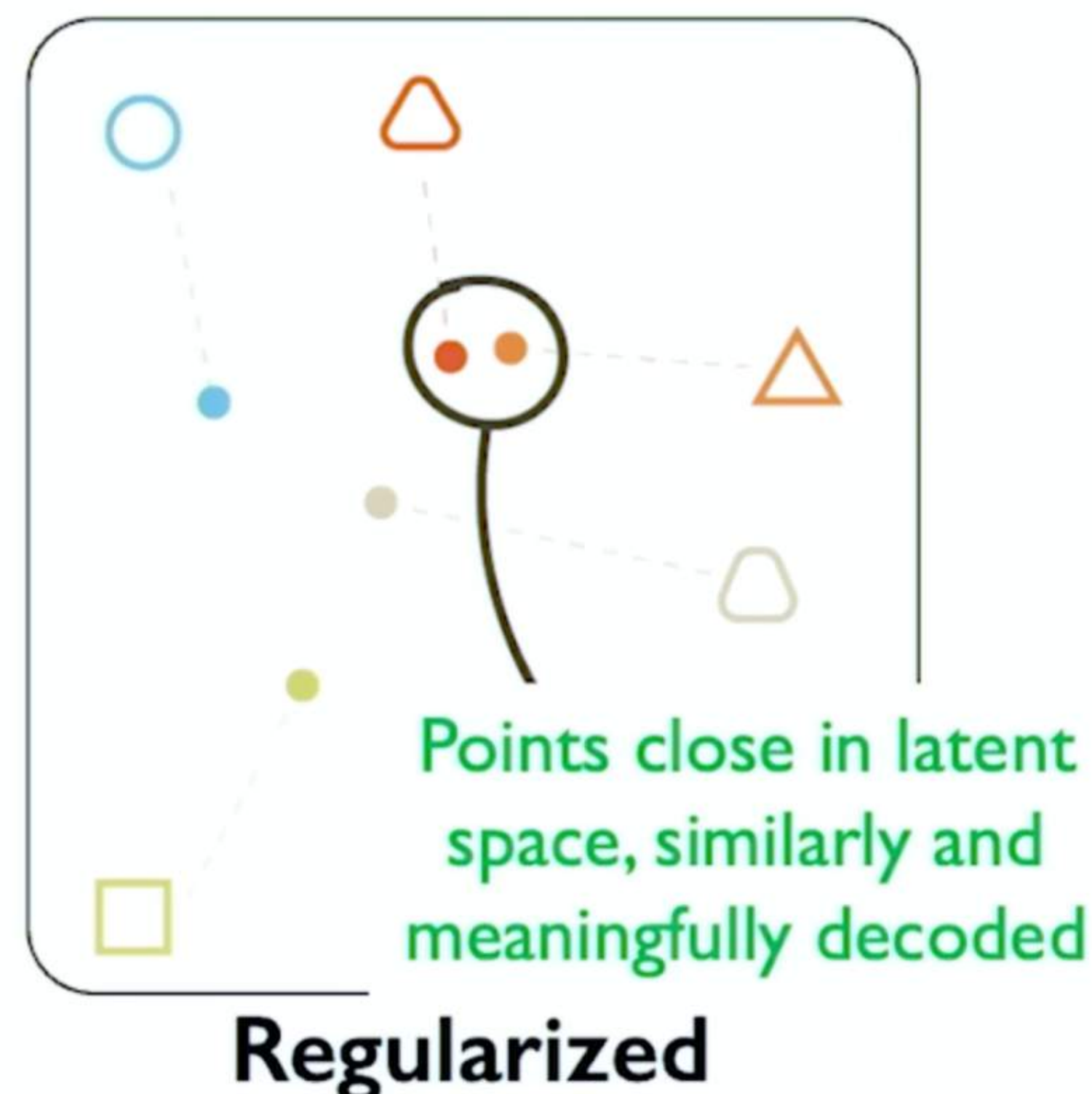
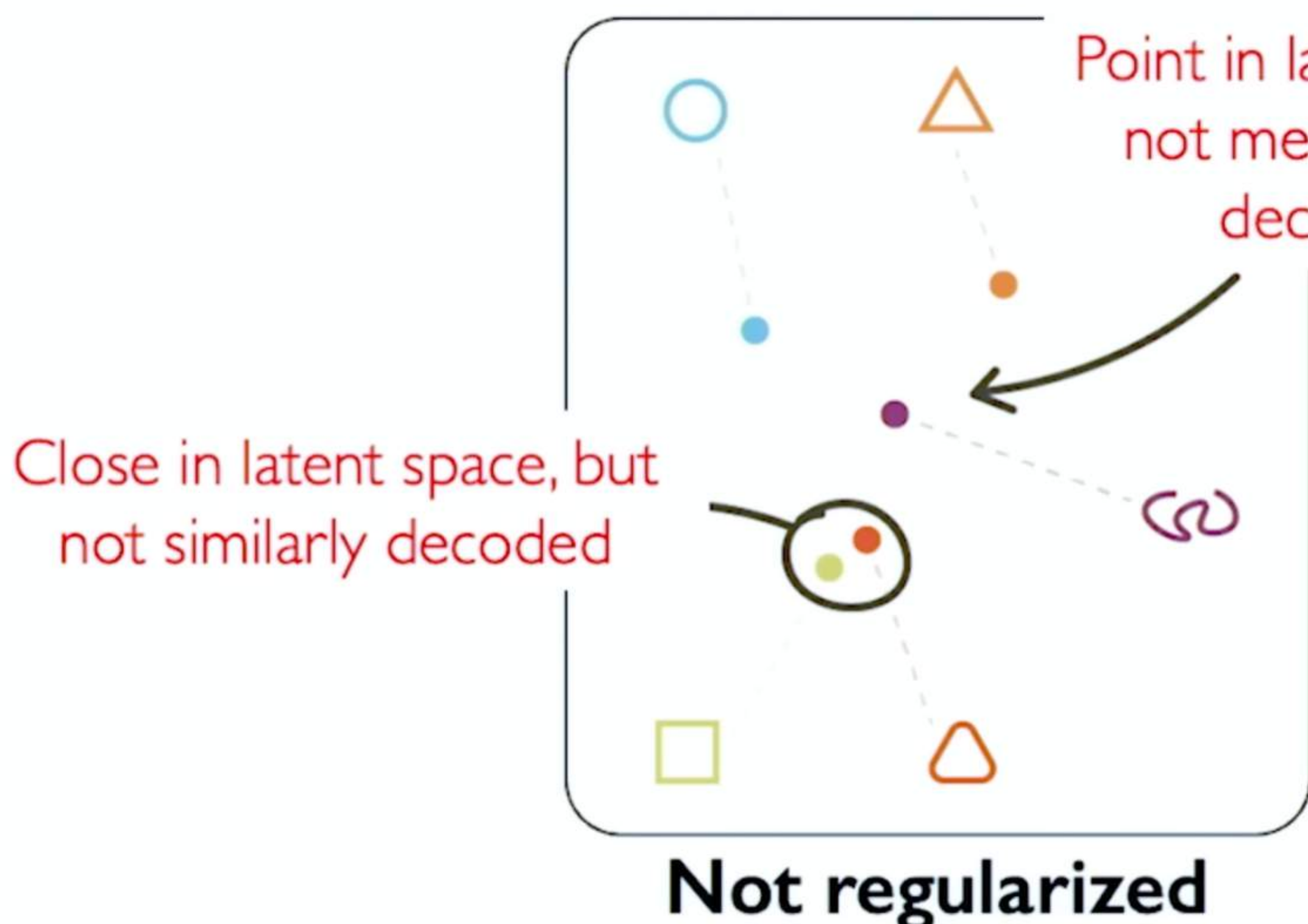
What properties do we want to achieve from regularization? 🤔

1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ “meaningful” content after decoding

Intuition on regularization and the Normal prior

What properties do we want to achieve from regularization? 🤔

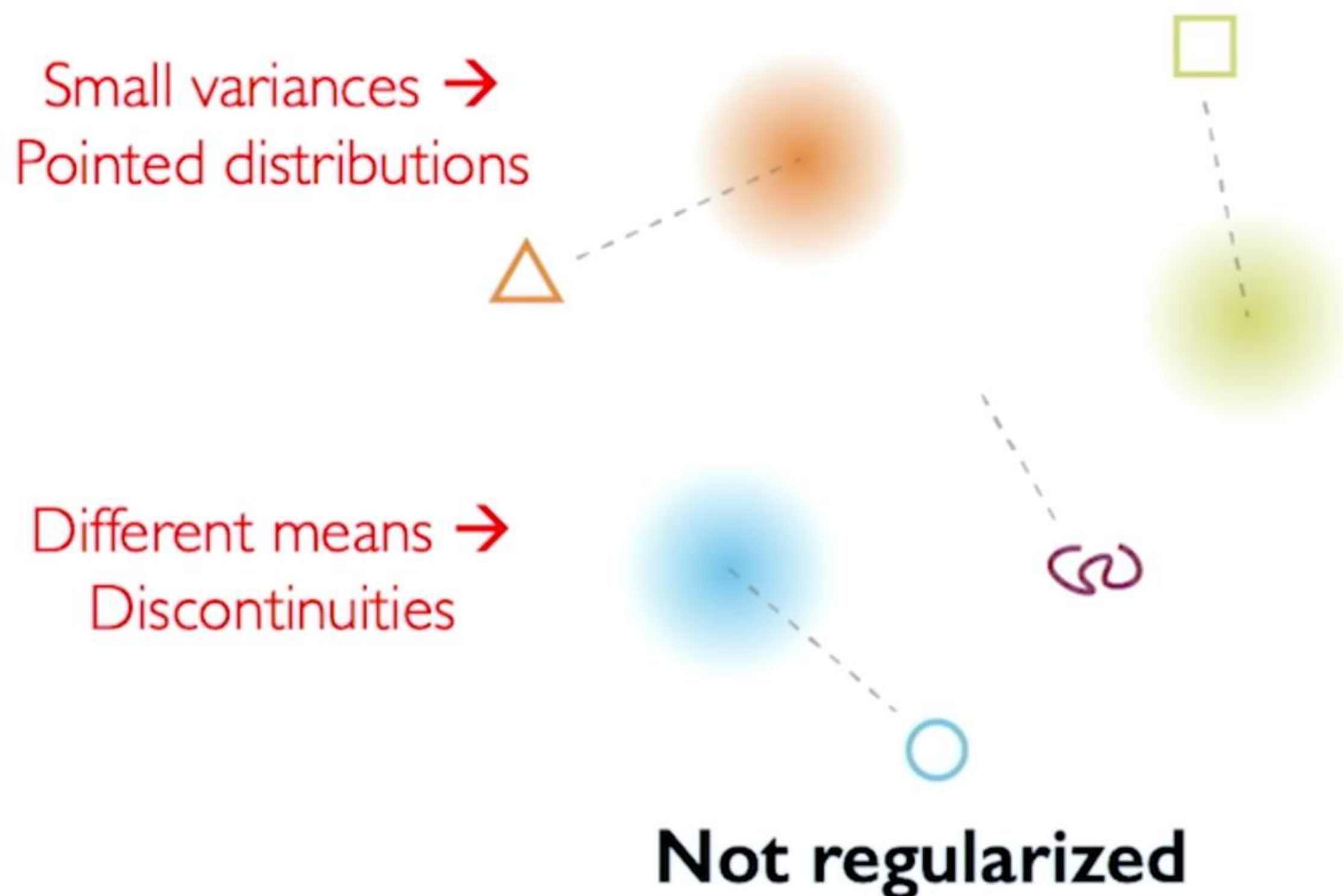
1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ “meaningful” content after decoding



Intuition on regularization and the Normal prior

1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ “meaningful” content after decoding

Encoding as a distribution does not guarantee these properties!



Intuition on regularization and the Normal prior

1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ “meaningful” content after decoding

Encoding as a distribution does not
guarantee these properties!

Small variances $\rightarrow$
Pointed distributions

Different means $\rightarrow$
Discontinuities

Not regularized

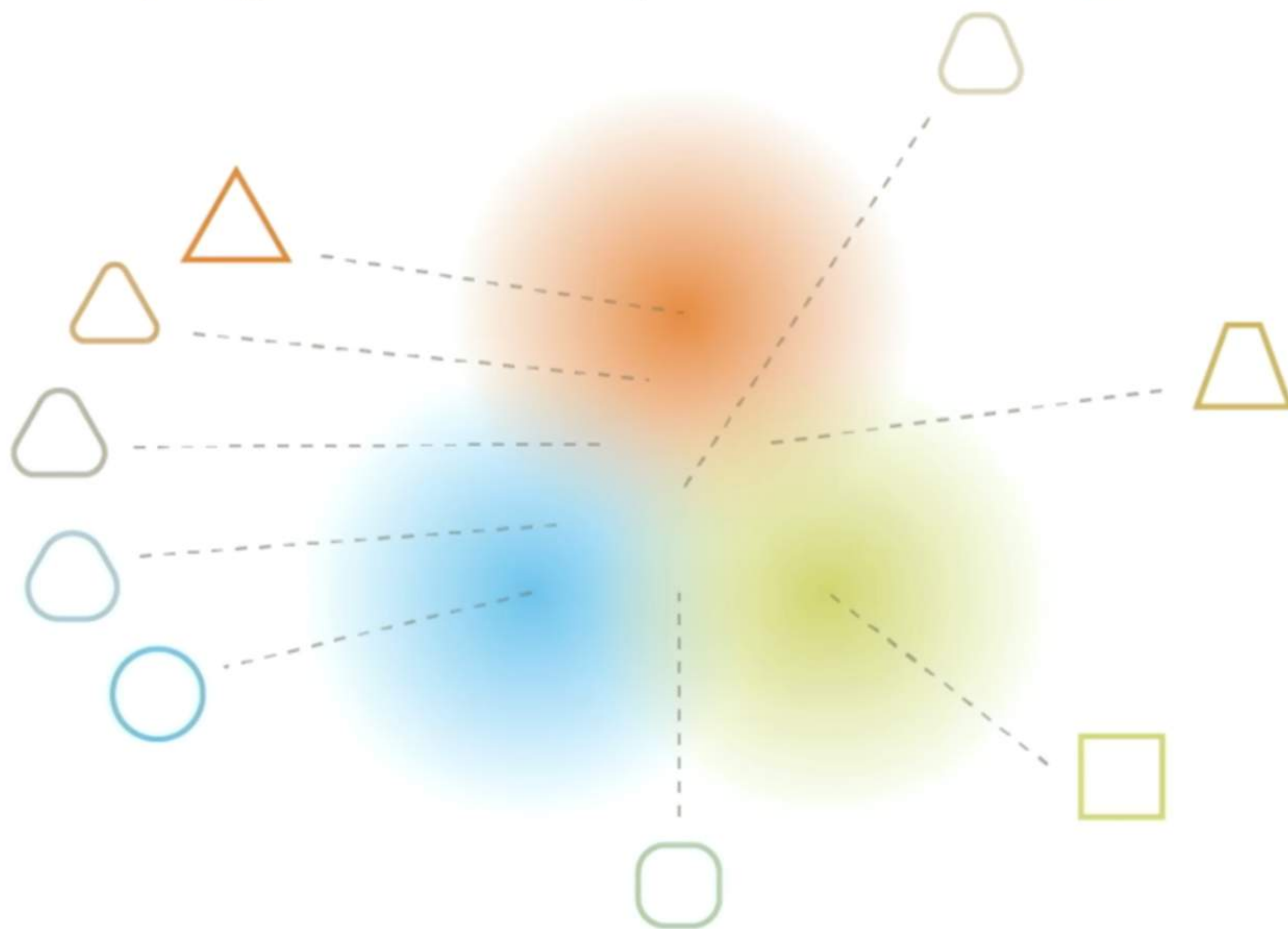
Normal prior $\rightarrow$
continuity + completeness



Regularized

Intuition on regularization and the Normal prior

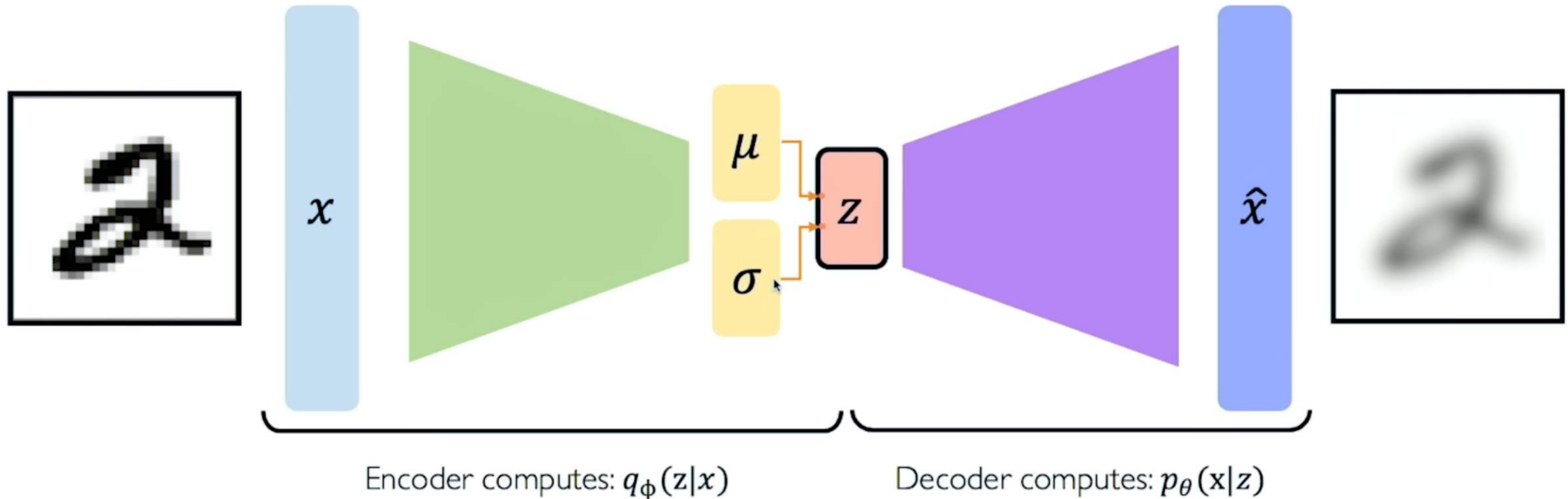
1. **Continuity**: points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness**: sampling from latent space $\rightarrow$ “meaningful” content after decoding



Regularization with Normal prior helps enforce **information gradient** in the latent space.

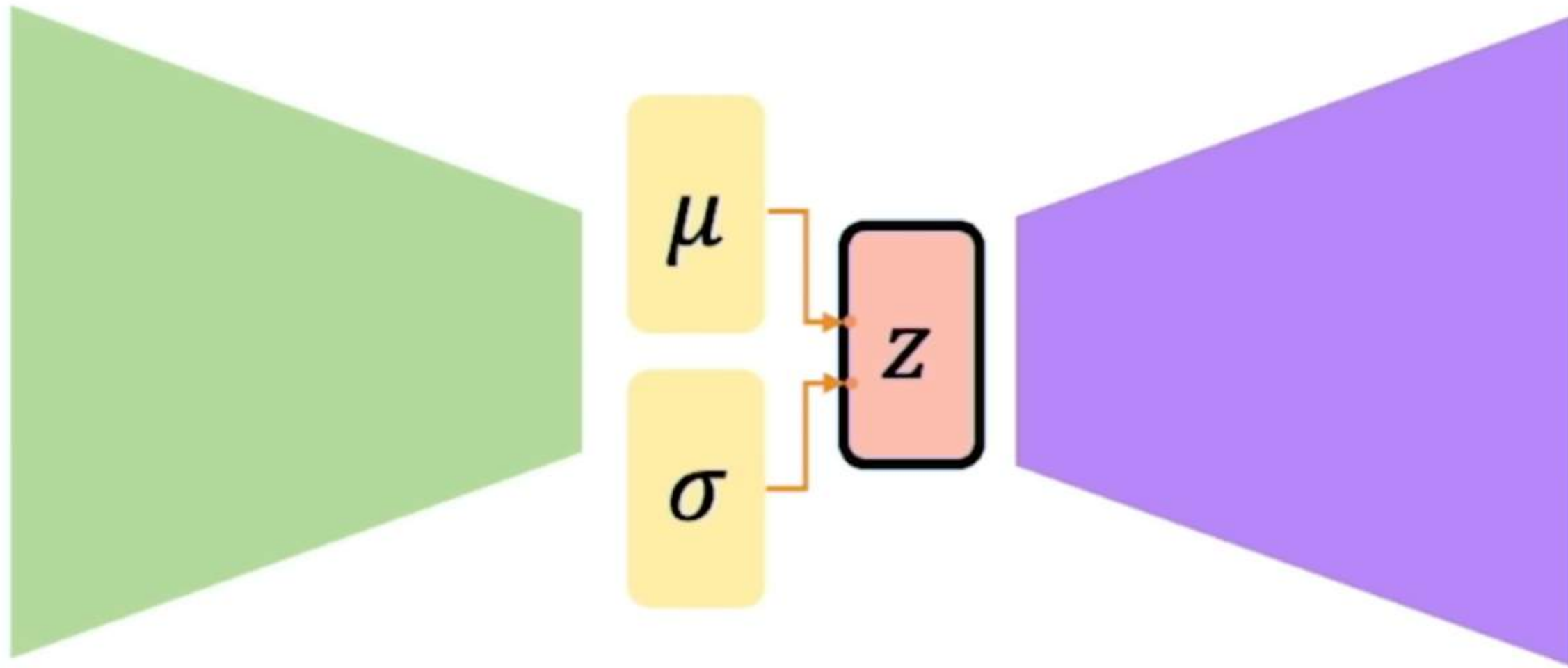
VAE computation graph

Problem: We cannot backpropagate gradients through sampling layers!



$$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + (\text{regularization term})$$

Reparametrizing the sampling layer



Key Idea:

~~$z \sim \mathcal{N}(\mu, \sigma^2)$~~

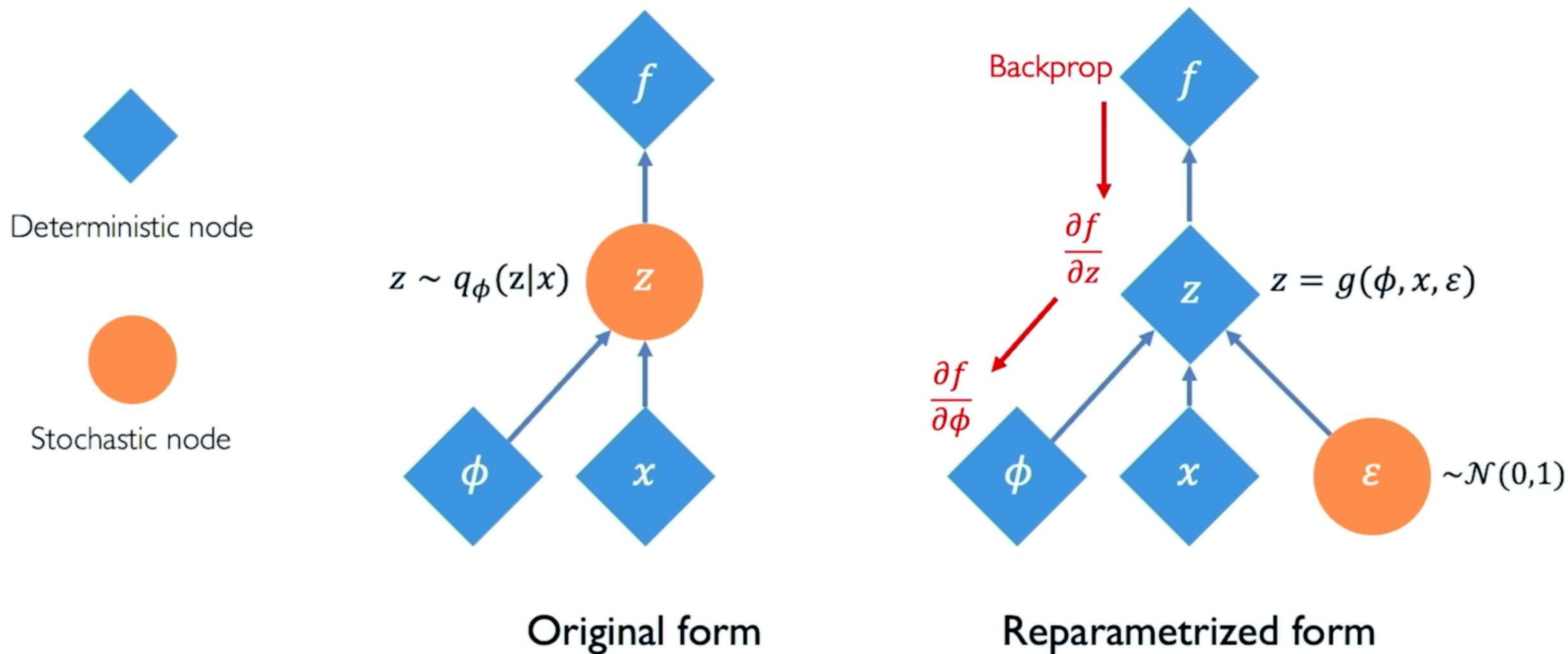
Consider the sampled latent vector z as a sum of

- a fixed μ vector,
- and fixed σ vector, scaled by random constants drawn from the prior distribution

$$\Rightarrow z = \mu + \sigma \odot \varepsilon$$

where $\varepsilon \sim \mathcal{N}(0,1)$

Reparametrizing the sampling layer



VAEs: Latent perturbation

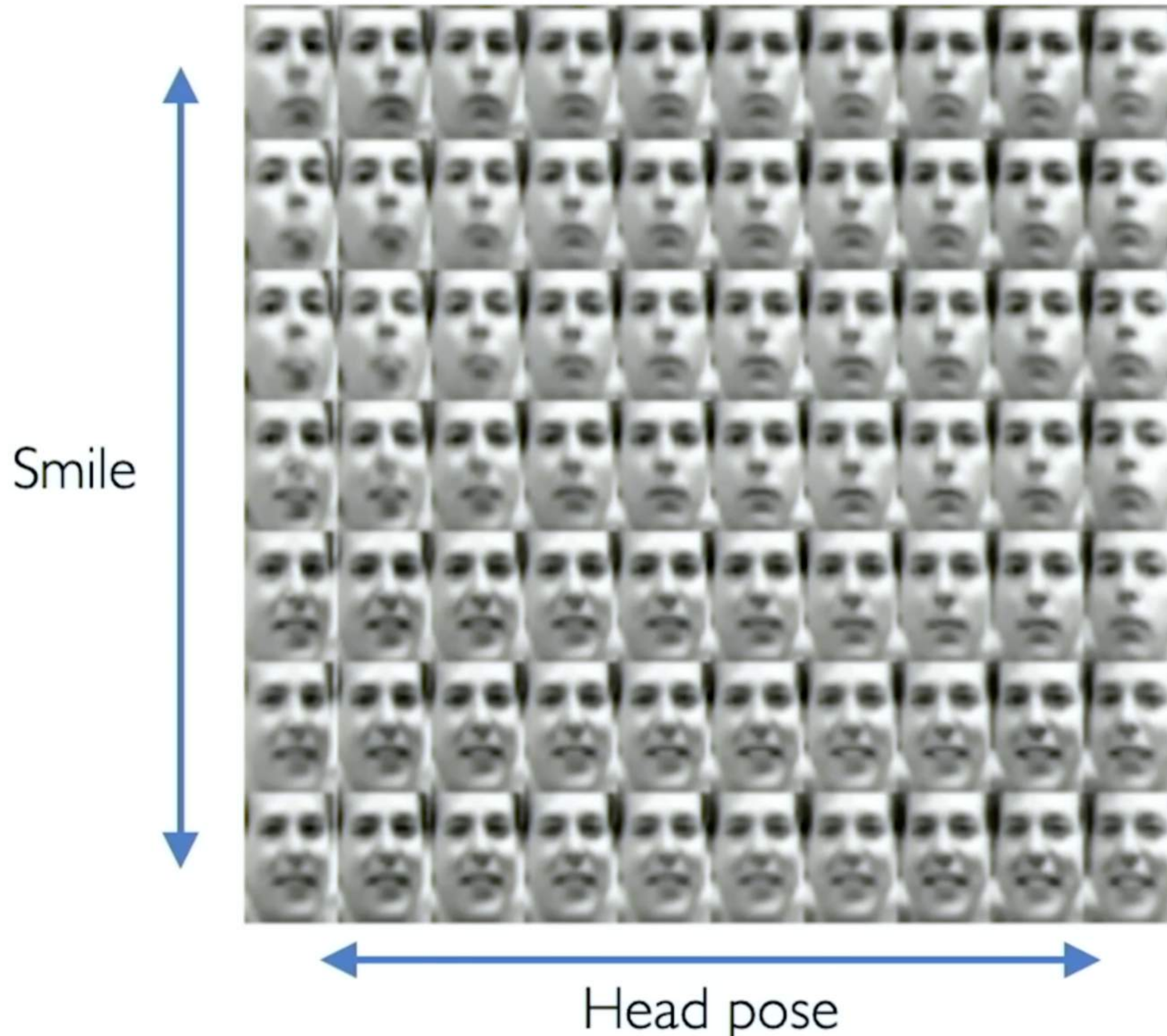
Slowly increase or decrease a **single latent variable**
Keep all other variables fixed



←—————→
Head pose

Different dimensions of z encodes **different interpretable latent features**

VAEs: Latent perturbation



Ideally, we want latent variables that are uncorrelated with each other

Enforce diagonal prior on the latent variables to encourage independence

Disentanglement

Latent space disentanglement with β -VAEs

β -VAE loss:

$$\mathcal{L}(\theta, \phi; \mathbf{x}, \mathbf{z}, \beta) = \underbrace{\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction term}} - \underbrace{\beta D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))}_{\text{Regularization term}}$$

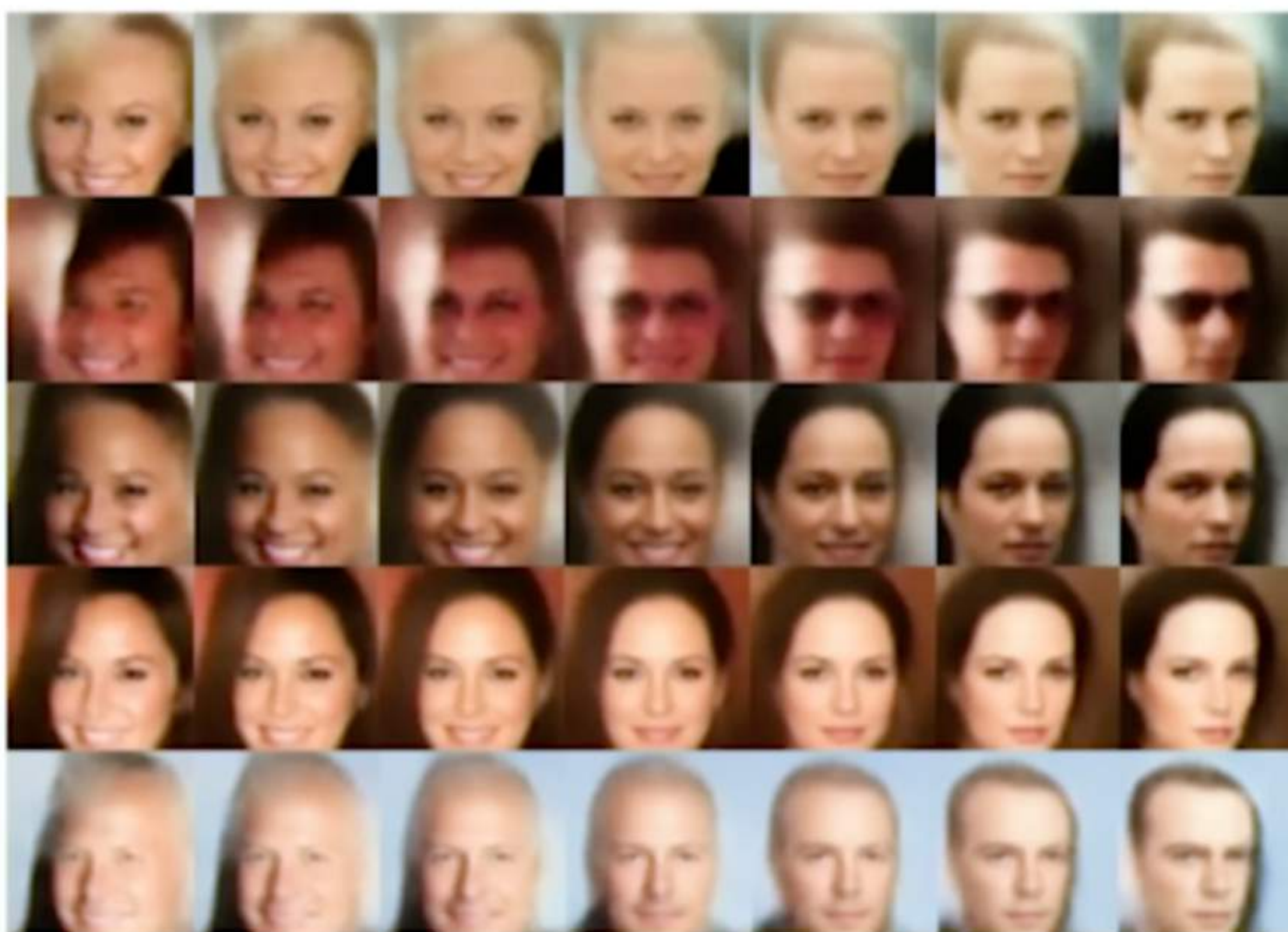
Reconstruction term

Regularization term

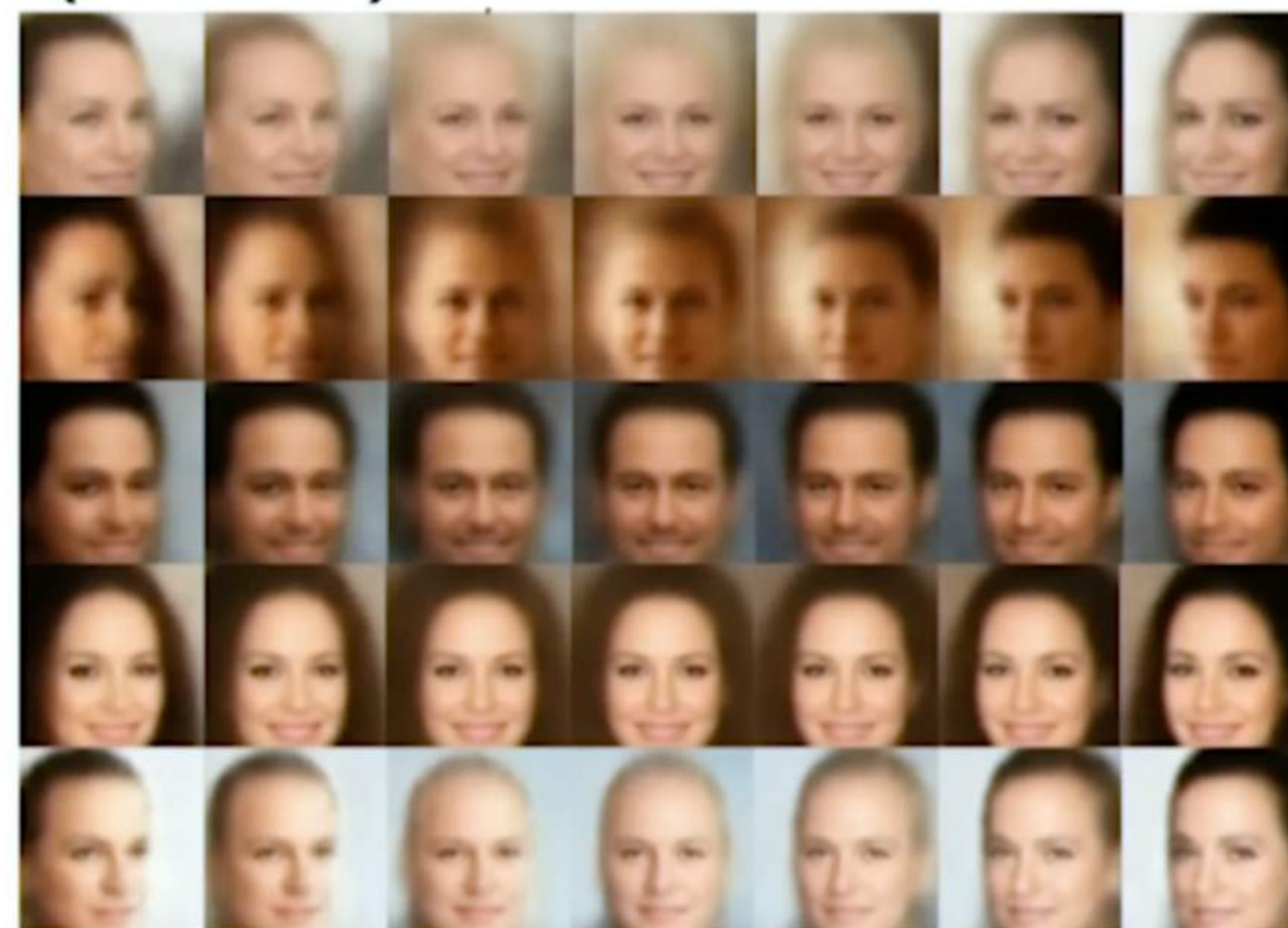
$\beta > 1$: constrain latent bottleneck, encourage efficient latent encoding $\rightarrow$ disentanglement

Head rotation (azimuth)

Smile also
changing!



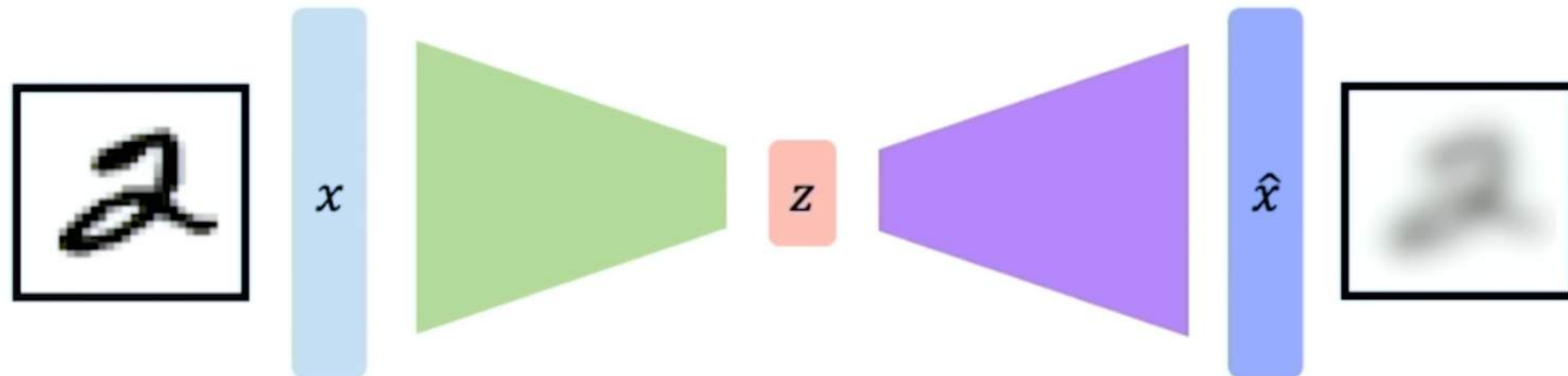
Standard VAE ($\beta = 1$)



β -VAE ($\beta = 250$)

VAE summary

1. Compress representation of world to something we can use to learn
2. Reconstruction allows for unsupervised learning (no labels!)
3. Reparameterization trick to train end-to-end
4. Interpret hidden latent variables using perturbation
5. Generating new examples



End